

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn import preprocessing, model_selection, metrics
from plotly.offline import init_notebook_mode, iplot
init_notebook_mode(connected=True)
import plotly.graph_objs as go
import warnings
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.cluster import DBSCAN
import plotly.express as px
```

```
In [2]: # Import data into dataframe
df = pd.read_csv('energydata_complete.csv')
```

```
In [3]: # Display the first few rows of the DataFrame
print(df.head())
```

	date	Appliances	lights	T1	RH_1	T2	RH_2	\
0	2016-01-11 17:00:00	60	30	19.89	47.596667	19.2	44.790000	
1	2016-01-11 17:10:00	60	30	19.89	46.693333	19.2	44.722500	
2	2016-01-11 17:20:00	50	30	19.89	46.300000	19.2	44.626667	
3	2016-01-11 17:30:00	50	40	19.89	46.066667	19.2	44.590000	
4	2016-01-11 17:40:00	60	40	19.89	46.333333	19.2	44.530000	

	T3	RH_3	T4	...	T9	RH_9	T_out	Press_mm_hg	\
0	19.79	44.730000	19.000000	...	17.033333	45.53	6.600000	733.5	
1	19.79	44.790000	19.000000	...	17.066667	45.56	6.483333	733.6	
2	19.79	44.933333	18.926667	...	17.000000	45.50	6.366667	733.7	
3	19.79	45.000000	18.890000	...	17.000000	45.40	6.250000	733.8	
4	19.79	45.000000	18.890000	...	17.000000	45.40	6.133333	733.9	

	RH_out	Windspeed	Visibility	Tdewpoint	rv1	rv2
0	92.0	7.000000	63.000000	5.3	13.275433	13.275433
1	92.0	6.666667	59.166667	5.2	18.606195	18.606195
2	92.0	6.333333	55.333333	5.1	28.642668	28.642668
3	92.0	6.000000	51.500000	5.0	45.410389	45.410389
4	92.0	5.666667	47.666667	4.9	10.084097	10.084097

[5 rows x 29 columns]

```
In [4]: # 1. Identifying Unique Values
# -----
# For each column, print the unique values
for col in df.columns:
    print(f"Unique values in '{col}':\n{df[col].unique()[:10]}") # Show first 10 unique values if too many
```

```

Unique values in 'date':
['2016-01-11 17:00:00' '2016-01-11 17:10:00' '2016-01-11 17:20:00'
 '2016-01-11 17:30:00' '2016-01-11 17:40:00' '2016-01-11 17:50:00'
 '2016-01-11 18:00:00' '2016-01-11 18:10:00' '2016-01-11 18:20:00'
 '2016-01-11 18:30:00']
Unique values in 'Appliances':
[ 60  50  70 230 580 430 250 100  90  80]
Unique values in 'lights':
[30 40 50 70 60 10 20  0]
Unique values in 'T1':
[19.89      19.85666667 19.79      19.92666667 20.06666667 20.13333333
 20.26      20.42666667 20.56666667 20.73      ]
Unique values in 'RH_1':
[47.59666667 46.69333333 46.3      46.06666667 46.33333333 46.02666667
 45.76666667 45.56      45.5975    46.09      ]
Unique values in 'T2':
[19.2      19.23      19.35666667 19.42666667 19.56666667 19.73
 19.85666667 20.03333333 20.16666667 20.2      ]
Unique values in 'RH_2':
[44.79      44.7225    44.62666667 44.59      44.53      44.5
 44.43333333 44.4      45.1      45.83333333]
Unique values in 'T3':
[19.79      19.73      19.89      20.03333333 20.1      20.2
 20.16666667 20.26      20.29      20.39      ]
Unique values in 'RH_3':
[44.73      44.79      44.93333333 45.      44.9      44.86333333
 44.82666667 45.49333333 47.52666667 48.46666667]
Unique values in 'T4':
[19.      18.92666667 18.89      18.96333333 19.03333333 19.175
 19.26      19.32333333 19.39      19.42666667]
Unique values in 'RH_4':
[45.56666667 45.9925    45.89      45.72333333 45.53      45.73
 45.79      45.86333333 46.09666667 46.43      ]
Unique values in 'T5':
[17.16666667 17.2      17.13333333 17.1      17.15      17.59333333
 18.06666667 17.66666667 17.6      17.56666667]
Unique values in 'RH_5':
[55.2      55.09      55.03      54.96666667 54.9      55.
 55.16333333 55.5      56.0425    56.49      ]
Unique values in 'T6':
[7.02666667 6.83333333 6.56      6.43333333 6.36666667 6.3
 6.26333333 6.19      6.12333333 6.0675     ]
Unique values in 'RH_6':
[84.25666667 84.06333333 83.15666667 83.42333333 84.89333333 85.76666667

```

```

86.09      86.42333333 87.22666667 87.62666667]
Unique values in 'T7':
[17.2      17.13333333 17.1      17.16666667 17.2475      17.53
 17.82333333 17.96333333 17.89      17.76      ]
Unique values in 'RH_7':
[41.62666667 41.56      41.43333333 41.29      41.23      41.26
 41.2      41.4      41.5      42.7175      ]
Unique values in 'T8':
[18.2      18.1      18.06666667 18.03333333 18.15      18.23
 18.35666667 18.53333333 18.66666667 18.73      ]
Unique values in 'RH_8':
[48.9      48.86333333 48.73      48.59      48.63333333 48.56
 48.66666667 49.19333333 49.2      49.63333333]
Unique values in 'T9':
[17.03333333 17.06666667 17.      16.89      16.96333333 16.82333333
 17.1      17.13333333 17.2      17.15      ]
Unique values in 'RH_9':
[45.53      45.56      45.5      45.4      45.29      45.32666667
 45.345      45.2      45.12666667 45.06      ]
Unique values in 'T_out':
[6.6      6.48333333 6.36666667 6.25      6.13333333 6.01666667
 5.9      5.91666667 5.93333333 5.95      ]
Unique values in 'Press_mm_hg':
[733.5      733.6      733.7      733.8      733.9
 734.      734.1      734.16666667 734.23333333 734.3      ]
Unique values in 'RH_out':
[92.      91.83333333 91.66666667 91.5      91.33333333 91.16666667
 91.      90.5      90.      89.5      ]
Unique values in 'Windspeed':
[7.      6.66666667 6.33333333 6.      5.66666667 5.33333333
 5.      5.16666667 5.5      5.83333333]
Unique values in 'Visibility':
[63.      59.16666667 55.33333333 51.5      47.66666667 43.83333333
 40.      37.      34.      31.      ]
Unique values in 'Tdewpoint':
[5.3      5.2      5.1      5.      4.9      4.8
 4.7      4.68333333 4.66666667 4.65      ]
Unique values in 'rv1':
[13.27543316 18.60619498 28.64266817 45.4103895 10.08409655 44.91948425
 47.23376343 33.03988962 31.45570219 3.08931352]
Unique values in 'rv2':
[13.27543316 18.60619498 28.64266817 45.4103895 10.08409655 44.91948425
 47.23376343 33.03988962 31.45570219 3.08931352]

```



```
In [5]: # 2. Counting Unique Values
# -----
# Use nunique to count the number of unique values in each column
unique_counts = df.nunique()
print("\nNumber of unique values in each column:")
print(unique_counts)
```

Number of unique values in each column:

date	19735
Appliances	92
lights	8
T1	722
RH_1	2547
T2	1650
RH_2	3376
T3	1426
RH_3	2618
T4	1390
RH_4	2987
T5	2263
RH_5	7571
T6	4446
RH_6	9709
T7	1955
RH_7	5891
T8	2228
RH_8	6649
T9	924
RH_9	3388
T_out	1730
Press_mm_hg	2189
RH_out	566
Windspeed	189
Visibility	413
Tdewpoint	1409
rv1	19735
rv2	19735

dtype: int64

```
In [103]: import missingno as mno
pd.DataFrame(data.isna().sum(), columns=["records missing"]).T
mno.matrix(data, figsize = (20, 20))
```

Out[103]: <Axes: >

1

date Appliances
lights T1 RH 1 T2 RH 2 T3 RH 3 T4 RH 4 T5 RH 5 T6 RH 6 T7 RH 7 T8 RH 8 T9 RH 9 T out
Press_mm_hg RH_out Windspeed Visibility Tdewpoint rv1 rv2 month weekday hour week
week of month cluster day_Friday day_Monday day_Saturday day_Sunday day_Thursday day_Tuesday
day_Wednesday outlier_flag

This figure shows a large, solid black rectangular area representing the main body of the dataset. A small '1' is located in the top-left corner. The x-axis labels are listed along the top edge, including 'date', 'Appliances', 'lights', 'T1', 'RH 1', 'T2', 'RH 2', 'T3', 'RH 3', 'T4', 'RH 4', 'T5', 'RH 5', 'T6', 'RH 6', 'T7', 'RH 7', 'T8', 'RH 8', 'T9', 'RH 9', 'T out', 'Press_mm_hg', 'RH_out', 'Windspeed', 'Visibility', 'Tdewpoint', 'rv1', 'rv2', 'month', 'weekday', 'hour', 'week', 'week of month', 'cluster', 'day_Friday', 'day_Monday', 'day_Saturday', 'day_Sunday', 'day_Thursday', 'day_Tuesday', 'day_Wednesday', and 'outlier_flag'.

19735

43

43

```
In [6]: # 3. Analyzing Categorical Columns
# -----
# If you want to focus on categorical columns only
categorical_columns = df.select_dtypes(include=['object', 'category']).columns
for col in categorical_columns:
    print(f"Unique values in categorical column '{col}': {df[col].unique()}") # Show all unique values

Unique values in categorical column 'date': ['2016-01-11 17:00:00' '2016-01-11 17:10:00' '2016-01-11 17:20:00' ...
'2016-05-27 17:40:00' '2016-05-27 17:50:00' '2016-05-27 18:00:00']
```

```
In [7]: # 4. Identifying Columns with Only One Unique Value (Constant Features)
# -----
constant_columns = [col for col in df.columns if df[col].nunique() == 1]
if constant_columns:
    print(f"\nColumns with only one unique value (constant features): {constant_columns}")
else:
    print("\nNo columns with a single unique value found.")
```

No columns with a single unique value found.

```
In [8]: df.isnull().sum()
```

```
Out [8]: date           0
Appliances           0
lights               0
T1                   0
RH_1                 0
T2                   0
RH_2                 0
T3                   0
RH_3                 0
T4                   0
RH_4                 0
T5                   0
RH_5                 0
T6                   0
RH_6                 0
T7                   0
RH_7                 0
T8                   0
RH_8                 0
T9                   0
RH_9                 0
T_out                0
Press_mm_hg          0
RH_out               0
Windspeed            0
Visibility            0
Tdewpoint            0
rv1                  0
rv2                  0
dtype: int64
```

```
In [9]: df.describe()
```

Out [9]:

	Appliances	lights	T1	RH_1	T2	RH_2	T3	RH_3	T4	
count	19735.000000	19735.000000	19735.000000	19735.000000	19735.000000	19735.000000	19735.000000	19735.000000	19735.000000	19735.000000
mean	97.694958	3.801875	21.686571	40.259739	20.341219	40.420420	22.267611	39.242500	20.855335	39.242500
std	102.524891	7.935988	1.606066	3.979299	2.192974	4.069813	2.006111	3.254576	2.042884	3.254576
min	10.000000	0.000000	16.790000	27.023333	16.100000	20.463333	17.200000	28.766667	15.100000	16.790000
25%	50.000000	0.000000	20.760000	37.333333	18.790000	37.900000	20.790000	36.900000	19.530000	20.760000
50%	60.000000	0.000000	21.600000	39.656667	20.000000	40.500000	22.100000	38.530000	20.666667	21.600000
75%	100.000000	0.000000	22.600000	43.066667	21.500000	43.260000	23.290000	41.760000	22.100000	22.600000
max	1080.000000	70.000000	26.260000	63.360000	29.856667	56.026667	29.236000	50.163333	26.200000	29.856667

8 rows x 28 columns

```

In [10]: # Convert the date column to datetime format
df['date'] = pd.to_datetime(df['date'])

# Extracting features from the 'date' column
df['month'] = df['date'].apply(lambda x: x.month)           # Month of the year (1-12)
df['weekday'] = df['date'].apply(lambda x: x.weekday())     # Day of the week (0=Monday, 6=Sunday)
df['hour'] = df['date'].apply(lambda x: x.hour)             # Hour of the day (0-23)
df['week'] = df['date'].apply(lambda x: x.isocalendar().week) # Week number of the year
df['day_of_week'] = df['date'].apply(lambda x: x.strftime('%A')) # Day name (e.g., Monday)
df['week_of_month'] = df['date'].apply(lambda x: (x.day-1) // 7 + 1) # Week of the month (1-5)

# Display the updated DataFrame
df.head()

```

Out[10]:

	date	Appliances	lights	T1	RH_1	T2	RH_2	T3	RH_3	T4	...	Visibility	Tdewpoint	rv1	
0	2016-01-11 17:00:00	60	30	19.89	47.596667	19.2	44.790000	19.79	44.730000	19.000000	...	63.000000	5.3	13.275433	13.2754
1	2016-01-11 17:10:00	60	30	19.89	46.693333	19.2	44.722500	19.79	44.790000	19.000000	...	59.166667	5.2	18.606195	18.6061
2	2016-01-11 17:20:00	50	30	19.89	46.300000	19.2	44.626667	19.79	44.933333	18.926667	...	55.333333	5.1	28.642668	28.6426
3	2016-01-11 17:30:00	50	40	19.89	46.066667	19.2	44.590000	19.79	45.000000	18.890000	...	51.500000	5.0	45.410389	45.4103
4	2016-01-11 17:40:00	60	40	19.89	46.333333	19.2	44.530000	19.79	45.000000	18.890000	...	47.666667	4.9	10.084097	10.0840

5 rows × 35 columns

In [11]: *# Dependent variable uniqueness*

df['Appliances'].unique()

Out[11]:

```
array([ 60,  50,  70, 230, 580, 430, 250, 100,  90,  80, 140,
        120, 190, 110, 400, 390, 240,  40,  30, 310, 380, 370,
         20, 260, 500, 450, 220, 170, 290, 130, 200, 210, 180,
        350, 300, 340, 150, 330, 420, 520, 360, 270, 550, 690,
        620, 610, 490, 410, 320, 740, 910, 510, 790, 460, 160,
        480, 280, 560, 1080, 750, 470, 440, 800, 540, 530, 660,
        630, 600,  10, 570, 830, 1070, 890, 590, 850, 710, 680,
        670, 780, 640, 650, 700, 770, 720, 760, 860, 880, 730,
        900, 820, 870, 840], dtype=int64)
```

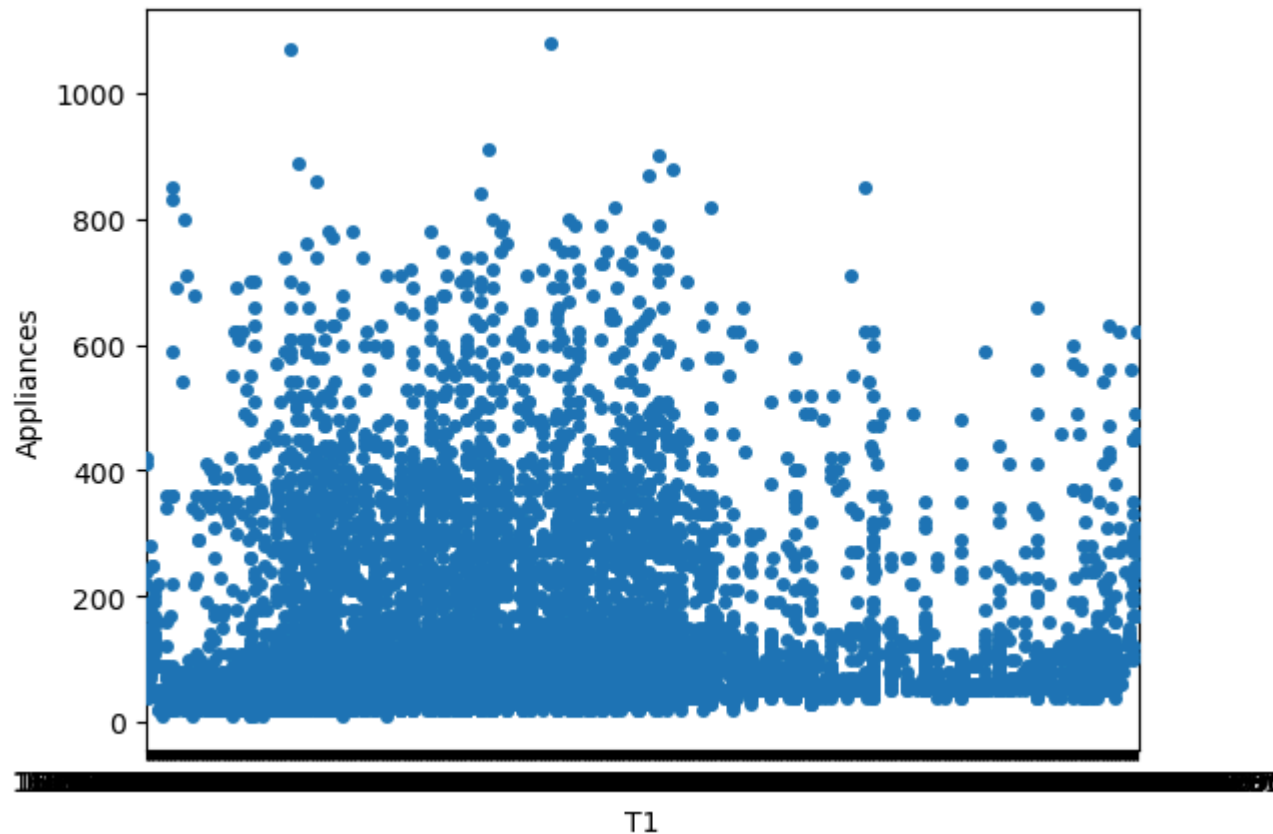
In [12]:

```
import plotly.graph_objects as go
from sklearn.tree import DecisionTreeRegressor
from sklearn.feature_selection import RFE
from sklearn.neighbors import LocalOutlierFactor
from tqdm import tqdm
from colorsys import yiq_to_rgb
from dataclasses import dataclass, field
```

```
import math
from pandas import read_csv
from tqdm import tqdm
warnings.filterwarnings('ignore')
from sklearn.decomposition import PCA
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from sklearn.feature_selection import RFE
from sklearn.linear_model import LogisticRegression
```

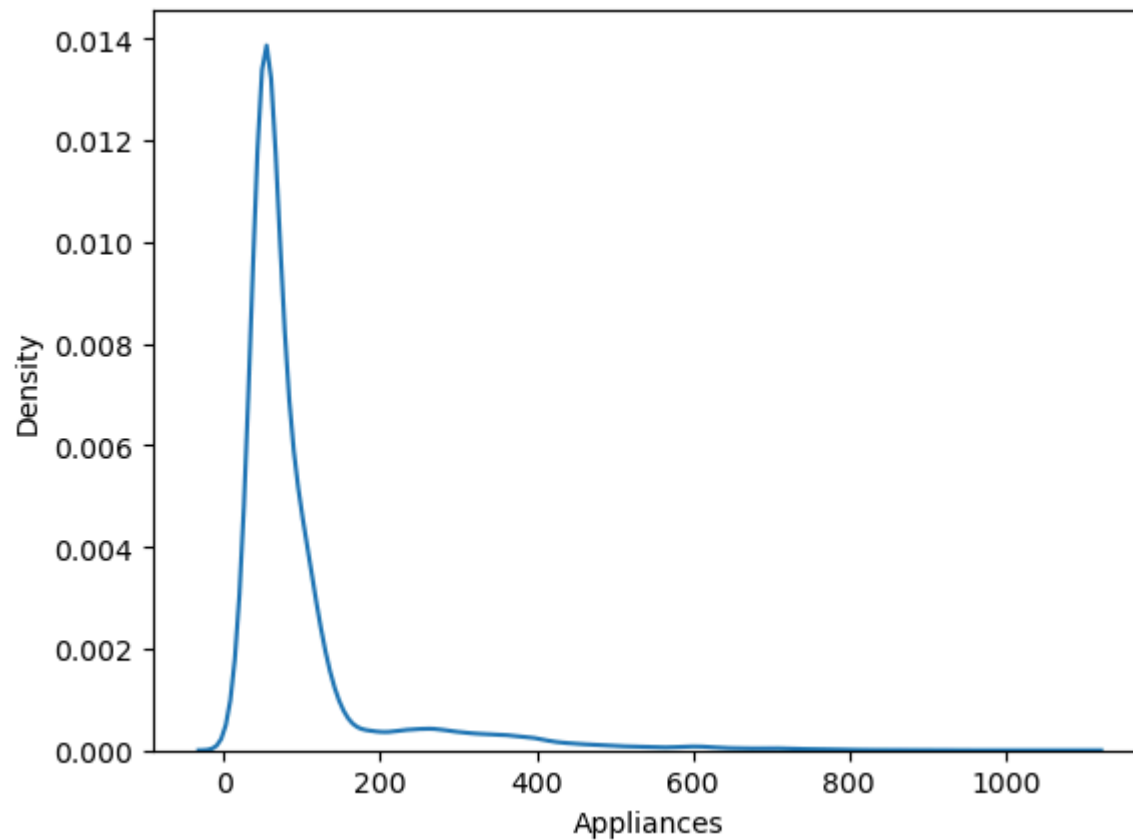
```
In [13]: # Create a dot plot (strip plot) to explore the relationship between features and target variable
sns.stripplot(x='T1', y='Appliances', data=df, jitter=True, size=5)

# Show the plot
plt.show()
```



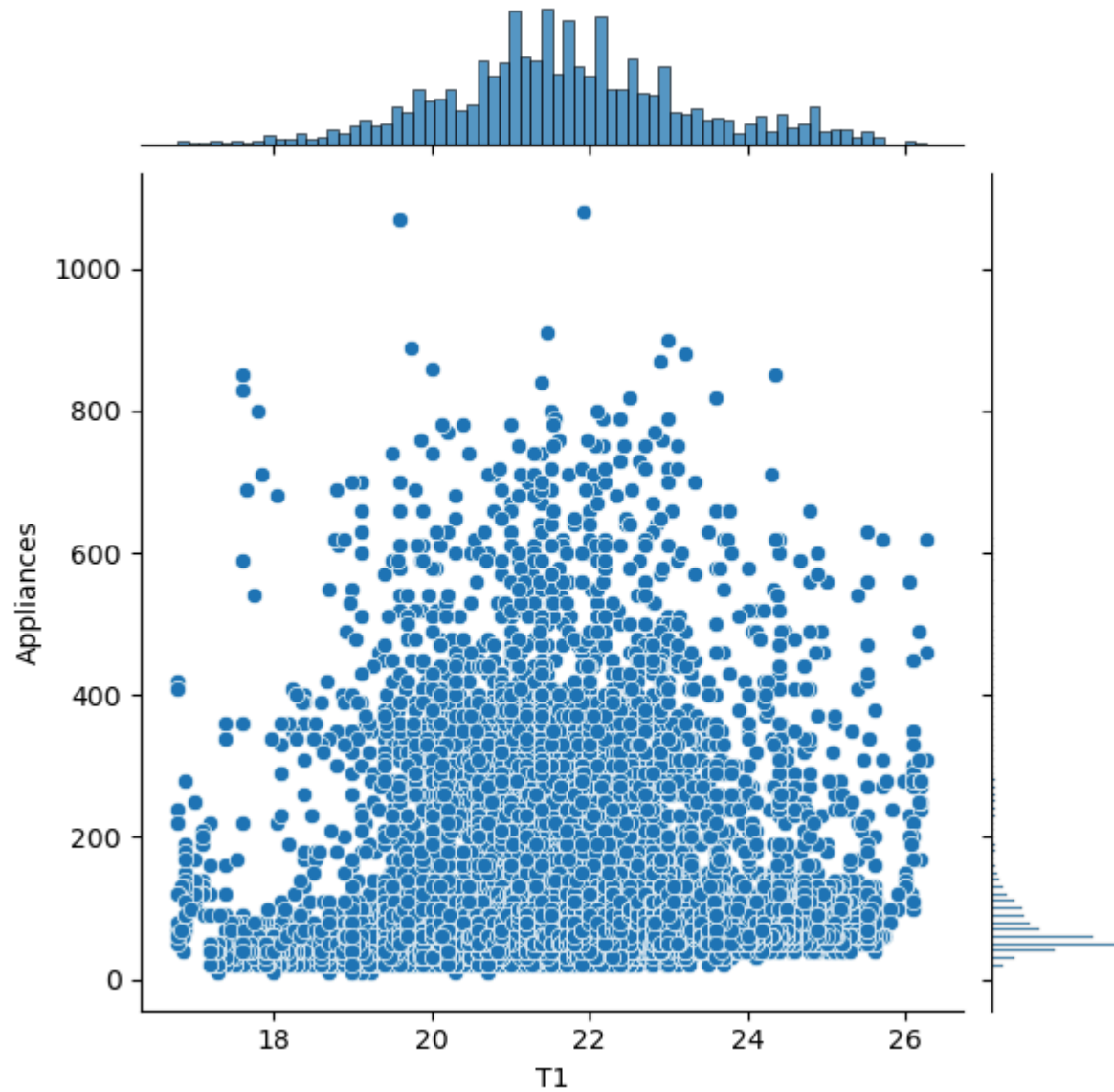

```
In [14]: sns.kdeplot(data=df, x='Appliances')
```

```
Out[14]: <Axes: xlabel='Appliances', ylabel='Density'>
```



```
In [15]: sns.jointplot(data=df, x='T1', y='Appliances')
```

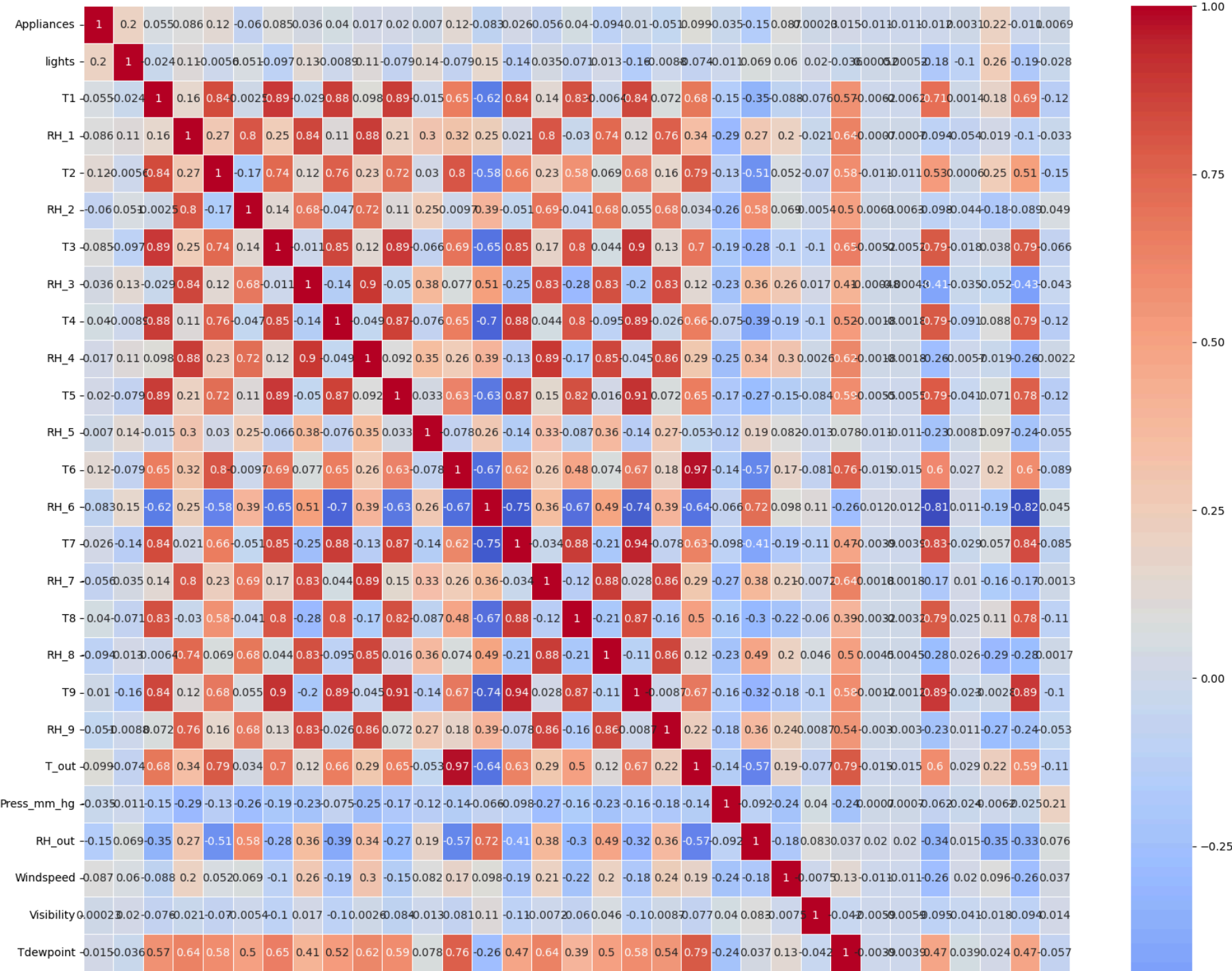
```
Out[15]: <seaborn.axisgrid.JointGrid at 0x25014620ed0>
```

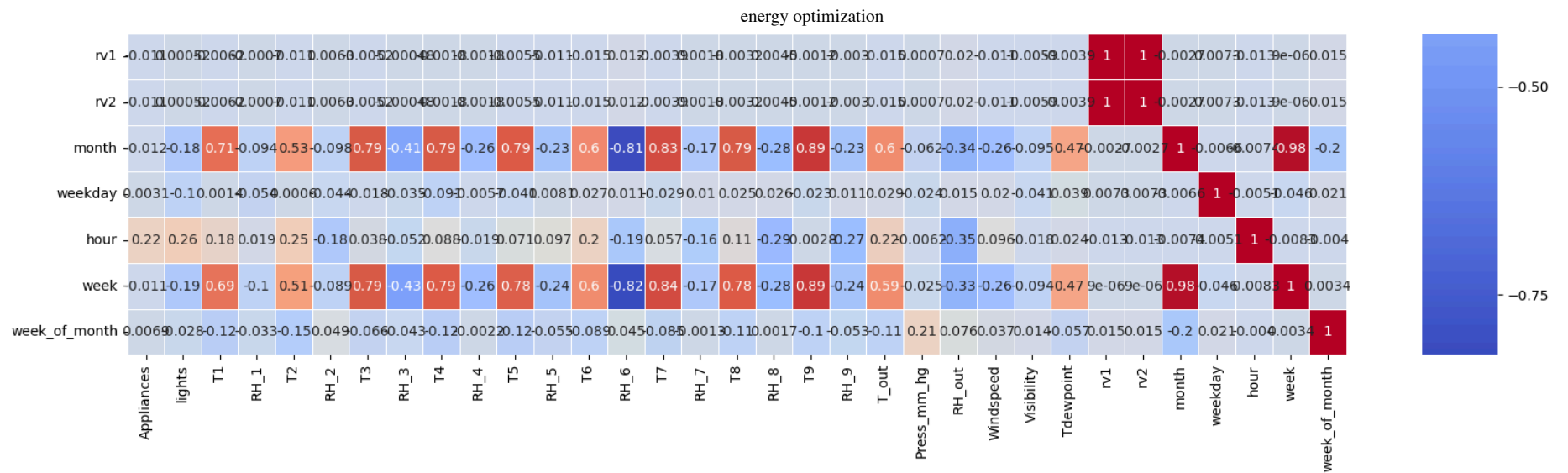


```
In [16]: corr_matrix = df.corr(method='pearson') # Specify method explicitly for clarity

# Create the heatmap with a larger size
fig, ax = plt.subplots(figsize=(20, 20)) # Using subplots to set the figure size
sns.heatmap(data=corr_matrix, annot=True, cmap='coolwarm', ax=ax, linewidths=0.5) # Added linewidth for clarity
```

```
# Show the plot  
plt.show()
```





```
In [17]: from statsmodels.stats.outliers_influence import variance_inflation_factor
# Extract the features to be checked for multicollinearity
X = df[['Appliances', 'lights', 'T1', 'RH_1', 'T2', 'RH_2', 'T3', 'RH_3', 'T4', 'RH_4', 'T5', 'RH_5', 'T6', 'RH_6', 'T7', 'RH_7', 'T8', 'RH_8', 'T9', 'RH_9', 'T_out', 'Press_mm_hg', 'RH_out', 'Windspeed', 'Visibility', 'Tdewpoint', 'rv1', 'rv2', 'month', 'weekday', 'hour', 'week', 'week_of_month']]

# Calculate the VIF for each feature
vif = pd.DataFrame()
vif['Features'] = X.columns
vif['VIF'] = [variance_inflation_factor(X.values, i) for i in range(X.shape[1])]

# Print the VIF values
vif
```

Out[17]:

	Features	VIF
0	Appliances	2.284702
1	lights	1.614563
2	T1	3606.384646
3	RH_1	1688.811559
4	T2	2519.584504
5	RH_2	2205.364940
6	T3	1276.028377
7	RH_3	1572.250962
8	T4	1027.802978
9	RH_4	1394.032518
10	T5	1187.467773
11	RH_5	45.212497
12	T6	89.644991
13	RH_6	40.427397
14	T7	1614.414817
15	RH_7	519.766076
16	T8	988.658164
17	RH_8	587.710860
18	T9	2605.026073
19	RH_9	640.567425
20	T_out	400.746379
21	Press_mm_hg	2088.596143
22	RH_out	1299.243530
23	Windspeed	5.272703
24	Visibility	12.037321

	Features	VIF
25	Tdewpoint	132.553551

```
In [18]: def remove_outliers_DBSCAN(df,eps,min_samples):
outlier_detection = DBSCAN(eps=eps, min_samples= min_samples)
clusters = outlier_detection.fit_predict(df.values.reshape(-1,1))
data = pd.DataFrame()
data['cluster'] = clusters
return data['cluster']
```

```
In [19]: df_cluster_input = df[['Appliances']]
clusters = remove_outliers_DBSCAN(df_cluster_input,0.5,10)
clusters.value_counts().sort_values(ascending=False)
```

```
Out[19]: 1      4368
0      3282
17     2019
2      1560
9      1205
...
4        11
58        11
59        11
42        10
56        10
Name: cluster, Length: 63, dtype: int64
```

```
In [20]: df_cluster=pd.DataFrame(clusters)
ind_outlier=df_cluster.index[df_cluster['cluster']==-1]
ind_outlier
```

```
Out[20]: Int64Index([ 393,  431,  432,  443,  731,  732,  867,  868, 1031,
                    1079,
                    ...,
                    15799, 15812, 15841, 16647, 17861, 18820, 18830, 19541, 19581,
                    19582],
                    dtype='int64', length=120)
```

```
In [21]: df['cluster']= df_cluster['cluster'].values
df.head()
```

Out[21]:

	date	Appliances	lights	T1	RH_1	T2	RH_2	T3	RH_3	T4	...	Tdewpoint	rv1	rv2	month
0	2016-01-11 17:00:00	60	30	19.89	47.596667	19.2	44.790000	19.79	44.730000	19.000000	...	5.3	13.275433	13.275433	1
1	2016-01-11 17:10:00	60	30	19.89	46.693333	19.2	44.722500	19.79	44.790000	19.000000	...	5.2	18.606195	18.606195	1
2	2016-01-11 17:20:00	50	30	19.89	46.300000	19.2	44.626667	19.79	44.933333	18.926667	...	5.1	28.642668	28.642668	1
3	2016-01-11 17:30:00	50	40	19.89	46.066667	19.2	44.590000	19.79	45.000000	18.890000	...	5.0	45.410389	45.410389	1
4	2016-01-11 17:40:00	60	40	19.89	46.333333	19.2	44.530000	19.79	45.000000	18.890000	...	4.9	10.084097	10.084097	1

5 rows × 36 columns

In [22]:

```
len(ind_outlier)
unique_clusters = df_cluster['cluster'].unique
unique_clusters
```

Out[22]:

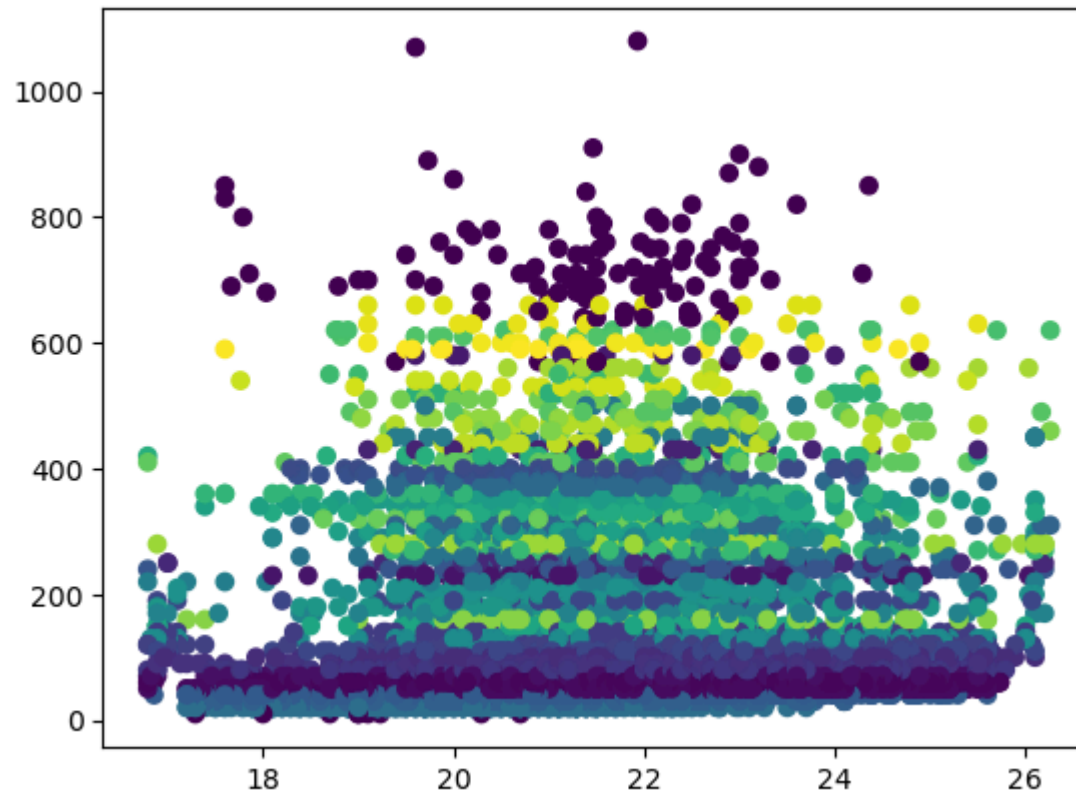
```
<bound method Series.unique of 0      0
1      0
2      1
3      1
4      0
..
19730   7
19731   8
19732  41
19733  38
19734   5
Name: cluster, Length: 19735, dtype: int64>
```

In [23]:

```
plt.scatter(df['T1'], df['Appliances'], c= df['cluster'])
```

Out[23]:

```
<matplotlib.collections.PathCollection at 0x250189d6dd0>
```

```
In [24]: # One-hot encode the 'day_of_week' column
weekdays_encoded = pd.get_dummies(df['day_of_week'], prefix='day') # Added prefix for clarity

# Concatenate the encoded columns back to the original DataFrame
data = pd.concat([df, weekdays_encoded], axis=1)

# Drop the original 'day_of_week' column (optional)
data.drop(columns=['day_of_week'], inplace=True)

# Display the updated DataFrame
data
```

Out [24]:

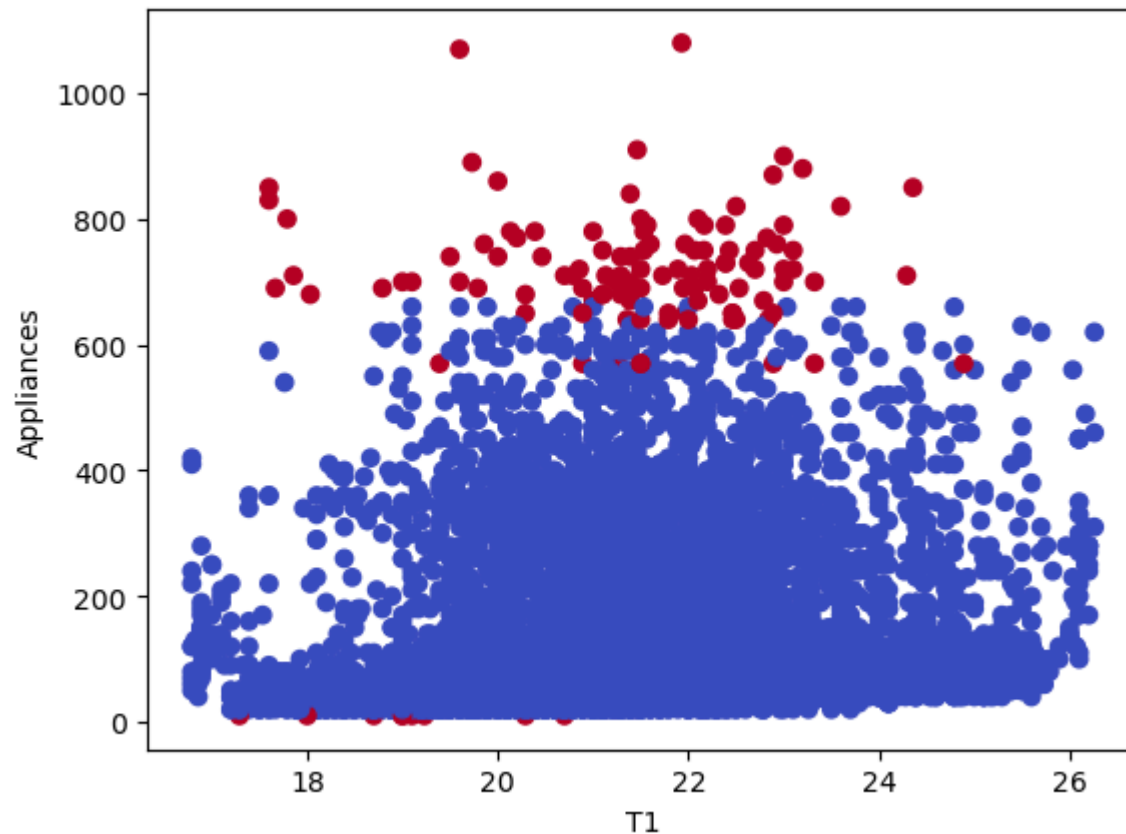
	date	Appliances	lights	T1	RH_1	T2	RH_2	T3	RH_3	T4	...	week	week_of_month
0	2016-01-11 17:00:00	60	30	19.890000	47.596667	19.200000	44.790000	19.790000	44.730000	19.000000	...	2	
1	2016-01-11 17:10:00	60	30	19.890000	46.693333	19.200000	44.722500	19.790000	44.790000	19.000000	...	2	
2	2016-01-11 17:20:00	50	30	19.890000	46.300000	19.200000	44.626667	19.790000	44.933333	18.926667	...	2	
3	2016-01-11 17:30:00	50	40	19.890000	46.066667	19.200000	44.590000	19.790000	45.000000	18.890000	...	2	
4	2016-01-11 17:40:00	60	40	19.890000	46.333333	19.200000	44.530000	19.790000	45.000000	18.890000	...	2	
...	...	...	...	...	...	...	...	...	...	...	...	...	...
19730	2016-05-27 17:20:00	100	0	25.566667	46.560000	25.890000	42.025714	27.200000	41.163333	24.700000	...	21	
19731	2016-05-27 17:30:00	90	0	25.500000	46.500000	25.754000	42.080000	27.133333	41.223333	24.700000	...	21	
19732	2016-05-27 17:40:00	270	10	25.500000	46.596667	25.628571	42.768571	27.050000	41.690000	24.700000	...	21	
19733	2016-05-27 17:50:00	420	10	25.500000	46.990000	25.414000	43.036000	26.890000	41.290000	24.700000	...	21	
19734	2016-05-27 18:00:00	430	10	25.500000	46.600000	25.264286	42.971429	26.823333	41.156667	24.700000	...	21	

19735 rows x 42 columns

```
In [25]: data['outlier_flag'] = data['cluster'].apply(lambda x: 1 if x == -1 else 0)

# Display the updated DataFrame
data.head()

# Plotting the outliers
plt.scatter(x=data['T1'], y=data['Appliances'], c=data['outlier_flag'], cmap='coolwarm') # Added cmap for coloring
plt.xlabel('T1') # Adding labels for clarity
plt.ylabel('Appliances')
plt.show()
```



```
In [26]: data_outliers_removed = data[data['outlier_flag'] == 0]
data_outliers = data[data['outlier_flag'] == 1]
data_outliers_removed.head()
```

Out [26]:

	date	Appliances	lights	T1	RH_1	T2	RH_2	T3	RH_3	T4	...	week_of_month	cluster	day_Friday	day_
0	2016-01-11 17:00:00	60	30	19.89	47.596667	19.2	44.790000	19.79	44.730000	19.000000	...	2	0	0	
1	2016-01-11 17:10:00	60	30	19.89	46.693333	19.2	44.722500	19.79	44.790000	19.000000	...	2	0	0	
2	2016-01-11 17:20:00	50	30	19.89	46.300000	19.2	44.626667	19.79	44.933333	18.926667	...	2	1	0	
3	2016-01-11 17:30:00	50	40	19.89	46.066667	19.2	44.590000	19.79	45.000000	18.890000	...	2	1	0	
4	2016-01-11 17:40:00	60	40	19.89	46.333333	19.2	44.530000	19.79	45.000000	18.890000	...	2	0	0	

5 rows x 43 columns

```
In [27]: fig = px.scatter(data, x="T1", y="Appliances", color="outlier_flag", marginal_y="violin",
                        marginal_x="box", trendline="ols", template="simple_white")
fig.show(renderer="colab")

fig = px.box(data, x="outlier_flag", y="Appliances")
fig.show(renderer="colab")
```



```
In [28]: #Outlier detection using IQR
def remove_outlier_IQR(df):
    Q1=df.quantile(0.25)
    Q3=df.quantile(0.75)
    IQR=Q3-Q1
    df_final=df[~((df<(Q1-1.5*IQR)) | (df>(Q3+1.5*IQR)))]
    return df_final

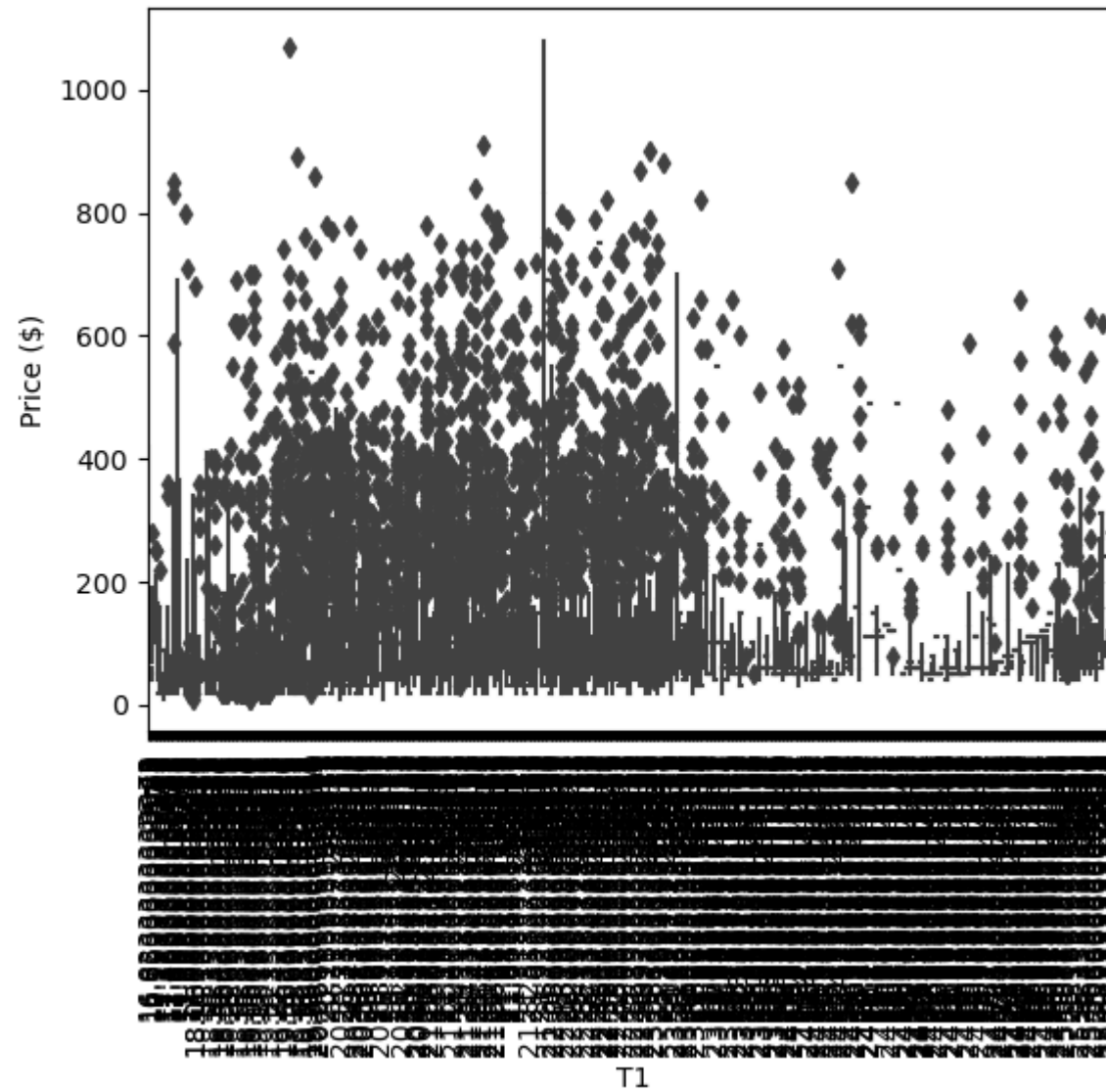
df_outlier_removed=remove_outlier_IQR(df_cluster_input.Appliances)
```

```
df_outlier_removed=pd.DataFrame(df_outlier_removed)
ind_diff=data.index.difference(df_outlier_removed.index)

for i in range(0, len(ind_diff),1):
    df_final=df_cluster_input.drop([ind_diff[i]])
    df=df_final

sns.boxplot(y='Appliances', x='T1',data=data)
plt.xticks(rotation=90)
plt.ylabel('Price ($)')
```

Out[28]: Text(0, 0.5, 'Price (\$)')



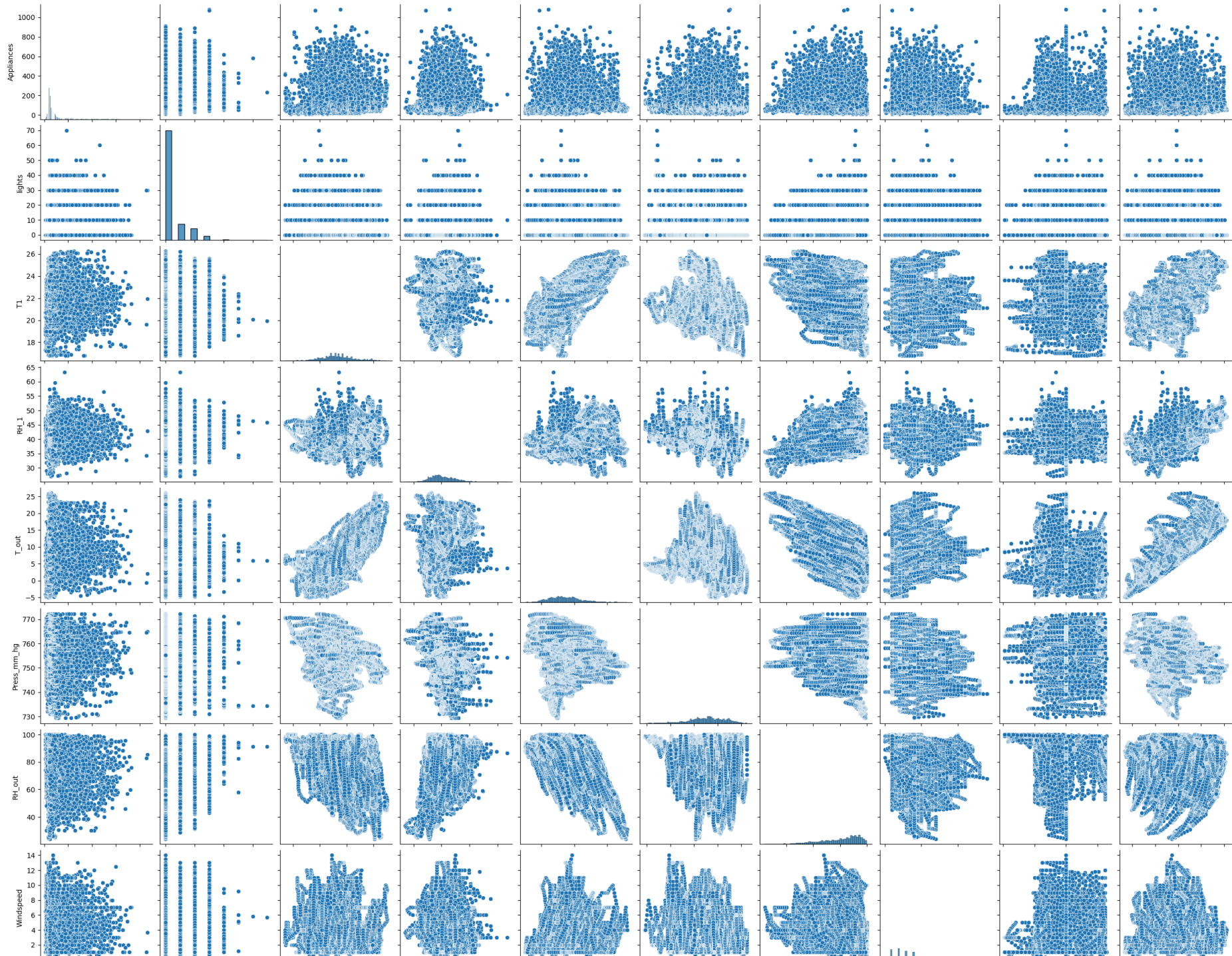
```
In [29]: df_outlier_removed
```

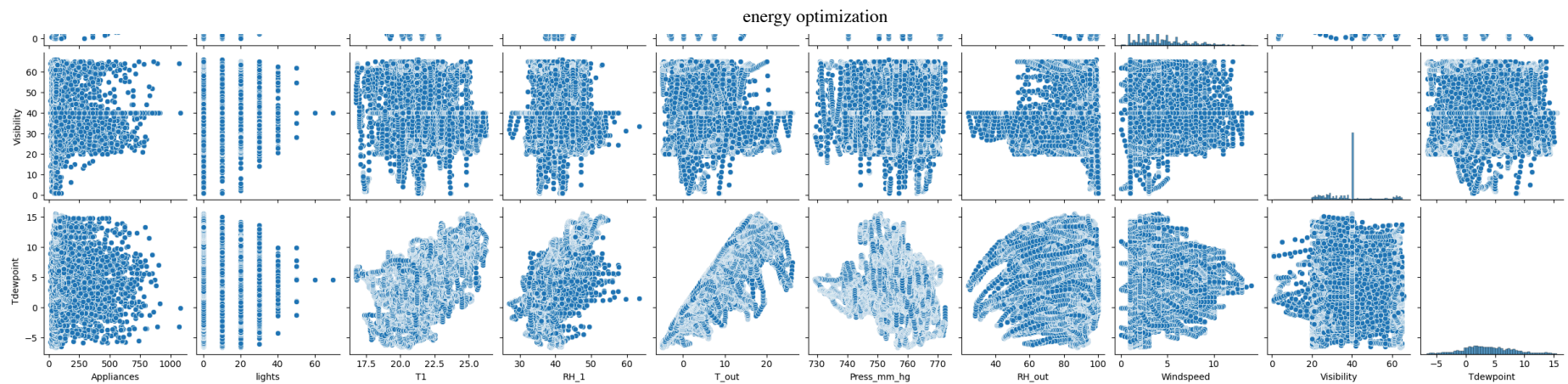

Out [29]:

Appliances	
0	60
1	60
2	50
3	50
4	60
...	...
19727	120
19728	110
19729	90
19730	100
19731	90

17597 rows × 1 columns

```
In [30]: data_1 = data[['Appliances', 'lights', 'T1', 'RH_1', 'T_out', 'Press_mm_hg', 'RH_out', 'Windspeed', 'Visibility', 'Tdewpoint']]
sns.pairplot(data_1, kind='scatter')
plt.show()
data.columns
```





```
Out[30]: Index(['date', 'Appliances', 'lights', 'T1', 'RH_1', 'T2', 'RH_2', 'T3',
        'RH_3', 'T4', 'RH_4', 'T5', 'RH_5', 'T6', 'RH_6', 'T7', 'RH_7', 'T8',
        'RH_8', 'T9', 'RH_9', 'T_out', 'Press_mm_hg', 'RH_out', 'Windspeed',
        'Visibility', 'Tdewpoint', 'rv1', 'rv2', 'month', 'weekday', 'hour',
        'week', 'week_of_month', 'cluster', 'day_Friday', 'day_Monday',
        'day_Saturday', 'day_Sunday', 'day_Thursday', 'day_Tuesday',
        'day_Wednesday', 'outlier_flag'],
        dtype='object')
```

```
In [31]: X = data.drop(columns=['Appliances', 'date', 'cluster', 'outlier_flag'], axis=1)
        y = data['Appliances']

        # First, split the data into 60% training and 40% for testing and validation
        X_train_1, X_temp_1, y_train_1, y_temp_1 = train_test_split(X, y, test_size=0.4, random_state=23)

        model = DecisionTreeRegressor()
        # fit the model
        model.fit(X_train_1, y_train_1)
        # get importance
        importance = model.feature_importances_
        # summarize feature importance
        impList = zip(X_train_1.columns, importance)
        for feature in sorted(impList, key = lambda t: t[1], reverse=True):
            print(feature)
```

```
( 'hour', 0.14232720665205634)
( 'T3', 0.06480878605147575)
( 'T8', 0.05353543967211084)
( 'RH_5', 0.04711334661340515)
( 'Press_mm_hg', 0.04537541898483164)
( 'RH_3', 0.04218525445469698)
( 'RH_2', 0.04131894937362588)
( 'RH_9', 0.04010455776118854)
( 'RH_1', 0.03956797480549171)
( 'RH_6', 0.038558209708412505)
( 'RH_8', 0.03424399497961252)
( 'RH_7', 0.03266399335600009)
( 'T1', 0.031032120293944566)
( 'lights', 0.027398607234627524)
( 'RH_4', 0.027346938663910587)
( 'T4', 0.02652564687193056)
( 'RH_out', 0.02650828024000976)
( 'T6', 0.024833479745904254)
( 'T7', 0.02296970868289258)
( 'T2', 0.022934053429575583)
( 'Visibility', 0.01974776149748391)
( 'rv2', 0.01918682739279463)
( 'Windspeed', 0.019152738350890805)
( 'Tdewpoint', 0.017908341443321072)
( 'rv1', 0.017497984795717555)
( 'week', 0.017003478474725987)
( 'T5', 0.016598467676326722)
( 'T_out', 0.012103334293326964)
( 'weekday', 0.011128785683441889)
( 'T9', 0.009145253409933454)
( 'week_of_month', 0.0039583556436615435)
( 'day_Sunday', 0.001701272669276315)
( 'day_Thursday', 0.0014481273059651014)
( 'day_Wednesday', 0.0008221653616006121)
( 'day_Monday', 0.0004443916423635073)
( 'day_Saturday', 0.000351627137274164)
( 'month', 0.00034018261782424024)
( 'day_Tuesday', 9.57611282846577e-05)
( 'day_Friday', 1.3175900083539868e-05)
```

```
In [32]: # Preprocessing
X = data.drop(columns=['Appliances','date'],axis=1)
y = data['Appliances']
```



```
# First, split the data into 60% training and 40% for testing and validation
X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.4, random_state=23)

# Then, split the testing and validation data equally into 50% each
X_test, X_val, y_test, y_val = train_test_split(X_temp, y_temp, test_size=0.5, random_state=23)
```

```
In [33]: X_train_sample = X_train.sample(frac=0.2, random_state=42)
y_train_sample = y_train.loc[X_train_sample.index]
sel_rfe_logistic.fit(X_train_sample, y_train_sample)
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[33], line 3
      1 X_train_sample = X_train.sample(frac=0.2, random_state=42)
      2 y_train_sample = y_train.loc[X_train_sample.index]
----> 3 sel_rfe_logistic.fit(X_train_sample, y_train_sample)

NameError: name 'sel_rfe_logistic' is not defined
```

```
In [34]: from sklearn.tree import DecisionTreeRegressor
model_tree = DecisionTreeRegressor()
sel_rfe_tree = RFE(estimator=model_tree, n_features_to_select=20, step=1)
sel_rfe_tree.fit(X_train, y_train)
```

```
Out[34]:
```

RFE

► **estimator: DecisionTreeRegressor**

► DecisionTreeRegressor

```
In [35]: feature_importance = model.feature_importances_
important_features = X_train.columns[np.argsort(feature_importance)[-20:]]
```

```
In [36]: #Feature selection using RFE with logistic regression
model_logistic = LogisticRegression(solver='lbfgs', multi_class='multinomial', max_iter=500)
sel_rfe_logistic = RFE(estimator=model_logistic, n_features_to_select=20, step=1)

sel_rfe_logistic.fit(X_train, y_train)
```

```
selected_features = X_train.columns[sel_rfe_logistic.support_]
selected_features
```

```
Out[36]: Index(['T1', 'RH_1', 'T2', 'RH_2', 'T3', 'RH_3', 'T4', 'RH_4', 'T5', 'T6',
        'T7', 'T8', 'RH_8', 'T9', 'RH_9', 'T_out', 'Tdewpoint', 'hour', 'week',
        'cluster'],
        dtype='object')
```

```
In [38]: #Outlier detection using the input variables
#Implementing LOF for the outlier detection
lof = LocalOutlierFactor(n_neighbors=20, contamination=.03)

y_pred = lof.fit_predict(X_train)

lofs_index = np.where(y_pred== -1)

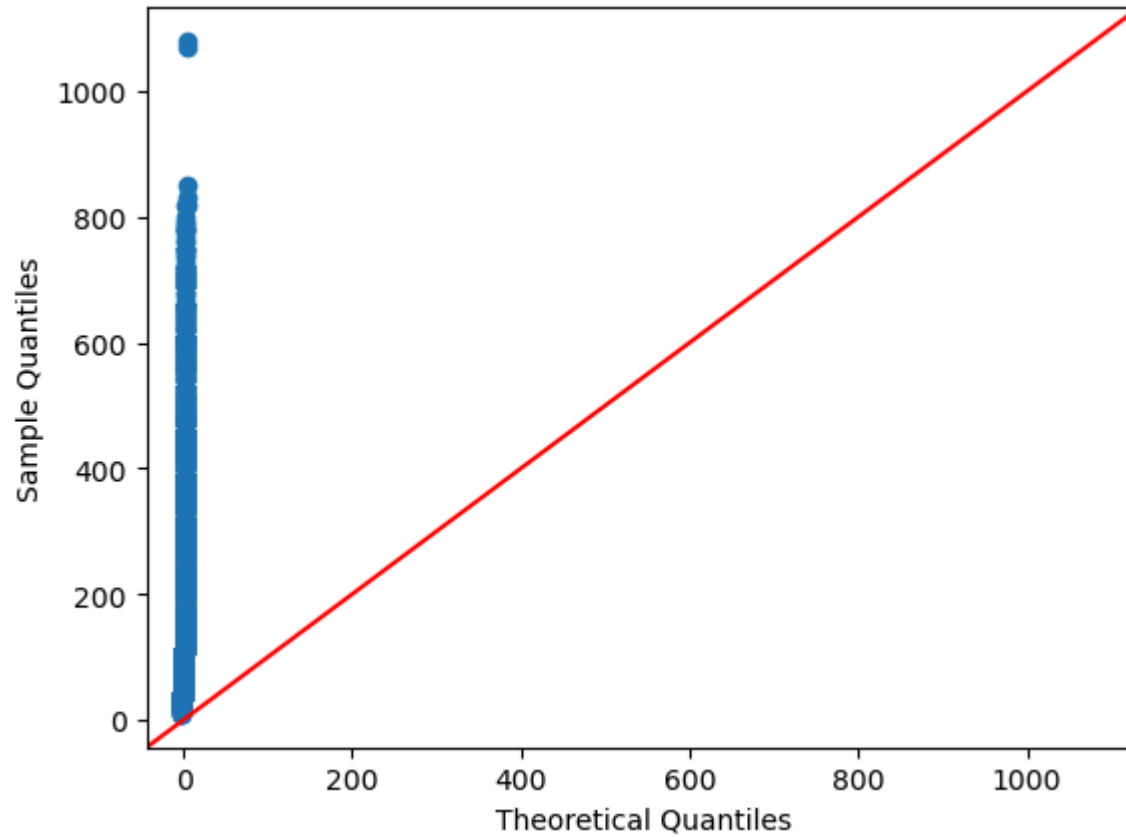
arr = list(lofs_index[0])
values = X_train[X_train.index.isin(arr)]
X_train['outlier'] = X_train.index.isin(values.index).astype(int)

outlier_df = X_train.copy()
outlier_df['Appliances'] = y_train.tolist()
X_train = X_train.drop('outlier',axis=1)

fig = px.box(outlier_df, y="Appliances", color="outlier",
             notched=True, # used notched shape
             title="Energy usage for regular data points and the data points with outliers"# add day column to hover c
             )
fig.show(renderer="colab")

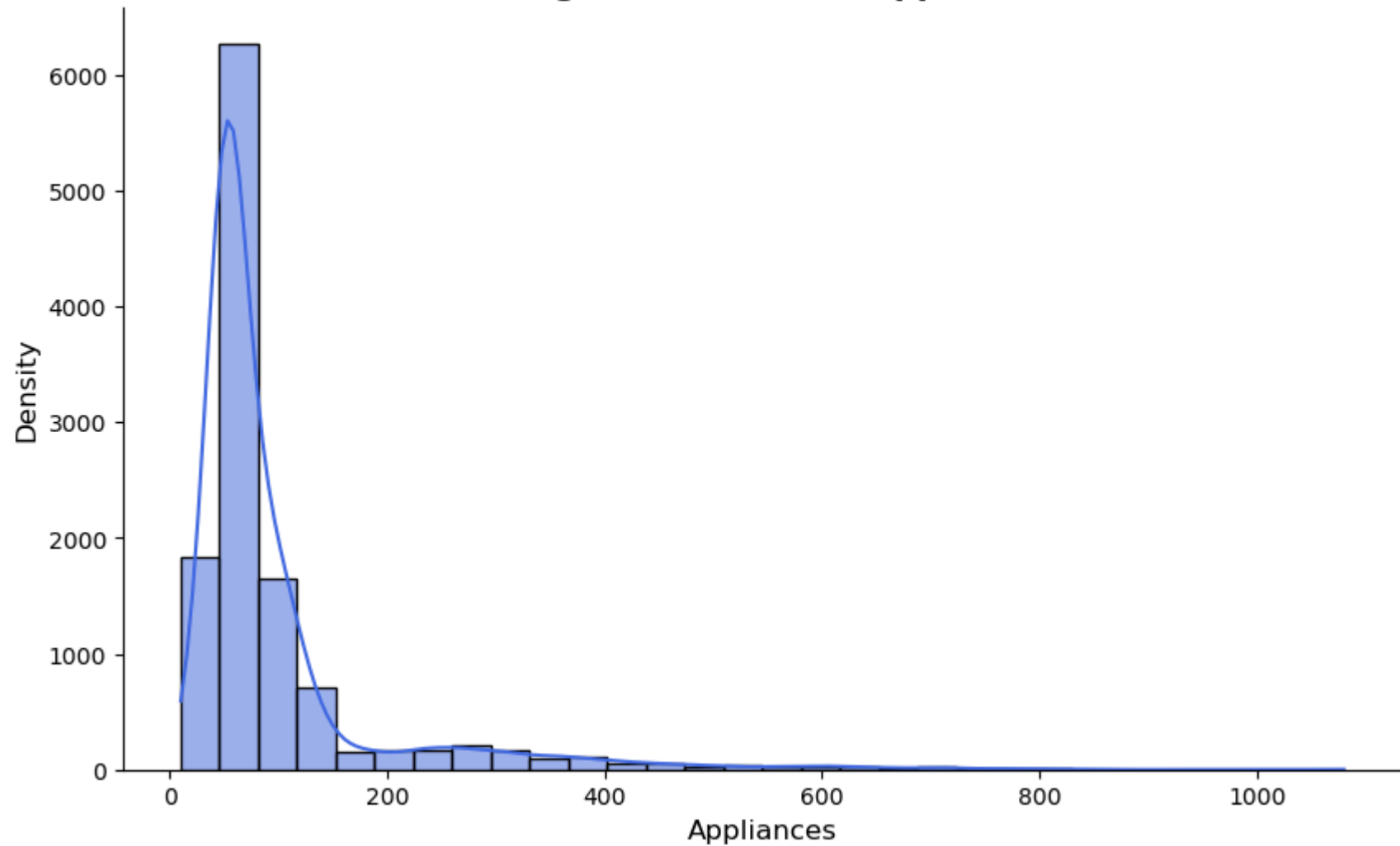
import statsmodels.api as sm
import pylab as py

sm.qqplot(outlier_df['Appliances'], line='45')
py.show()
```

```
In [39]: fig, ax = plt.subplots(figsize=(10, 6))
sns.histplot(data=outlier_df, x='Appliances', kde=True, color='royalblue', bins=30)
ax.set_title("Histogram and KDE of Appliances", fontsize=14, fontweight='bold')
ax.set_xlabel('Appliances', fontsize=12)
ax.set_ylabel('Density', fontsize=12)
sns.despine()
plt.show()
```


Histogram and KDE of Appliances

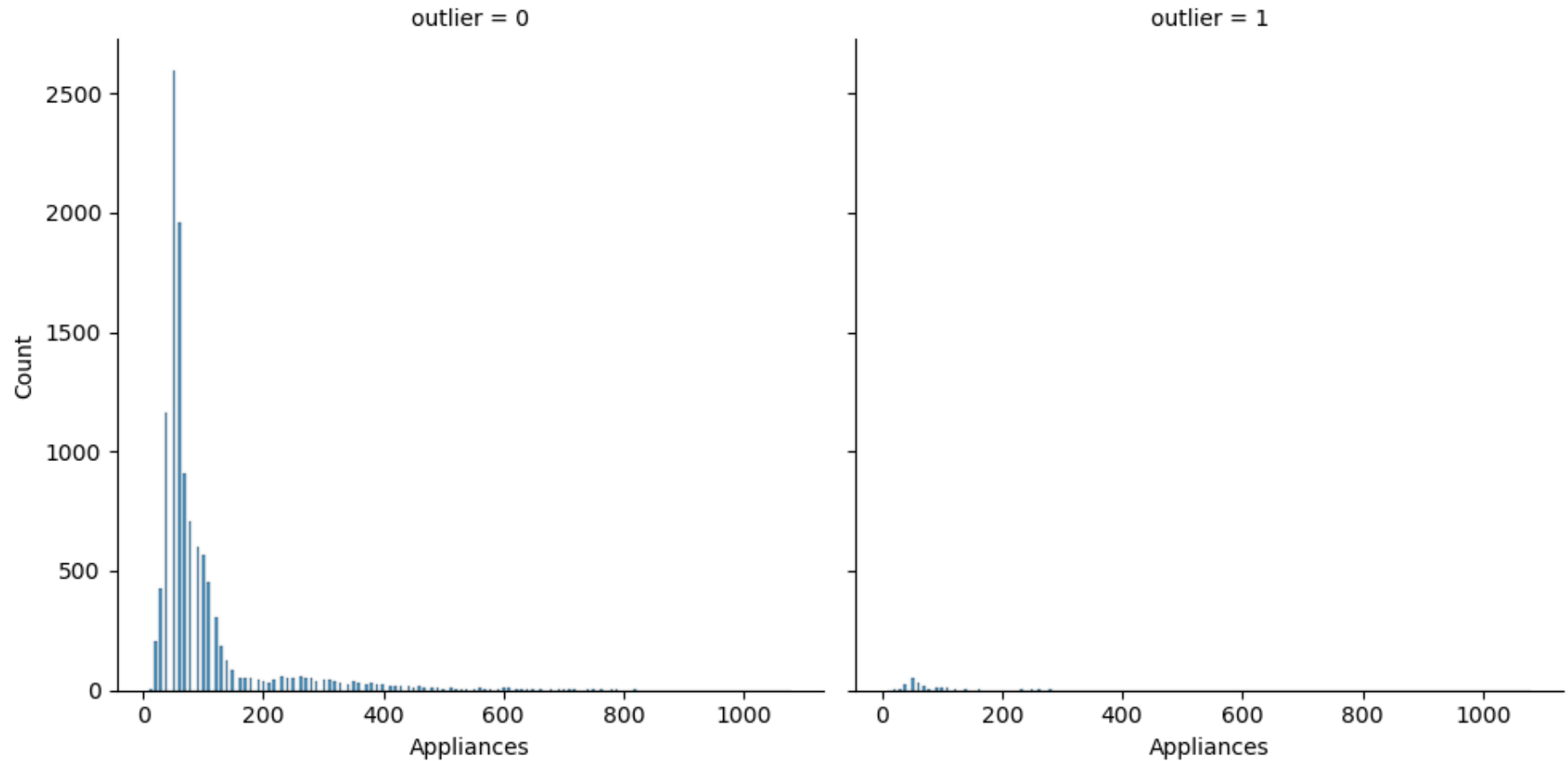


```
In [40]: skewness = outlier_df['Appliances'].skew()  
print(f'Skewness for Appliances = {skewness}')
```

Skewness for Appliances = 3.4181660638456384

```
In [41]: sns.displot(outlier_df, x="Appliances", col="outlier")
```

```
Out[41]: <seaborn.axisgrid.FacetGrid at 0x25038100050>
```

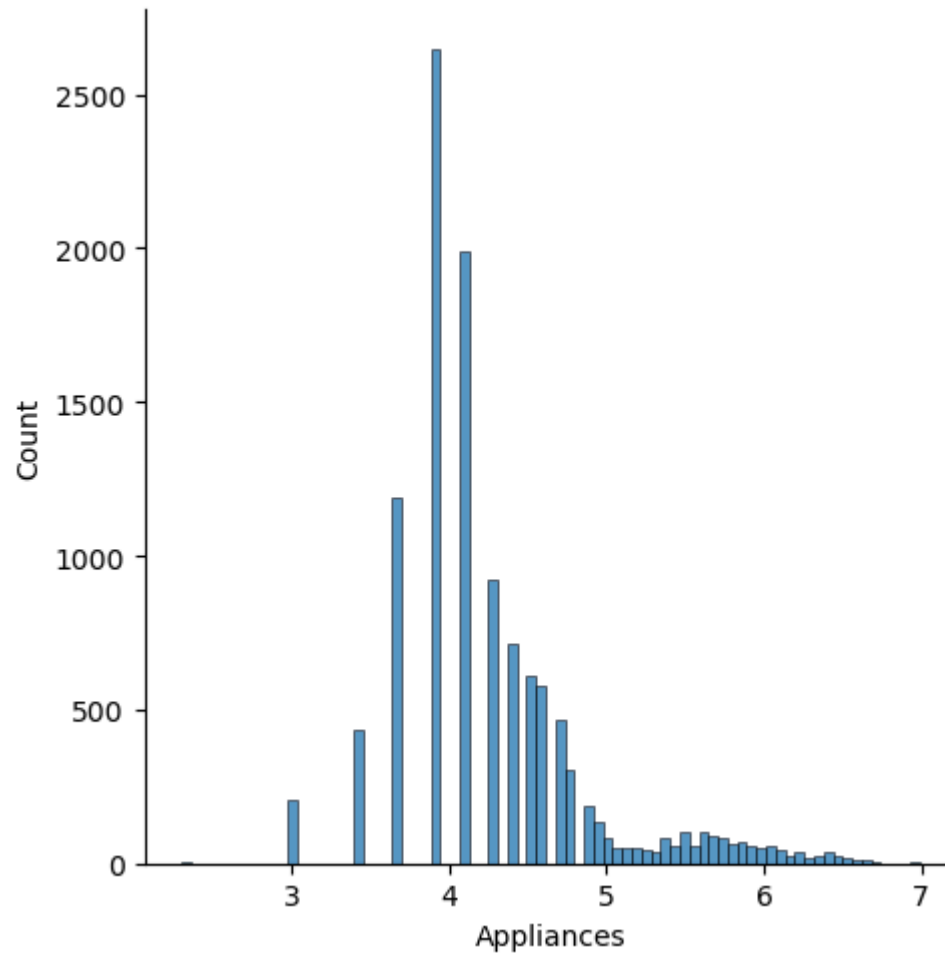


```
In [42]: appliances_log = np.log(outlier_df['Appliances'])  
print(f'Skewness for Appliances =', appliances_log.skew())
```

Skewness for Appliances = 1.1609365204344781

```
In [43]: sns.displot(appliances_log)
```

```
Out[43]: <seaborn.axisgrid.FacetGrid at 0x2503613c210>
```



```
In [44]: # Scale the features - do not run
scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
X_val = scaler.transform(X_val)
y_train_log = np.log(y_train)
```

```
In [47]: from sklearn.metrics import mean_squared_error, r2_score
import numpy as np

@dataclass
```

```
class KNearestNeighbors:
    X: np.array
    y: np.array
    k: int = 3
    distance_metric: str = 'euclidean'
    errors: list = field(default_factory=list)

    def __post_init__(self):
        pass

    def euclidean_distance(self, x1, x2):
        return np.sqrt(np.sum((x1 - x2) ** 2))

    def manhattan_distance(self, x1, x2):
        return np.sum(np.abs(x1 - x2))

    def minkowski_distance(self, x1, x2, p=3):
        return np.sum(np.abs(x1 - x2) ** p) ** (1 / p)

    def compute_distance(self, x1, x2):
        if self.distance_metric == 'euclidean':
            return self.euclidean_distance(x1, x2)
        elif self.distance_metric == 'manhattan':
            return self.manhattan_distance(x1, x2)
        elif self.distance_metric == 'minkowski':
            return self.minkowski_distance(x1, x2)
        else:
            raise ValueError(f"Unknown distance metric: {self.distance_metric}")

    def get_neighbors(self, X_train, y_train, test_instance):
        distances = []
        for i in range(len(X_train)):
            distance = self.compute_distance(test_instance, X_train[i])
            distances.append((y_train[i], distance))
        distances.sort(key=lambda x: x[1])
        neighbors = [distances[i][0] for i in range(self.k)]
        return neighbors

    def predict_instance(self, X_train, y_train, test_instance):
        neighbors = self.get_neighbors(X_train, y_train, test_instance)
        prediction = Counter(neighbors).most_common(1)[0][0]
        return prediction

    def predict(self, X_test):
```

```
predictions = []
for test_instance in X_test:
    prediction = self.predict_instance(self.X, self.y, test_instance)
    predictions.append(prediction)
return np.array(predictions)

def sse(self, y_true, y_pred):
    return ((y_true - y_pred) ** 2).sum()

def rmse(self, y_true, y_pred):
    return np.sqrt(mean_squared_error(y_true, y_pred))

def r2_squared(self, y_true, y_pred):
    return r2_score(y_true, y_pred)

def error_report(self, X_test, y_test):
    y_pred = self.predict(X_test)
    sse_val = self.sse(y_test, y_pred)
    rmse_val = self.rmse(y_test, y_pred)
    r2_val = self.r2_squared(y_test, y_pred)

    print(f"SSE: {sse_val}")
    print(f"RMSE: {rmse_val}")
    print(f"R^2: {r2_val}")

def mse_var_bias(self, X_test, y_test):
    y_pred = self.predict(X_test)
    mse = mean_squared_error(y_test, y_pred)
    bias = np.mean((y_test - np.mean(y_pred)) ** 2)
    variance = np.var(y_pred)

    print(f"MSE: {mse:.2f}")
    print(f"Bias: {bias:.2f}")
    print(f"Variance: {variance:.2f}")

def fit(self, X_test=None, y_test=None, X_val=None, y_val=None):
    print(f"Training KNN with k={self.k} and distance metric={self.distance_metric}")

    if X_test is not None and y_test is not None:
        print("For Test Data")
        self.error_report(X_test, y_test)

    if X_val is not None and y_val is not None:
        print("For Validation Data")
```

```

        self.error_report(X_val, y_val)
        print("\nBias Variance Tradeoff")
        self.mse_var_bias(X_test, y_test)

knn = KNearestNeighbors(
    X=X_train,
    y=np.array(y_train),
    k=5,
    distance_metric='euclidean'
)

# Fit and check performance metrics
knn.fit(X_test=X_test, y_test=y_test, X_val=X_val, y_val=y_val)

```

Training KNN with k=5 and distance metric=euclidean

For Test Data

SSE: 20287400

RMSE: 71.69347526618864

R²: 0.5258759903245372

For Validation Data

SSE: 20336500

RMSE: 71.78017987899484

R²: 0.5384832717872021

Bias Variance Tradeoff

MSE: 5139.95

Bias: 10935.80

Variance: 7754.43

```

In [48]: knn = KNearestNeighbors(
    X=X_train, # All columns except Appliances
    y=np.array(y_train), # Appliances column
    k=5, # Number of neighbors to consider
    distance_metric='euclidean' # Euclidean distance for neighbors
).fit(X_test=X_test, y_test=y_test, X_val=X_val, y_val=y_val)

```

Training KNN with k=5 and distance metric=euclidean

For Test Data

SSE: 20287400

RMSE: 71.69347526618864

R²: 0.5258759903245372

For Validation Data

SSE: 20336500

RMSE: 71.78017987899484

R²: 0.5384832717872021

Bias Variance Tradeoff

MSE: 5139.95

Bias: 10935.80

Variance: 7754.43

```
In [50]: dff = [
    [{"Approach": "K-Nearest Neighbors (KNN)", "Learning Rate": "0.001", "Lambda Value": "-", "RMSE": "71.69", "R^2": "0.53"},
    {"Approach": "K-Nearest Neighbors (KNN) - Validation", "Learning Rate": "0.001", "Lambda Value": "-", "RMSE": "71.78", "R^2": "0.54"}]

# Set the float format to scientific notation
pd.options.display.float_format = '{:.4e}'.format

# Create a pandas DataFrame with the data
results_table = pd.DataFrame(dff)

# Center-align the values and the column names in the DataFrame
styled_table = results_table.style.set_properties(**{'text-align': 'center'}).set_table_styles([
    {'selector': 'th',
     'props': [('text-align', 'center')]}],
])

# Display the styled DataFrame
styled_table
```

```
Out[50]:
```

	Approach	Learning Rate	Lambda Value	RMSE	R ²
0	K-Nearest Neighbors (KNN)	0.001	-	71.69	0.53
1	K-Nearest Neighbors (KNN) - Validation	0.001	-	71.78	0.54

```
In [51]: from dataclasses import dataclass, field
import numpy as np
```

```
from tqdm import tqdm
from sklearn.metrics import r2_score

@dataclass
class LinearRegression:
    X : np.array
    y : np.array
    learningRate : float
    epsilon : float
    maxIteration : int = 50000
    gd : int = 0
    lambda_value : float = 0.0
    sample_size : int = 100
    s : int = 0
    errors : list = field(default_factory=list)

    def __post_init__(self):
        # You can initialize the weights here if necessary
        pass

    def addX0(self, X):
        return np.column_stack([np.ones(X.shape[0]), X])

    def rank(self, X):
        rank = np.linalg.matrix_rank(X)
        return rank

    def checkFullrank(self, X):
        rank = self.rank(X)
        if rank == min(X.shape):
            print("It's full rank")
            self.Fullrank = True
        else:
            self.Fullrank = False

    def checkLowRank(self, X):
        if X.shape[1] > X.shape[0]:
            self.LowRank = True
        else:
            self.LowRank = False

    def normalEquation(self, X, y):
        normEq_term = self.lambda_value * np.identity(X.shape[1])
        self.w = np.linalg.inv(X.T.dot(X) + normEq_term).dot(X.T).dot(y)
```



```
self.error_report(X, y)

def predict(self, X):
    return X.dot(self.w)

def sse(self, X, y):
    y_hat = self.predict(X)
    return ((y_hat - y) ** 2).sum()

def costFunction(self, X, y):
    sse = self.sse(X, y)
    return sse / 2

def rmse(self, X, y):
    return np.sqrt(self.sse(X, y) / X.shape[0])

def r2_squared(self, X, y):
    y_hat = self.predict(X)
    return np.exp(r2_score(y, y_hat))

def error_report(self, X, y):
    print(f"SSE: {self.costFunction(X, y)}")
    print(f"RMSE: {self.rmse(X, y)}")
    print(f"R^2: {self.r2_squared(X, y)}")

def costDerivative(self, X, y):
    y_hat = self.predict(X)
    m = y.shape[0]
    return 1 / m * X.T.dot(y_hat - y)

def gradientDescent(self, X, y):
    self.errors = []
    prev_error = float(np.inf)
    linear_reg_term = self.lambda_value * self.w
    for i in tqdm(range(self.maxIteration)):
        grad = self.costDerivative(X, y)
        self.w -= self.learningRate * (grad + linear_reg_term)
        current_error = self.rmse(X, y)
        if np.isnan(current_error):
            print("The values are out of domain")
            break
        self.errors.append(current_error)
        if np.allclose(current_error, prev_error, rtol=self.epsilon):
            print("The model stopped learning")
```

```

        self.s = i + 1
        break
    prev_error = current_error
    self.error_report(X, y)

def stochasticgradientDescent(self, X, y):
    self.errors = []
    prev_error = float(np.inf)
    for i in tqdm(range(self.maxIteration)):
        index = np.random.choice(np.arange(len(X)), self.sample_size, replace=False)
        X_sample = X[index]
        y_sample = y[index]
        grad_sample = self.costDerivative(X_sample, y_sample)
        linear_reg_term = self.lambda_value * self.w
        self.w -= self.learningRate * (grad_sample + linear_reg_term)
        current_error = self.rmse(X, y)
        self.errors.append(current_error)
        if abs(current_error - prev_error) < self.epsilon:
            print("The model stopped learning")
            self.s = i + 1
            break
        prev_error = current_error
    self.error_report(X, y)

def mse_var_bias(self, X, y):
    mse = self.costFunction(X, y) / len(y)
    y_hat = self.predict(X)
    mse_bias = np.mean((y - np.mean(y_hat)) ** 2)
    mse_var = np.var(y_hat)
    print(f"MSE: {mse:.2f}")
    print(f"Bias: {mse_bias:.2f}")
    print(f"Variance: {mse_var:.2f}")

def fit(self, X_test=None, y_test=None, X_val=None, y_val=None):
    X_train = self.addX0(self.X)
    y_train = self.y

    self.checkFullrank(X_train)
    self.checkLowRank(X_train)

    if self.Fullrank and not self.LowRank and X_train.shape[0] < 10000 and self.gd == 0:
        print("Solving using closed form")
        self.normalEquation(X_train, y_train)

```

```

elif self.gd == 1:
    print("Solving using Gradient Descent")
    self.w = np.zeros(X_train.shape[1])
    self.gradientDescent(X_train, y_train)
    if X_test is not None and y_test is not None:
        print("For Test Data")
        X_testdata = self.addX0(X_test)
        self.error_report(X_testdata, y_test)

    if X_val is not None and y_val is not None:
        print("For Validation Data")
        X_valid = self.addX0(X_val)
        self.error_report(X_valid, y_val)
        print("\nBias Variance Tradeoff")
        self.mse_var_bias(X_testdata, y_test)

else:
    print("Solving using Stochastic Gradient")
    self.w = np.zeros(X_train.shape[1])
    self.stochasticgradientDescent(X_train, y_train)

print(f"Weights: {self.w}")

# Assuming X_train, y_train, X_test, y_test, X_val, and y_val are defined
lf = LinearRegression(
    X=X_train,
    y=np.array(y_train),
    learningRate=0.001,
    maxIteration=500,
    epsilon=0.001,
    gd=1,
    lambda_value=0.01
).fit(X_test=X_test, y_test=y_test, X_val=X_val, y_val=y_val)

```

Solving using Gradient Descent

0%||
s]

| 1/500 [00:00<00:02, 199.58it/

The model stopped learning

SSE: 115620747.04180431

RMSE: 139.74577702408487

R²: 0.39929486633004657

For Test Data

SSE: 40217611.18957591

RMSE: 142.75442209048234

R²: 0.41486555686316523

For Validation Data

SSE: 41384857.98767487

RMSE: 144.81120562205865

R²: 0.4154567602029395

Bias Variance Tradeoff

MSE: 10189.41

Bias: 20408.27

Variance: 0.06

Weights: [1.93955841e-01 4.05939078e-02 1.16894407e-02 1.77950305e-02

2.50782424e-02 -1.11881927e-02 1.77299166e-02 8.63870670e-03

8.88987482e-03 5.47339200e-03 5.11846459e-03 3.15830903e-03

2.46959470e-02 -1.68794808e-02 5.31866742e-03 -9.37337777e-03

7.51840916e-03 -1.81929471e-02 2.37419771e-03 -9.24922026e-03

2.12669351e-02 -5.98487404e-03 -3.04709834e-02 1.83621591e-02

4.02985841e-05 4.69890470e-03 -2.67379717e-03 -2.67379717e-03

-2.47538657e-03 -1.31394260e-03 4.42380097e-02 -2.21473217e-03

1.97195999e-03 1.10971498e-01 5.56493543e-03 1.36233161e-02

5.18684460e-03 -2.27255716e-03 -7.04354974e-03 -9.27316049e-03

-5.32552366e-03 8.89446757e-02]

```
In [52]: lf = LinearRegression(X = X_train, # All columns except Appliances
                                y = np.array(y_train), # Appliances column
                                learningRate=0.0001,
                                maxIteration = 500,
                                epsilon=0.001,
                                gd=1,
                                lambda_value=0.001
                                ).fit()
```

Solving using Gradient Descent

0%||
s]

| 1/500 [00:00<00:02, 249.35it/

```
The model stopped learning
SSE: 115976894.1310322
RMSE: 139.96084129749775
R^2: 0.3969427103264449
Weights: [ 1.94043165e-02  4.06219748e-03  1.17445159e-03  1.78116650e-03
 2.51453689e-03 -1.12073943e-03  1.77809383e-03  8.63730187e-04
 8.94071283e-04  5.48026602e-04  5.16475872e-04  3.15882159e-04
 2.47632419e-03 -1.69361488e-03  5.36540663e-04 -9.37701653e-04
 7.56027998e-04 -1.82133218e-03  2.41799155e-04 -9.25904829e-04
 2.13339462e-03 -5.99365345e-04 -3.05337564e-03  1.83841630e-03
 3.22322515e-06  4.73482192e-04 -2.67916657e-04 -2.67916657e-04
-2.43860173e-04 -1.31837400e-04  4.42946292e-03 -2.17820876e-04
 1.96641614e-04  1.11026935e-02  5.56836071e-04  1.36386830e-03
 5.18837655e-04 -2.27438306e-04 -7.04954812e-04 -9.28037602e-04
-5.33047106e-04  8.89830786e-03]
```

```
def fit(self, X_test=None, y_test=None, X_val=None, y_val=None):
    X_train = self.addX0(self.X)
    y_train = self.y

    self.checkFullrank(X_train)
    self.checkLowRank(X_train)

    if self.Fullrank and not self.LowRank and X_train.shape[0] < 10000 and self.gd == 0:
        print("Solving using closed form")
        self.normalEquation(X_train, y_train)

    elif self.gd == 1:
        print("Solving using Gradient Descent")
        self.w = np.zeros(X_train.shape[1])
        self.gradientDescent(X_train, y_train)
        if X_test is not None and y_test is not None:
            print("For Test Data")
            X_testdata = self.addX0(X_test)
            self.error_report(X_testdata, y_test)

            # Generate CSV output
            y_pred = self.predict(X_testdata)
            results_df = pd.DataFrame({
                'Actual': y_test,
                'Predicted': y_pred,
            })
            results_df.to_csv("linear_output.csv", index=False)
            print("Output saved to 'linear_output.csv'.")

            if X_val is not None and y_val is not None:
                print("For Validation Data")
                X_valid = self.addX0(X_val)
                self.error_report(X_valid, y_val)
                print("\nBias Variance Tradeoff")
                self.mse_var_bias(X_testdata, y_test)

        else:
            print("Solving using Stochastic Gradient")
            self.w = np.zeros(X_train.shape[1])
            self.stochasticgradientDescent(X_train, y_train)

    print(f"Weights: {self.w}")

# Assuming X_train, y_train, X_test, y_test, X_val, and y_val are defined
```

```
lf = LinearRegression(
    X=X_train,
    y=np.array(y_train),
    learningRate=0.001,
    maxIteration=500,
    epsilon=0.001,
    gd=1,
    lambda_value=0.01
).fit(X_test=X_test, y_test=y_test, X_val=X_val, y_val=y_val)
```

Solving using Gradient Descent

```
0%|| | 1/500 [00:00<00:02, 236.39it/s]
```

The model stopped learning

SSE: 115620747.04180431

RMSE: 139.74577702408487

R²: 0.39929486633004657

For Test Data

SSE: 40217611.18957591

RMSE: 142.75442209048234

R²: 0.41486555686316523

For Validation Data

SSE: 41384857.98767487

RMSE: 144.81120562205865

R²: 0.4154567602029395

Bias Variance Tradeoff

MSE: 10189.41

Bias: 20408.27

Variance: 0.06

Weights: [1.93955841e-01 4.05939078e-02 1.16894407e-02 1.77950305e-02

2.50782424e-02 -1.11881927e-02 1.77299166e-02 8.63870670e-03

8.88987482e-03 5.47339200e-03 5.11846459e-03 3.15830903e-03

2.46959470e-02 -1.68794808e-02 5.31866742e-03 -9.37337777e-03

7.51840916e-03 -1.81929471e-02 2.37419771e-03 -9.24922026e-03

2.12669351e-02 -5.98487404e-03 -3.04709834e-02 1.83621591e-02

4.02985841e-05 4.69890470e-03 -2.67379717e-03 -2.67379717e-03

-2.47538657e-03 -1.31394260e-03 4.42380097e-02 -2.21473217e-03

1.97195999e-03 1.10971498e-01 5.56493543e-03 1.36233161e-02

5.18684460e-03 -2.27255716e-03 -7.04354974e-03 -9.27316049e-03

-5.32552366e-03 8.89446757e-02]

```
In [55]: def gradientDescent(self, X, y):
    self.errors = []
    prev_error = float(np.inf)
    linear_reg_term = self.lambda_value * self.w
    for i in tqdm(range(self.maxIteration)):
        grad = self.costDerivative(X, y)
        self.w -= self.learningRate * (grad + linear_reg_term)
        current_error = self.rmse(X, y)
        self.errors.append(current_error) # Store RMSE
        if np.isnan(current_error):
            print("The values are out of Domain")
            break
        if np.allclose(current_error, prev_error, rtol=self.epsilon):
            print("The model stopped learning")
            self.s = i + 1
            break
        prev_error = current_error
    self.error_report(X, y)

    def stochasticgradientDescent(self, X, y, sample_size):
        self.errors = []
        prev_error = float(np.inf)
        for i in tqdm(range(self.maxIteration)):
            index = np.random.choice(np.arange(len(X)), self.sample_size, replace=False)
            X_sample = X[index]
            y_sample = y[index]
            grad_sample = self.costDerivative(X_sample, y_sample)
            linear_reg_term = self.lambda_value * self.w
            self.w = self.w - self.learningRate * (grad_sample + linear_reg_term)
            current_error = self.rmse(X, y)
            self.errors.append(current_error) # Store RMSE
            if abs(current_error - prev_error) < self.epsilon:
                print("The model stopped learning")
                self.s = i + 1
                break
            prev_error = current_error
        self.error_report(X, y)
```

```
In [56]: import matplotlib.pyplot as plt

    def plot_rmse(self):
        plt.figure(figsize=(10, 6))
        plt.plot(self.errors, label='RMSE', color='blue')
```



```
plt.title('Change in RMSE Over Iterations')
plt.xlabel('Iterations')
plt.ylabel('RMSE')
plt.legend()
plt.grid()
plt.show()
```

```
In [57]: # import pandas as pd

dff = [
    {"Approach": "Gradient Descent", "Learning Rate": "0.0001", "Lambda Value": "0.001", "RMSE": "139.9", "R^2": "0.39"},
    {"Approach": "Gradient Descent", "Learning Rate": "0.001", "Lambda Value": "0.01", "RMSE": "139.7", "R^2": "0.38"},
    {"Approach": "Gradient Descent", "Learning Rate": "0.01", "Lambda Value": "0.1", "RMSE": "69.01", "R^2": "1.70"},
    {"Approach": "Stochastic Gradient Descent", "Learning Rate": "0.0001", "Lambda Value": "0.001", "RMSE": "134.3", "R^2": "0.39"},
    {"Approach": "Stochastic Gradient Descent", "Learning Rate": "0.001", "Lambda Value": "0.01", "RMSE": "99.01", "R^2": "0.38"},
    {"Approach": "Stochastic Gradient Descent", "Learning Rate": "0.01", "Lambda Value": "0.1", "RMSE": "72.63", "R^2": "1.70"}
]

# Set the float format to scientific notation
pd.options.display.float_format = '{:.4e}'.format

# Create a pandas DataFrame with the data
results_table = pd.DataFrame(dff)

# Center-align the values and the column names in the DataFrame
styled_table = results_table.style.set_properties(**{'text-align': 'center'}).set_table_styles([
    {'selector': 'th',
     'props': [('text-align', 'center')]}
])

# Display the styled DataFrame
styled_table
```

Out [57]:

	Approach	Learning Rate	Lambda Value	RMSE	R^2
0	Gradient Descent	0.0001	0.001	139.9	0.39
1	Gradient Descent	0.001	0.01	139.7	0.38
2	Gradient Descent	0.01	0.1	69.01	1.70
3	Stochastic Gradient Descent	0.0001	0.001	134.3	0.46
4	Stochastic Gradient Descent	0.001	0.01	99.01	1.03
5	Stochastic Gradient Descent	0.01	0.1	72.63	1.61

In [58]: *#Linear Regression with Stochastic Gradient Descent and Regularization*

X_train

Out[58]: array([[-0.47995445, 0.32725972, -0.76573124, ..., 2.39970032,
 -0.41532195, -0.07711562],
 [-0.47995445, 1.77207107, -0.45246966, ..., -0.4167187 ,
 -0.41532195, -0.07711562],
 [0.7671688 , -0.49565457, -1.52941219, ..., -0.4167187 ,
 -0.41532195, -0.07711562],
 ...,
 [-0.47995445, 0.0697064 , -0.4355366 , ..., 2.39970032,
 -0.41532195, -0.07711562],
 [2.01429204, -0.11246547, 0.34931068, ..., 2.39970032,
 -0.41532195, -0.07711562],
 [-0.47995445, -0.67782643, -1.65471683, ..., -0.4167187 ,
 -0.41532195, -0.07711562]])

In [59]: *#Ridge Regression*

```
@dataclass
class RidgeRegression:
    """
    epsilon: Tolerance
    maxIteration: Maximum number of Iteration before the loop breaks.
    gd: Picks the option for gradient descent
        1: Gradient Descent
        2: Stochastic Descent
    lambda_value:
        0: Means no Regularization
        >0: Means Regularization
```

```

"""
X : np.array
y : np.array
learningRate : float
epsilon : float
errors : list = field(default_factory=list)
maxIteration : int = 50000
gd : int = 0
lambda_value : int = 0
sample_size : int = 100
s : int = 0

def splitData(self):
    X_train, X_test, y_train, y_test = train_test_split(self.X,
                                                         self.y,
                                                         test_size=0.3,
                                                         shuffle=True)

    return X_train, X_test, y_train, y_test

def addX0(self, X):

    return np.column_stack([np.ones(X.shape[0]), X])

def normalizeTrain(self, X):
    """Z score normalization on the training data set
    Args:
        X: Dataset
    Returns:
        X_norm: Normalized z score Training dataset
        mean: Mean of Training data
        std: Standard deviation of Training data
    """
    mean = np.mean(X, axis=0)
    std = np.std(X, axis=0)
    X_norm = (X - mean)/std
    X_norm = self.addX0(X_norm)
    return X_norm, mean, std

def predict(self, X):
    return X.dot(self.w)

def sse(self, X, y):

```

```

y_hat = self.predict(X)

return ((y_hat - y) ** 2).sum() # Norm 2

def costFunction(self, X, y):
    sse = self.sse(X, y)
    return sse/2 # Division by 2 to remove the derived power of 2

def rmse(self, X, y):
    return np.sqrt((self.sse(X, y))/ X.shape[0])

def r2_squared(self, X, y):
    y_hat = self.predict(X)
    return (np.exp(r2_score(y,y_hat)))

def error_report(self, X, y):
    print()
    print("SSE: {}".format(self.costFunction(X,y)))
    print("RMSE: {}".format(self.rmse(X, y)))
    print("R^2: {}".format(self.r2_squared(X, y)))

def costDerivative(self, X, y):
    y_hat = self.predict(X)
    m = y.shape[0]
    return 1/m * X.T.dot(y_hat - y)

def gradientDescent(self, X, y):
    self.errors = []
    prev_error = float(np.inf)
    for i in tqdm(range(self.maxIteration)):
        grad = self.costDerivative(X, y)
        # adding regularization
        linear_reg_term = self.lambda_value * self.w
        self.w = self.w - self.learningRate * (grad + linear_reg_term)
        current_error = self.rmse(X, y)
        if math.isnan(current_error):
            print("Out of Domain: Values Tending to Infinite")
            break
        self.errors.append(current_error)
        if abs(current_error - prev_error) < self.epsilon:
            print("The model stopped learning")
            self.s = i+1

```

```
        break
    prev_error = current_error
    # Showing error metrics
    self.error_report(X,y)

# Creating function for stochastic gradient
def stochasticgradientDescent(self, X, y, sample_size):

    self.errors = []
    prev_error = float(np.inf)
    for i in tqdm(range(self.maxIteration)):

        index = np.random.choice(np.arange(len(X)), self.sample_size, replace = False)
        X_sample = X[index]
        y_sample = y[index]
        grad_sample = self.costDerivative(X_sample, y_sample)
        # adding regularization term
        linear_reg_term = self.lambda_value * self.w
        self.w = self.w - self.learningRate * (grad_sample + linear_reg_term)
        current_error = self.rmse(X, y)
        self.errors.append(current_error)
        if abs(current_error - prev_error) < self.epsilon:
            print("The model stopped learning")
            self.s = i+1
            break
        prev_error = current_error

    # Showing error metrics
    self.error_report(X,y)

def mse_var_bias(self, X, y):
    mse = self.costFunction(X, y) / len(y)
    y_hat = self.predict(X)
    mse_bias = np.mean((y - np.mean(y_hat)) ** 2)
    mse_var = np.var(y_hat)
    print("MSE: {:.2f}".format(mse))
    print("Bias: {:.2f}".format(mse_bias))
    print("Variance: {:.2f}".format(mse_var))

def fit(self):
    X_train = self.addX0(self.X)
```



```
Weights: [ 2.90657208e-01  6.08018559e-02  1.73623214e-02  2.66402212e-02
 3.74072625e-02 -1.67218269e-02  2.64358282e-02  1.29620370e-02
 1.31763884e-02  8.18832127e-03  7.53360004e-03  4.73554439e-03
 3.68332925e-02 -2.51421771e-02  7.83253952e-03 -1.40484917e-02
 1.11473269e-02 -2.72253780e-02  3.42519221e-03 -1.38428693e-02
 3.16906929e-02 -8.94986182e-03 -4.55094171e-02  2.74735504e-02
 8.56438666e-05  6.93638158e-03 -3.99380266e-03 -3.99380266e-03
-3.82718986e-03 -1.95698756e-03  6.61785193e-02 -3.43540060e-03
 2.97514603e-03  1.66281610e-01  8.33655627e-03  2.03865216e-02
 7.77535136e-03 -3.40304683e-03 -1.05463813e-02 -1.38869159e-02
-7.97269315e-03  1.33295355e-01]
```

Solving using Gradient Descent

59/126

SSE: 105021473.09338245
RMSE: 133.18639384084787
R^2: 0.47605508863978485
For Test Data

SSE: 36581915.982246734
RMSE: 136.14906356871992
R²: 0.49171023038814304
For Validation Data

SSE: 37652661.051923685
RMSE: 138.12722028989532
R²: 0.4921460730574879

Bias Variance Tradeoff

MSE: 9268.28

Bias: 19337.60

Variance: 40.55

```
Weights: [ 5.65147064e+00  1.16319638e+00  2.63128344e-01  5.04480141e-01
 6.41041966e-01 -3.01281105e-01  4.51841316e-01  2.58576577e-01
 1.87247396e-01  1.51052022e-01  8.23678348e-02  9.16980914e-02
 6.29296925e-01 -4.14667391e-01  8.78342267e-02 -2.73870656e-01
 1.61464014e-01 -5.10115991e-01  5.35464348e-03 -2.59769104e-01
 5.27095062e-01 -1.65208834e-01 -8.05962696e-01  5.08679533e-01
 1.42339942e-02  8.45011789e-02 -7.07063126e-02 -7.07063126e-02
-1.28119805e-01 -3.16749318e-02  1.22429740e+00 -1.19942020e-01
 6.73298571e-02  3.22253002e+00  1.60379893e-01  3.80735391e-01
 1.52226172e-01 -6.46290914e-02 -2.00184012e-01 -2.64549520e-01
-1.50805181e-01  2.59457405e+00]
```

```
In [62]: ridge = RidgeRegression(X = X_train, # All columns except Appliances
                                y = np.array(y_train), # Appliances column
                                learningRate=0.01,
                                maxIteration = 60,
                                epsilon=0.001,
                                gd=1,
                                lambda_value=0.1
                                ).fit()
```

Solving using Gradient Descent

[illegible]

SSE: 52968702.810781136
 RMSE: 94.58678758280433
 R²: 1.1289543595813567
 For Test Data

SSE: 18738944.78031519
 RMSE: 97.44375257185615
 R²: 1.1321606856742013
 For Validation Data

SSE: 19164773.610180676
 RMSE: 98.54470218096701
 R²: 1.1389979489089441

Bias Variance Tradeoff

MSE: 4747.64

Bias: 13788.14

Variance: 1466.68

Weights: [4.27829465e+01 7.42306367e+00 4.83480386e-01 3.44336718e+00
 2.53136825e+00 -1.25485114e+00 2.69944021e+00 1.81291788e+00
 2.72168177e-01 6.42204747e-01 -3.13370237e-01 3.04330899e-01
 2.42977398e+00 -1.47644724e+00 -2.36213197e-01 -2.04668933e+00
 5.95697572e-01 -3.25609344e+00 -6.15082780e-01 -1.54248162e+00
 1.51587698e+00 -1.15636975e+00 -3.31484771e+00 2.29378500e+00
 4.67924308e-01 -3.20359165e-01 -2.55006731e-01 -2.55006731e-01
 -1.40960681e+00 2.31962810e-02 6.30534244e+00 -1.34181454e+00
 6.49929665e-01 2.37491003e+01 1.07083606e+00 2.15002055e+00
 1.13438489e+00 -3.73107419e-01 -1.27318937e+00 -1.68086077e+00
 -9.41333351e-01 1.98229272e+01]

In [63]: `# import pandas as pd`

```
dff = [
    {"Approach": "Gradient Descent", "Learning Rate": "0.0001", "Lambda Value": "0.001", "RMSE": "139.6", "R^2": "0.40"},
    {"Approach": "Gradient Descent", "Learning Rate": "0.001", "Lambda Value": "0.01", "RMSE": "133.04", "R^2": "0.47"},
    {"Approach": "Gradient Descent", "Learning Rate": "0.01", "Lambda Value": "0.1", "RMSE": "94.06", "R^2": "1.13"},
    {"Approach": "Stochastic Gradient Descent", "Learning Rate": "0.0001", "Lambda Value": "0.001", "RMSE": "139.65", "R^2": "0.40"},
    {"Approach": "Stochastic Gradient Descent", "Learning Rate": "0.001", "Lambda Value": "0.01", "RMSE": "133.04", "R^2": "0.47"},
    {"Approach": "Stochastic Gradient Descent", "Learning Rate": "0.01", "Lambda Value": "0.1", "RMSE": "94.06", "R^2": "1.13"}
]
```

`# Set the float format to scientific notation`

`pd.options.display.float_format = '{:.4e}'.format`

```
# Create a pandas DataFrame with the data
results_table = pd.DataFrame(dff)

# Center-align the values and the column names in the DataFrame
styled_table = results_table.style.set_properties(**{'text-align': 'center'}).set_table_styles([
    {'selector': 'th',
     'props': [('text-align', 'center')]}
])

# Display the styled DataFrame
styled_table
```

Out[63]:

	Approach	Learning Rate	Lambda Value	RMSE	R ²
0	Gradient Descent	0.0001	0.001	139.6	0.40
1	Gradient Descent	0.001	0.01	133.04	0.47
2	Gradient Descent	0.01	0.1	94.06	1.13
3	Stochastic Gradient Descent	0.0001	0.001	139.65	0.40
4	Stochastic Gradient Descent	0.001	0.01	133.04	0.47
5	Stochastic Gradient Descent	0.01	0.1	94.06	1.13

In [64]:

```
@dataclass
class LassoRegression:
    """
    epsilon: Tolerance
    maxIteration: Maximum number of Iteration before the loop breaks.
    gd: Picks the option for gradient descent
        1: Gradient Descent
        2: Stochastic Descent
    alpha:
        0: Means no Regularization
        >0: Means Regularization

    """
    X : np.array
    y : np.array
    learningRate : float
    epsilon : float
    errors : list = field(default_factory=list)
```

```

maxIteration : int = 50000
gd : int = 0
lambda_value : int = 0
alpha : int = 0
sample_size : int = 100
s : int = 0

def splitData(self):
    X_train, X_test, y_train, y_test = train_test_split(self.X,
                                                         self.y,
                                                         test_size=0.3,
                                                         shuffle=True)

    return X_train, X_test, y_train, y_test

def addX0(self, X):

    return np.column_stack([np.ones(X.shape[0]), X])

def normalizeTrain(self, X):
    """Z score normalization on the training data set
    Args:
        X: Dataset
    Returns:
        X_norm: Normalized z score Training dataset
        mean: Mean of Training data
        std: Standard deviation of Training data
    """
    mean = np.mean(X, axis=0)
    std = np.std(X, axis=0)
    X_norm = (X - mean)/std
    X_norm = self.addX0(X_norm)
    return X_norm, mean, std

def predict(self, X):
    return X.dot(self.w)

def sse(self, X, y):
    y_hat = self.predict(X)

    return ((y_hat - y) ** 2).sum() # Norm 2

def costFunction(self, X, y):
    sse = self.sse(X, y)

```

```

    return sse/2 # Division by 2 to remove the derived power of 2

def rmse(self, X, y):
    return np.sqrt((self.sse(X, y))/ X.shape[0])

def r2_squared(self, X, y):
    y_hat = self.predict(X)
    return (np.exp(r2_score(y,y_hat)))

def error_report(self, X, y):
    print()
    print("SSE: {}".format(self.costFunction(X,y)))
    print("RMSE: {}".format(self.rmse(X, y)))
    print("R^2: {}".format(self.r2_squared(X, y)))

def costDerivative(self, X, y):
    y_hat = self.predict(X)
    m = y.shape[0]
    return 1/m * X.T.dot(y_hat - y)

def gradientDescent(self, X, y):
    self.errors = []
    prev_error = float(np.inf)
    for i in tqdm(range(self.maxIteration)):
        grad = self.costDerivative(X, y)
        # adding regularization
        lasso_reg_term = self.lambda_value * np.sign(self.w)
        self.w = self.w - self.learningRate * (grad + lasso_reg_term)
        current_error = self.rmse(X, y)
        if math.isnan(current_error):
            print("Out of Domain: Values Tending to Infinite")
            break
        self.errors.append(current_error)
        if abs(current_error - prev_error) < self.epsilon:
            print("The model stopped learning")
            self.s = i+1
            break
        prev_error = current_error
    # Showing error metrics
    self.error_report(X,y)

```

Creating function for stochastic gradient

```
def stochasticgradientDescent(self, X, y, sample_size):

    self.errors = []
    prev_error = float(np.inf)
    for i in tqdm(range(self.maxIteration)):

        index = np.random.choice(np.arange(len(X)), self.sample_size, replace=False)
        X_sample = X[index]
        y_sample = y[index]
        grad_sample = self.costDerivative(X_sample, y_sample)
        # adding regularization term
        lasso_reg_term = self.lambda_value * np.sign(self.w)
        self.w = self.w - self.learningRate * (grad_sample + lasso_reg_term)
        current_error = self.rmse(X, y)
        self.errors.append(current_error)
        if abs(current_error - prev_error) < self.epsilon:
            print("The model stopped learning")
            self.s = i+1
            break
        prev_error = current_error

    # Showing error metrics
    self.error_report(X,y)

def mse_var_bias(self, X, y):
    mse = self.costFunction(X, y) / len(y)
    y_hat = self.predict(X)
    mse_bias = np.mean((y - np.mean(y_hat)) ** 2)
    mse_var = np.var(y_hat)
    print("MSE: {:.2f}".format(mse))
    print("Bias: {:.2f}".format(mse_bias))
    print("Variance: {:.2f}".format(mse_var))

def fit(self):
    X_train = self.addX0(self.X)
    y_train = self.y

    if self.gd == 1:
        print("Solving using Gradient Descent")
        self.w = np.zeros(X_train.shape[1])
        self.gradientDescent(X_train, y_train)
```



```
Bias Variance Tradeoff
MSE: 10053.82
Bias: 20252.61
Variance: 1.36
```

Solving using Gradient Descent

[illegible]

```
Bias Variance Tradeoff
MSE: 8706.68
Bias: 18671.85
Variance: 99.63
```

Solving using Gradient Descent

```
s] |██████████| 100% | 100/100 [00:00<00:00, 2619.84it/s]
```


SSE: 37513750.00401126
 RMSE: 79.60054609913603
 R²: 1.4588917688367082
 For Test Data

SSE: 13462853.971645763
 RMSE: 82.59428581000651
 R²: 1.4488009580495291
 For Validation Data

SSE: 13589566.954170223
 RMSE: 82.98206604815026
 R²: 1.466966218832527

Bias Variance Tradeoff
 MSE: 3410.91
 Bias: 12091.05
 Variance: 2684.00

```
In [68]: # import pandas as pd

dff = [
    {"Approach": "Gradient Descent", "Learning Rate": "0.0001", "Lambda Value": "0.001", "RMSE": "138.7", "R^2": "0.40"},
    {"Approach": "Gradient Descent", "Learning Rate": "0.001", "Lambda Value": "0.01", "RMSE": "129.02", "R^2": "0.52"},
    {"Approach": "Gradient Descent", "Learning Rate": "0.01", "Lambda Value": "0.1", "RMSE": "79.6", "R^2": "1.45"},
    {"Approach": "Stochastic Gradient Descent", "Learning Rate": "0.0001", "Lambda Value": "0.001", "RMSE": "138.7", "R^2": "0.40"},
    {"Approach": "Stochastic Gradient Descent", "Learning Rate": "0.001", "Lambda Value": "0.01", "RMSE": "129.4", "R^2": "0.52"},
    {"Approach": "Stochastic Gradient Descent", "Learning Rate": "0.01", "Lambda Value": "0.1", "RMSE": "79.82", "R^2": "1.45"}
]

# Set the float format to scientific notation
pd.options.display.float_format = '{:.4e}'.format

# Create a pandas DataFrame with the data
results_table = pd.DataFrame(dff)

# Center-align the values and the column names in the DataFrame
styled_table = results_table.style.set_properties(**{'text-align': 'center'}).set_table_styles([
    {'selector': 'th',
     'props': [('text-align', 'center')]}
])
```

```
# Display the styled DataFrame
styled_table
```

Out [68]:

	Approach	Learning Rate	Lambda Value	RMSE	R ²
0	Gradient Descent	0.0001	0.001	138.7	0.40
1	Gradient Descent	0.001	0.01	129.02	0.52
2	Gradient Descent	0.01	0.1	79.6	1.45
3	Stochastic Gradient Descent	0.0001	0.001	138.7	0.41
4	Stochastic Gradient Descent	0.001	0.01	129.4	0.52
5	Stochastic Gradient Descent	0.01	0.1	79.82	1.45

In [69]:

```
dff = [
    {"Approach": "Linear Regression (Gradient Descent)", "Learning Rate": "0.0001", "Lambda Value": "0.001", "RMSE": "138.7", "R^2": "0.40"},
    {"Approach": "Linear Regression (Stochastic Gradient Descent)", "Learning Rate": "0.0001", "Lambda Value": "0.001", "RMSE": "138.7", "R^2": "0.41"},
    {"Approach": "Ridge Regression (Gradient Descent)", "Learning Rate": "0.001", "Lambda Value": "0.01", "RMSE": "129.02", "R^2": "0.52"},
    {"Approach": "Ridge Regression (Stochastic Gradient Descent)", "Learning Rate": "0.001", "Lambda Value": "0.01", "RMSE": "129.4", "R^2": "0.52"},
    {"Approach": "Lasso Regression (Gradient Descent)", "Learning Rate": "0.001", "Lambda Value": "0.01", "RMSE": "129.4", "R^2": "0.52"},
    {"Approach": "Lasso Regression (Stochastic Gradient Descent)", "Learning Rate": "0.001", "Lambda Value": "0.01", "RMSE": "129.4", "R^2": "0.52"},
    {"Approach": "Neural Nets", "Learning Rate": "-", "Lambda Value": "-", "RMSE": "2.99", "R^2": "0.96"}
]

# Set the float format to scientific notation
pd.options.display.float_format = '{:.4e}'.format

# Create a pandas DataFrame with the data
results_table = pd.DataFrame(dff)

# Center-align the values and the column names in the DataFrame
styled_table = results_table.style.set_properties(**{'text-align': 'left'}).set_table_styles([
    {'selector': 'th',
     'props': [('text-align', 'center')]}
])

# Display the styled DataFrame
styled_table
```

Out [69]:

	Approach	Learning Rate	Lambda Value	RMSE	R^2
0	Linear Regression (Gradient Descent)	0.0001	0.001	139.9	0.39
1	Linear Regression (Stochastic Gradient Descent)	0.0001	0.001	134.3	0.46
2	Ridge Regression (Gradient Descent)	0.001	0.01	133.04	0.47
3	Ridge Regression (Stochastic Gradient Descent)	0.001	0.01	133.04	0.47
4	Lasso Regression (Gradient Descent)	0.001	0.01	129.02	0.52
5	Lasso Regression (Stochastic Gradient Descent)	0.001	0.01	129.4	0.52
6	Neural Nets	-	-	2.99	0.96

```

In [70]: import numpy as np
import pandas as pd
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout

# Load the data
neural_data = pd.read_csv('energydata_complete.csv')

# Preprocess data
neural_data.drop(columns=['date'], inplace=True) # drop non-numeric date column
X_neural = neural_data.iloc[:, :-1].values # features
y_neural = neural_data.iloc[:, -1].values.reshape(-1, 1) # target

# Convert string data to float
X_neural = np.concatenate([X_neural[:, :2], X_neural[:, 3:]], axis=1)
X_neural = np.concatenate([X_neural[:, :8], X_neural[:, 9:]], axis=1)
X_neural = X_neural.astype(np.float32)

# Define the model
model = Sequential()
model.add(Dense(64, input_shape=(X_neural.shape[1],), activation='relu'))
model.add(Dropout(0.2))
model.add(Dense(32, activation='relu'))
model.add(Dropout(0.2))
model.add(Dense(1, activation='linear'))

# Compile the model
model.compile(loss='mean_squared_error', optimizer='adam')

```

```
# Train the model
model.fit(X_neural, y_neural, epochs=100, batch_size=32, validation_split=0.4)

# Make predictions
y_pred_neural = model.predict(X_neural)

# Create a DataFrame to hold predictions
predictions_df = pd.DataFrame({
    'Actual': y_neural.flatten(),
    'Predicted': y_pred_neural.flatten()
})

# Save to CSV
predictions_df.to_csv('predictions.csv', index=False)

print("Predictions saved to 'predictions.csv'")
```

Epoch 1/100
371/371  1s 1ms/step - loss: 1043.7106 - val_loss: 194.6398

Epoch 2/100
371/371  0s 927us/step - loss: 215.6583 - val_loss: 98.5640

Epoch 3/100
371/371  0s 894us/step - loss: 99.5333 - val_loss: 60.5037

Epoch 4/100
371/371  0s 920us/step - loss: 61.2740 - val_loss: 20.6345

Epoch 5/100
371/371  0s 900us/step - loss: 47.1989 - val_loss: 25.4356

Epoch 6/100
371/371  0s 868us/step - loss: 39.6766 - val_loss: 14.3433

Epoch 7/100
371/371  0s 852us/step - loss: 37.6860 - val_loss: 16.6726

Epoch 8/100
371/371  0s 868us/step - loss: 31.3451 - val_loss: 19.3555

Epoch 9/100
371/371  0s 869us/step - loss: 29.5494 - val_loss: 3.1825

Epoch 10/100
371/371  0s 880us/step - loss: 27.6711 - val_loss: 16.7494

Epoch 11/100
371/371  0s 873us/step - loss: 24.5627 - val_loss: 31.1620

Epoch 12/100
371/371  0s 899us/step - loss: 24.2898 - val_loss: 40.7825

Epoch 13/100
371/371  0s 881us/step - loss: 25.4571 - val_loss: 19.2350

Epoch 14/100
371/371  0s 895us/step - loss: 20.9181 - val_loss: 19.7000

Epoch 15/100
371/371  0s 833us/step - loss: 20.7982 - val_loss: 22.4505

Epoch 16/100
371/371  0s 908us/step - loss: 19.9185 - val_loss: 23.9159

Epoch 17/100
371/371  0s 844us/step - loss: 20.1462 - val_loss: 24.0796

Epoch 18/100
371/371  0s 863us/step - loss: 19.8994 - val_loss: 30.0173

Epoch 19/100
371/371  0s 851us/step - loss: 17.4924 - val_loss: 31.7411

Epoch 20/100
371/371  0s 921us/step - loss: 17.0423 - val_loss: 19.2610

Epoch 21/100
371/371  0s 919us/step - loss: 16.1197 - val_loss: 35.6497

Epoch 22/100
371/371  0s 936us/step - loss: 15.4246 - val_loss: 22.4053

Epoch 23/100
371/371  **0s** 869us/step - loss: 15.4013 - val_loss: 20.5821

Epoch 24/100
371/371  **0s** 919us/step - loss: 14.5944 - val_loss: 27.4452

Epoch 25/100
371/371  **0s** 896us/step - loss: 14.3525 - val_loss: 36.1783

Epoch 26/100
371/371  **0s** 956us/step - loss: 13.8082 - val_loss: 36.5861

Epoch 27/100
371/371  **0s** 909us/step - loss: 14.2159 - val_loss: 13.4660

Epoch 28/100
371/371  **0s** 912us/step - loss: 15.1474 - val_loss: 21.0488

Epoch 29/100
371/371  **0s** 964us/step - loss: 13.7589 - val_loss: 34.1036

Epoch 30/100
371/371  **0s** 893us/step - loss: 15.4306 - val_loss: 41.2735

Epoch 31/100
371/371  **0s** 921us/step - loss: 13.4850 - val_loss: 18.2911

Epoch 32/100
371/371  **0s** 913us/step - loss: 12.3898 - val_loss: 12.9925

Epoch 33/100
371/371  **0s** 919us/step - loss: 12.1499 - val_loss: 19.9550

Epoch 34/100
371/371  **0s** 897us/step - loss: 11.9167 - val_loss: 20.5692

Epoch 35/100
371/371  **0s** 900us/step - loss: 11.1503 - val_loss: 42.5441

Epoch 36/100
371/371  **0s** 876us/step - loss: 12.9303 - val_loss: 38.8855

Epoch 37/100
371/371  **0s** 895us/step - loss: 11.3929 - val_loss: 17.7084

Epoch 38/100
371/371  **0s** 881us/step - loss: 10.8745 - val_loss: 17.2683

Epoch 39/100
371/371  **0s** 957us/step - loss: 10.4640 - val_loss: 16.5432

Epoch 40/100
371/371  **0s** 927us/step - loss: 11.3384 - val_loss: 40.8775

Epoch 41/100
371/371  **0s** 912us/step - loss: 12.1166 - val_loss: 31.5557

Epoch 42/100
371/371  **0s** 907us/step - loss: 10.6171 - val_loss: 23.9411

Epoch 43/100
371/371  **0s** 912us/step - loss: 10.0470 - val_loss: 22.5630

Epoch 44/100
371/371  **0s** 926us/step - loss: 10.5288 - val_loss: 20.0647

Epoch 45/100
371/371  0s 892us/step - loss: 10.0899 - val_loss: 35.0373

Epoch 46/100
371/371  0s 883us/step - loss: 10.7078 - val_loss: 30.4543

Epoch 47/100
371/371  0s 922us/step - loss: 9.7672 - val_loss: 21.8734

Epoch 48/100
371/371  0s 900us/step - loss: 9.8280 - val_loss: 25.9221

Epoch 49/100
371/371  0s 981us/step - loss: 10.2347 - val_loss: 22.1269

Epoch 50/100
371/371  0s 926us/step - loss: 9.8699 - val_loss: 19.2913

Epoch 51/100
371/371  0s 905us/step - loss: 9.2835 - val_loss: 22.1880

Epoch 52/100
371/371  0s 929us/step - loss: 9.3692 - val_loss: 35.4288

Epoch 53/100
371/371  0s 950us/step - loss: 9.7959 - val_loss: 14.6338

Epoch 54/100
371/371  0s 951us/step - loss: 9.6780 - val_loss: 14.3671

Epoch 55/100
371/371  0s 959us/step - loss: 9.1199 - val_loss: 24.1476

Epoch 56/100
371/371  0s 935us/step - loss: 9.0796 - val_loss: 20.3457

Epoch 57/100
371/371  0s 907us/step - loss: 9.1405 - val_loss: 17.8261

Epoch 58/100
371/371  0s 931us/step - loss: 8.8855 - val_loss: 23.5801

Epoch 59/100
371/371  0s 942us/step - loss: 9.2885 - val_loss: 21.8126

Epoch 60/100
371/371  0s 910us/step - loss: 8.5720 - val_loss: 23.9480

Epoch 61/100
371/371  0s 869us/step - loss: 8.8894 - val_loss: 24.0861

Epoch 62/100
371/371  0s 902us/step - loss: 9.3318 - val_loss: 18.8278

Epoch 63/100
371/371  0s 898us/step - loss: 8.5507 - val_loss: 13.9047

Epoch 64/100
371/371  0s 921us/step - loss: 8.4337 - val_loss: 19.7621

Epoch 65/100
371/371  0s 881us/step - loss: 9.3291 - val_loss: 36.5301

Epoch 66/100
371/371  0s 905us/step - loss: 9.5361 - val_loss: 18.6813

Epoch 67/100
371/371  **0s** 896us/step - loss: 8.1045 - val_loss: 18.6037

Epoch 68/100
371/371  **0s** 954us/step - loss: 8.4953 - val_loss: 15.1071

Epoch 69/100
371/371  **0s** 900us/step - loss: 8.6406 - val_loss: 18.2387

Epoch 70/100
371/371  **0s** 893us/step - loss: 7.7753 - val_loss: 17.1356

Epoch 71/100
371/371  **0s** 887us/step - loss: 8.2683 - val_loss: 17.0320

Epoch 72/100
371/371  **0s** 886us/step - loss: 8.3175 - val_loss: 13.3435

Epoch 73/100
371/371  **0s** 920us/step - loss: 7.6704 - val_loss: 10.7790

Epoch 74/100
371/371  **0s** 926us/step - loss: 8.2000 - val_loss: 21.3020

Epoch 75/100
371/371  **0s** 897us/step - loss: 7.5797 - val_loss: 15.6639

Epoch 76/100
371/371  **0s** 904us/step - loss: 7.7896 - val_loss: 20.3205

Epoch 77/100
371/371  **0s** 915us/step - loss: 7.4312 - val_loss: 21.3647

Epoch 78/100
371/371  **0s** 927us/step - loss: 7.5457 - val_loss: 20.1393

Epoch 79/100
371/371  **0s** 905us/step - loss: 8.7714 - val_loss: 12.7668

Epoch 80/100
371/371  **0s** 876us/step - loss: 8.4024 - val_loss: 8.7411

Epoch 81/100
371/371  **0s** 897us/step - loss: 7.8195 - val_loss: 18.6220

Epoch 82/100
371/371  **0s** 919us/step - loss: 7.3069 - val_loss: 12.6346

Epoch 83/100
371/371  **0s** 946us/step - loss: 7.5503 - val_loss: 24.1109

Epoch 84/100
371/371  **0s** 915us/step - loss: 7.2694 - val_loss: 16.0964

Epoch 85/100
371/371  **0s** 903us/step - loss: 7.4033 - val_loss: 22.1291

Epoch 86/100
371/371  **0s** 932us/step - loss: 7.2733 - val_loss: 28.1595

Epoch 87/100
371/371  **0s** 909us/step - loss: 7.9987 - val_loss: 12.0413

Epoch 88/100
371/371  **0s** 995us/step - loss: 8.2094 - val_loss: 16.8672


```

Epoch 89/100
371/371  0s 935us/step - loss: 7.3690 - val_loss: 10.7107
Epoch 90/100
371/371  0s 902us/step - loss: 7.8961 - val_loss: 12.7769
Epoch 91/100
371/371  0s 907us/step - loss: 7.4229 - val_loss: 15.6211
Epoch 92/100
371/371  0s 938us/step - loss: 7.7013 - val_loss: 7.4841
Epoch 93/100
371/371  0s 935us/step - loss: 9.6015 - val_loss: 13.0779
Epoch 94/100
371/371  0s 883us/step - loss: 9.0649 - val_loss: 17.2618
Epoch 95/100
371/371  0s 949us/step - loss: 7.5717 - val_loss: 21.3506
Epoch 96/100
371/371  0s 914us/step - loss: 7.6988 - val_loss: 15.8509
Epoch 97/100
371/371  0s 956us/step - loss: 7.5375 - val_loss: 20.0813
Epoch 98/100
371/371  0s 888us/step - loss: 7.6157 - val_loss: 15.7708
Epoch 99/100
371/371  0s 956us/step - loss: 7.4999 - val_loss: 18.2832
Epoch 100/100
371/371  0s 953us/step - loss: 7.2098 - val_loss: 16.0817
617/617  0s 497us/step
Predictions saved to 'predictions.csv'

```

```

In [71]: # Calculate R-square, RMSE, SSE
r2 = r2_score(y_neural, y_pred_neural)
rmse = mean_squared_error(y_neural, y_pred_neural, squared=False)
sse = mean_squared_error(y_neural, y_pred_neural, squared=True) * len(y_neural)
print('R2: {:.2f}'.format(r2))
print("RMSE: {:.2f}".format(rmse))
print("SSE: {:.2f}".format(sse))

```

```

R2: 0.92
RMSE: 4.04
SSE: 321764.33

```

```

In [82]: import numpy as np
import pandas as pd
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout

```

```
# Load the data
neural_data = pd.read_csv('energydata_complete.csv')

# Preprocess data
neural_data.drop(columns=['date'], inplace=True) # drop non-numeric date column
X_neural = neural_data.iloc[:, :-1].values # features
y_neural = neural_data.iloc[:, -1].values.reshape(-1, 1) # target

# Conv
```

Epoch 1/100
371/371  **2s** 2ms/step - loss: 2814.2034 - val_loss: 204.7134

Epoch 2/100
371/371  **1s** 1ms/step - loss: 253.1375 - val_loss: 115.4348

Epoch 3/100
371/371  **1s** 1ms/step - loss: 171.5621 - val_loss: 29.4348

Epoch 4/100
371/371  **1s** 2ms/step - loss: 92.3917 - val_loss: 44.5641

Epoch 5/100
371/371  **1s** 2ms/step - loss: 69.2663 - val_loss: 20.4805

Epoch 6/100
371/371  **1s** 1ms/step - loss: 57.9414 - val_loss: 24.1155

Epoch 7/100
371/371  **1s** 2ms/step - loss: 50.6242 - val_loss: 21.4346

Epoch 8/100
371/371  **1s** 1ms/step - loss: 46.2735 - val_loss: 21.8983

Epoch 9/100
371/371  **1s** 1ms/step - loss: 41.9064 - val_loss: 21.1056

Epoch 10/100
371/371  **1s** 1ms/step - loss: 37.6299 - val_loss: 20.7974

Epoch 11/100
371/371  **1s** 1ms/step - loss: 36.0660 - val_loss: 23.9125

Epoch 12/100
371/371  **1s** 1ms/step - loss: 35.3250 - val_loss: 25.8242

Epoch 13/100
371/371  **1s** 1ms/step - loss: 33.6053 - val_loss: 29.7547

Epoch 14/100
371/371  **1s** 1ms/step - loss: 31.5557 - val_loss: 19.8525

Epoch 15/100
371/371  **1s** 1ms/step - loss: 29.6880 - val_loss: 23.1090

Epoch 16/100
371/371  **1s** 1ms/step - loss: 27.3281 - val_loss: 18.8292

Epoch 17/100
371/371  **1s** 1ms/step - loss: 26.9915 - val_loss: 15.0606

Epoch 18/100
371/371  **1s** 1ms/step - loss: 26.4991 - val_loss: 9.5518

Epoch 19/100
371/371  **1s** 1ms/step - loss: 24.6701 - val_loss: 8.6263

Epoch 20/100
371/371  **1s** 1ms/step - loss: 23.9698 - val_loss: 9.7037

Epoch 21/100
371/371  **1s** 1ms/step - loss: 21.8603 - val_loss: 12.5914

Epoch 22/100
371/371  **1s** 1ms/step - loss: 20.8672 - val_loss: 14.3323

Epoch 23/100
371/371  **1s** 1ms/step - loss: 20.7818 - val_loss: 2.6352

Epoch 24/100
371/371  **1s** 2ms/step - loss: 20.4779 - val_loss: 8.7789

Epoch 25/100
371/371  **1s** 2ms/step - loss: 19.7379 - val_loss: 20.3400

Epoch 26/100
371/371  **1s** 2ms/step - loss: 18.3562 - val_loss: 7.2952

Epoch 27/100
371/371  **1s** 2ms/step - loss: 17.4788 - val_loss: 11.8962

Epoch 28/100
371/371  **1s** 1ms/step - loss: 16.9568 - val_loss: 17.2930

Epoch 29/100
371/371  **1s** 2ms/step - loss: 16.3137 - val_loss: 9.1836

Epoch 30/100
371/371  **1s** 1ms/step - loss: 16.5628 - val_loss: 20.4907

Epoch 31/100
371/371  **1s** 1ms/step - loss: 15.4750 - val_loss: 4.8812

Epoch 32/100
371/371  **1s** 1ms/step - loss: 15.3872 - val_loss: 10.8305

Epoch 33/100
371/371  **1s** 1ms/step - loss: 14.9454 - val_loss: 19.0156

Epoch 34/100
371/371  **1s** 1ms/step - loss: 13.9824 - val_loss: 2.3854

Epoch 35/100
371/371  **1s** 1ms/step - loss: 13.9282 - val_loss: 5.9499

Epoch 36/100
371/371  **1s** 1ms/step - loss: 13.5670 - val_loss: 15.1196

Epoch 37/100
371/371  **1s** 1ms/step - loss: 12.0613 - val_loss: 11.9746

Epoch 38/100
371/371  **1s** 1ms/step - loss: 13.3184 - val_loss: 1.2025

Epoch 39/100
371/371  **1s** 1ms/step - loss: 12.3006 - val_loss: 4.6480

Epoch 40/100
371/371  **1s** 1ms/step - loss: 12.1553 - val_loss: 7.3929

Epoch 41/100
371/371  **1s** 1ms/step - loss: 12.7970 - val_loss: 6.1075

Epoch 42/100
371/371  **1s** 1ms/step - loss: 11.6192 - val_loss: 3.5150

Epoch 43/100
371/371  **1s** 1ms/step - loss: 11.1686 - val_loss: 2.3266

Epoch 44/100
371/371  **1s** 1ms/step - loss: 12.1410 - val_loss: 10.2233

Epoch 45/100
371/371  **1s** 1ms/step - loss: 11.1145 - val_loss: 2.4668

Epoch 46/100
371/371  **1s** 1ms/step - loss: 10.8032 - val_loss: 11.8179

Epoch 47/100
371/371  **1s** 2ms/step - loss: 11.0138 - val_loss: 5.0780

Epoch 48/100
371/371  **1s** 1ms/step - loss: 11.2139 - val_loss: 9.0516

Epoch 49/100
371/371  **1s** 1ms/step - loss: 11.2488 - val_loss: 1.7943

Epoch 50/100
371/371  **1s** 1ms/step - loss: 10.8982 - val_loss: 3.9361

Epoch 51/100
371/371  **1s** 1ms/step - loss: 10.3595 - val_loss: 5.3463

Epoch 52/100
371/371  **1s** 1ms/step - loss: 10.5524 - val_loss: 2.8457

Epoch 53/100
371/371  **1s** 1ms/step - loss: 10.5931 - val_loss: 2.4819

Epoch 54/100
371/371  **1s** 1ms/step - loss: 12.4846 - val_loss: 8.7855

Epoch 55/100
371/371  **1s** 1ms/step - loss: 10.0807 - val_loss: 2.8308

Epoch 56/100
371/371  **1s** 2ms/step - loss: 10.4965 - val_loss: 2.1114

Epoch 57/100
371/371  **1s** 2ms/step - loss: 9.9498 - val_loss: 3.7496

Epoch 58/100
371/371  **1s** 1ms/step - loss: 10.3696 - val_loss: 3.8387

Epoch 59/100
371/371  **1s** 1ms/step - loss: 9.4398 - val_loss: 0.7422

Epoch 60/100
371/371  **1s** 1ms/step - loss: 10.2327 - val_loss: 10.7618

Epoch 61/100
371/371  **1s** 1ms/step - loss: 10.8367 - val_loss: 1.8981

Epoch 62/100
371/371  **1s** 1ms/step - loss: 9.9169 - val_loss: 0.8016

Epoch 63/100
371/371  **1s** 1ms/step - loss: 11.0416 - val_loss: 9.0687

Epoch 64/100
371/371  **1s** 1ms/step - loss: 9.3001 - val_loss: 6.7540

Epoch 65/100
371/371  **1s** 2ms/step - loss: 9.5228 - val_loss: 2.7130

Epoch 66/100
371/371  **1s** 1ms/step - loss: 9.3034 - val_loss: 10.6234

Epoch 67/100
371/371  **1s** 2ms/step - loss: 8.9495 - val_loss: 2.6784

Epoch 68/100
371/371  **1s** 1ms/step - loss: 9.1638 - val_loss: 4.3019

Epoch 69/100
371/371  **1s** 2ms/step - loss: 8.9512 - val_loss: 1.5885

Epoch 70/100
371/371  **1s** 2ms/step - loss: 9.3063 - val_loss: 10.5620

Epoch 71/100
371/371  **1s** 1ms/step - loss: 9.5529 - val_loss: 21.7600

Epoch 72/100
371/371  **1s** 2ms/step - loss: 9.9008 - val_loss: 12.9469

Epoch 73/100
371/371  **1s** 1ms/step - loss: 10.3853 - val_loss: 2.2969

Epoch 74/100
371/371  **1s** 1ms/step - loss: 8.2844 - val_loss: 0.8758

Epoch 75/100
371/371  **1s** 1ms/step - loss: 8.9444 - val_loss: 5.8502

Epoch 76/100
371/371  **1s** 1ms/step - loss: 8.4126 - val_loss: 0.8217

Epoch 77/100
371/371  **1s** 1ms/step - loss: 8.7244 - val_loss: 7.6608

Epoch 78/100
371/371  **1s** 1ms/step - loss: 8.7848 - val_loss: 2.0442

Epoch 79/100
371/371  **1s** 1ms/step - loss: 9.1666 - val_loss: 4.0779

Epoch 80/100
371/371  **1s** 1ms/step - loss: 8.1050 - val_loss: 1.0063

Epoch 81/100
371/371  **1s** 2ms/step - loss: 8.1957 - val_loss: 3.9679

Epoch 82/100
371/371  **1s** 1ms/step - loss: 8.2732 - val_loss: 1.3102

Epoch 83/100
371/371  **1s** 1ms/step - loss: 9.0190 - val_loss: 5.3849

Epoch 84/100
371/371  **1s** 1ms/step - loss: 7.9589 - val_loss: 3.6260

Epoch 85/100
371/371  **1s** 1ms/step - loss: 7.9872 - val_loss: 2.6500

Epoch 86/100
371/371  **1s** 1ms/step - loss: 8.9142 - val_loss: 7.0555

Epoch 87/100
371/371  **1s** 1ms/step - loss: 7.6158 - val_loss: 4.2860

Epoch 88/100
371/371  **1s** 1ms/step - loss: 8.3570 - val_loss: 2.5437

```

Epoch 89/100
371/371  1s 1ms/step - loss: 8.3117 - val_loss: 2.8762
Epoch 90/100
371/371  1s 1ms/step - loss: 7.6827 - val_loss: 6.5400
Epoch 91/100
371/371  1s 1ms/step - loss: 8.0555 - val_loss: 13.5961
Epoch 92/100
371/371  1s 1ms/step - loss: 7.8291 - val_loss: 1.3605
Epoch 93/100
371/371  1s 1ms/step - loss: 9.0795 - val_loss: 2.4972
Epoch 94/100
371/371  1s 2ms/step - loss: 7.6699 - val_loss: 0.9888
Epoch 95/100
371/371  1s 1ms/step - loss: 7.5083 - val_loss: 4.2721
Epoch 96/100
371/371  1s 1ms/step - loss: 7.4372 - val_loss: 0.8252
Epoch 97/100
371/371  1s 1ms/step - loss: 7.3364 - val_loss: 0.9107
Epoch 98/100
371/371  1s 1ms/step - loss: 7.1478 - val_loss: 1.1526
Epoch 99/100
371/371  1s 1ms/step - loss: 7.5455 - val_loss: 3.4043
Epoch 100/100
371/371  1s 1ms/step - loss: 8.5175 - val_loss: 4.3212
617/617  1s 851us/step
Predictions saved to 'predictions.csv'

```

```

In [84]: import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Load your data
neural_data = pd.read_csv('energydata_complete.csv')

# Preprocess data
neural_data.drop(columns=['date'], inplace=True) # Drop non-numeric date column
X = neural_data.iloc[:, :-1].values # Features
y = neural_data.iloc[:, -1].values # Target

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

```

```
# Create polynomial features
poly = PolynomialFeatures(degree=2) # Adjust degree as necessary
X_poly = poly.fit_transform(X_train)

# Fit polynomial regression model
poly_model = LinearRegression()
poly_model.fit(X_poly, y_train)

# Predict
X_test_poly = poly.transform(X_test)
y_pred_poly = poly_model.predict(X_test_poly)

# Calculate metrics
mse_poly = mean_squared_error(y_test, y_pred_poly)
rmse_poly = np.sqrt(mse_poly)
r2_poly = r2_score(y_test, y_pred_poly)

print("Polynomial Regression Metrics:")
print("MSE:", mse_poly)
print("RMSE:", rmse_poly)
print("R²:", r2_poly)
```

Polynomial Regression Metrics:
MSE: 1.4291145673082487e-24
RMSE: 1.1954557989772138e-12
R²: 1.0

```
In [72]: X_for_pca = data.drop(columns=['Appliances', 'date'], axis=1)
y_for_pca = data['Appliances']
```

```
In [73]: # Create a PCA object
pca = PCA()

# Fit the PCA model on the data
pca.fit(X_for_pca)

# Calculate the cumulative sum of explained variance
cumulative_variance = np.cumsum(pca.explained_variance_ratio_)

# Find the number of components that explain 95% of the variance
n_components = np.argmax(cumulative_variance >= 0.95) + 1

# Print the number of components
```



```
print("Number of components to keep:", n_components)

# Create a new PCA object with the desired number of components
pca_transformer = PCA(n_components=n_components)
```

Number of components to keep: 9

```
In [74]: # Scale the features
scaler1 = StandardScaler()
pca_scaled = scaler1.fit_transform(X_for_pca)
```

```
In [75]: # Fit the PCA model on the data and transform it
X_transformed = pca_transformer.fit_transform(pca_scaled)

# Print the shape of the transformed data
print(X_transformed.shape)
```

(19735, 9)

```
In [78]: # First, split the data into 60% training and 40% for testing and validation
X_train_pca, X_temp_pca, y_train_pca, y_temp_pca = train_test_split(X_transformed, y_for_pca, test_size=0.3, random_state=23)

# Then, split the testing and validation data equally into 50% each
X_test_pca, X_val_pca, y_test_pca, y_val_pca = train_test_split(X_temp_pca, y_temp_pca, test_size=0.5, random_state=23)
```

```
In [79]: pd.DataFrame(X_train_pca).head()
```

```
Out[79]:
```

	0	1	2	3	4	5	6	7	8
0	-6.8559e+00	2.1893e+00	1.9159e+00	-3.0446e-01	1.4527e+00	-1.8227e+00	7.7700e-01	1.2995e-01	-1.3725e-01
1	-4.0526e+00	3.3657e+00	2.7687e+00	8.0013e-01	9.6867e-01	-2.4659e+00	1.6391e+00	1.1102e+00	-1.9544e-01
2	-2.1915e+00	-2.7805e+00	2.9072e-01	2.1578e+00	1.4418e+00	-4.6083e-01	3.3978e-01	-2.6751e-01	-2.2697e+00
3	6.1465e-01	-8.7456e-01	-1.6257e-01	9.6899e-01	-8.6167e-02	7.4842e-01	-7.1536e-01	-1.9754e-01	-5.7640e-01
4	-1.7321e+00	-2.3171e+00	-2.6559e+00	1.0403e+00	2.2204e+00	-5.4800e-01	-1.7113e-01	1.3306e+00	1.7252e+00

```
In [80]: #Linear Regression with PCA
@dataclass
class LinearRegressionPCA:
```

```

""" Linear Regression
epsilon: Tolerance
maxIteration: Maximum number of Iteration before the loop breaks.
gd: Picks the option for gradient descent
    0: Normal Equation
    1: Gradient Descent
    2: Stochastic Descent
lambda_value:
    0: Means no Regularization
    >0: Means Regularization

"""
X : np.array
y : np.array
learningRate : float
epsilon : float
errors : list = field(default_factory=list)
maxIteration : int = 50000
gd : int = 0
lambda_value : int = 0
sample_size : int = 100
s : int = 0

def addX0(self, X):

    return np.column_stack([np.ones(X.shape[0]), X])

def rank(self, X):
    v, s, u = np.linalg.svd(X)
    rank = sum([True if i > 0 else False for i in s])
    # with Numpy, the rank is calculated like below
    rank = np.linalg.matrix_rank(X)
    return rank

def checkFullrank(self, X):
    rank = self.rank(X)
    if rank == min(X.shape):
        print("its full rank")
        self.Fullrank = True
    else:
        self.Fullrank = False

def checkLowRank(self, X):
    if X.shape[1] > X.shape[0]:

```

```
        self.LowRank = True
    else:
        self.LowRank = False

    def normalEquation(self, X, y):
        # adding regularization term
        normEq_term = self.lambda_value * np.identity(X.shape[1])
        self.w = np.linalg.inv(X.T.dot(X) + normEq_term).dot(X.T).dot(y)
        # Show error metrics
        self.error_report(X,y)

    def predict(self, X):
        return X.dot(self.w)

    def sse(self, X, y):
        y_hat = self.predict(X)

        return ((y_hat - y) ** 2).sum() # Norm 2

    def costFunction(self, X, y):
        sse = self.sse(X, y)
        return sse/2 # Division by 2 to remove the derived power of 2

    def rmse(self, X, y):
        return np.sqrt((self.sse(X, y))/ X.shape[0])

    def r2_squared(self, X, y):
        y_hat = self.predict(X)
        return np.exp(r2_score(y,y_hat))

    def error_report(self, X, y):
        print()
        print("SSE: {}".format(self.costFunction(X,y)))
        print("RMSE: {}".format(self.rmse(X, y)))
        print("R^2: {}".format(self.r2_squared(X, y)))

    def costDerivative(self, X, y):
        y_hat = self.predict(X)
        m = y.shape[0]
        return 1/m * X.T.dot(y_hat - y)

    def gradientDescent(self, X, y):
        self.errors = []
```

```

prev_error = float(np.inf)
linear_reg_term = self.lambda_value * self.w
for i in tqdm(range(self.maxIteration)):
    grad = self.costDerivative(X, y)
    self.w -= self.learningRate * (grad + linear_reg_term)
    current_error = self.rmse(X, y)
    if np.isnan(current_error):
        print("The values are out of Domain")
        break
    self.errors.append(current_error)
    if np.allclose(current_error, prev_error, rtol=self.epsilon):
        print("The model stopped learning")
        self.s = i + 1
        break
    prev_error = current_error
# Showing error metrics
self.error_report(X, y)

#Creating function for stochastic gradient
def stochasticgradientDescent(self, X, y, sample_size):

    self.errors = []
    prev_error = float(np.inf)
    for i in tqdm(range(self.maxIteration)):

        index = np.random.choice(np.arange(len(X)), self.sample_size, replace = False)
        X_sample = X[index]
        y_sample = y[index]
        grad_sample = self.costDerivative(X_sample, y_sample)
        # adding regularization term
        linear_reg_term = self.lambda_value * self.w
        self.w = self.w - self.learningRate * (grad_sample + linear_reg_term)
        current_error = self.rmse(X, y)
        self.errors.append(current_error)
        if abs(current_error - prev_error) < self.epsilon:
            print()
            print("The model stopped learning")
            self.s = i+1
            break
        prev_error = current_error

# Showing error metrics

```

```
self.error_report(X,y)

def mse_var_bias(self, X, y):
    mse = self.costFunction(X, y) / len(y)
    y_hat = self.predict(X)
    mse_bias = np.mean((y - np.mean(y_hat)) ** 2)
    mse_var = np.var(y_hat)
    print("MSE: {:.2f}".format(mse))
    print("Bias: {:.2f}".format(mse_bias))
    print("Variance: {:.2f}".format(mse_var))

def fit(self):
    X_train_pca = self.addX0(self.X)
    y_train_pca = self.y

    self.checkFullrank(X_train_pca)
    self.checkLowRank(X_train_pca)

    if self.Fullrank and not self.LowRank and X_train_pca.shape[0] < 10000 \
    and self.gd == 0:
        print("Solving using closed form")
        self.normalEquation(X_train_pca, y_train_pca)

    elif self.gd == 1:
        print("Solving using Gradient Descent")
        self.w = np.zeros(X_train_pca.shape[1])
        self.gradientDescent(X_train_pca, y_train_pca)

        print("For Test Data")
        X_testdata_pca = self.addX0(X_test_pca)
        self.error_report(X_testdata_pca, y_test_pca)

        print("For Validation Data")
        X_valid_pca = self.addX0(X_val_pca)
        self.error_report(X_valid_pca, y_val_pca)

        print("\nBias Variance Tradeoff")
        self.mse_var_bias(X_testdata_pca, y_test_pca)

    else:
        print("Solving using Stochastic Gradient")
        self.w = np.zeros(X_train_pca.shape[1])
        self.stochasticgradientDescent(X_train_pca, y_train_pca, self.sample_size)
```

```
print()
print("Weights: {}".format(self.w))
```

```
In [81]: lr_pca = LinearRegressionPCA(X = X_train_pca, # All columns except Appliances
                                     y = np.array(y_train_pca), # Appliances column
                                     learningRate=0.0001,
                                     maxIteration = 500,
                                     epsilon=0.001,
                                     gd=1,
                                     lambda_value=0.001
                                     ).fit()
```

its full rank

Solving using Gradient Descent

```
0%|| | 1/500 [00:00<00:00, 997.46it/s]
```

The model stopped learning

SSE: 135304948.4978031
 RMSE: 139.96263565688065
 R^2: 0.39630434217244215
 For Test Data

SSE: 32193019.716368124
 RMSE: 147.48572930591413
 R^2: 0.4251983429248604
 For Validation Data

SSE: 30345507.50122504
 RMSE: 143.16702375594312
 R^2: 0.41197782064659594

Bias Variance Tradeoff
 MSE: 10876.02
 Bias: 21753.46
 Variance: 0.00

```
Weights: [ 0.01941166  0.00378636 -0.00036454  0.00729374  0.00632434 -0.00186163
           0.002817   0.00118308  0.00307694  0.00021554]
```

```
In [82]: lr_pca = LinearRegressionPCA(X = X_train_pca, # All columns except Appliances
                                     y = np.array(y_train_pca), # Appliances column
```

```

learningRate=0.001,
maxIteration = 500,
epsilon=0.001,
gd=1,
lambda_value=0.01
).fit()

```

its full rank

Solving using Gradient Descent

0%|| | 1/500 [00:00<00:00, 863.38it/s]

The model stopped learning

SSE: 134990652.8347539

RMSE: 139.7999837172258

R^2: 0.3980809235935526

For Test Data

SSE: 32122254.2633158

RMSE: 147.32354141002637

R^2: 0.42693585406669

For Validation Data

SSE: 30277331.41562956

RMSE: 143.00610939974976

R^2: 0.413727888507966

Bias Variance Tradeoff

MSE: 10852.11

Bias: 21718.35

Variance: 0.04

Weights: [0.19402962 0.03768056 -0.00363065 0.07286365 0.06317963 -0.01860063
0.02815102 0.01182471 0.03075169 0.00215395]

```

In [83]: lr_pca = LinearRegressionPCA(X = X_train_pca, # All columns except Appliances
y = np.array(y_train_pca), # Appliances column
learningRate=0.01,
maxIteration = 500,
epsilon=0.001,
gd=1,
lambda_value=0.1
).fit()

```

its full rank
Solving using Gradient Descent

23% | ████████████████████ | 116/500 [00:00<00:00, 4748.76it/s]

The model stopped learning

SSE: 66307809.79375986
RMSE: 97.98004024464049
R²: 1.0579678937969437
For Test Data

SSE: 16394913.88086512
RMSE: 105.25038961494657
R²: 1.0567561622442965
For Validation Data

SSE: 15409300.251478381
RMSE: 102.02045424529501
R²: 1.0427895890970391

Bias Variance Tradeoff
MSE: 5538.82
Bias: 12801.18
Variance: 1160.26

Weights: [6.71972697e+01 1.77364855e+00 -2.98197873e-02 1.51463826e+01
1.32049676e+01 -4.26205942e+00 7.83708384e+00 3.74769201e+00
9.48457591e+00 6.25197869e-01]

```
In [84]: dff = [
    {"Approach": "Gradient Descent", "Learning Rate": "0.0001", "Lambda Value": "0.001", "RMSE": "139.9", "R^2": "0.39"},
    {"Approach": "Gradient Descent", "Learning Rate": "0.001", "Lambda Value": "0.01", "RMSE": "139.8", "R^2": "0.3"},
    {"Approach": "Gradient Descent", "Learning Rate": "0.01", "Lambda Value": "0.1", "RMSE": "97.86", "R^2": "1.06"},
    {"Approach": "Stochastic Gradient Descent", "Learning Rate": "0.0001", "Lambda Value": "0.001", "RMSE": "135.6", "R^2": "0.39"},
    {"Approach": "Stochastic Gradient Descent", "Learning Rate": "0.001", "Lambda Value": "0.01", "RMSE": "111.2", "R^2": "0.3"},
    {"Approach": "Stochastic Gradient Descent", "Learning Rate": "0.01", "Lambda Value": "0.1", "RMSE": "94.3", "R^2": "1.06"}
]

# Set the float format to scientific notation
pd.options.display.float_format = '{:.4e}'.format

# Create a pandas DataFrame with the data
results_table = pd.DataFrame(dff)
```



```
# Center-align the values and the column names in the DataFrame
styled_table = results_table.style.set_properties(**{'text-align': 'center'}).set_table_styles([
    {'selector': 'th',
     'props': [('text-align', 'center')]}
])

# Display the styled DataFrame
styled_table
```

Out [84]:

	Approach	Learning Rate	Lambda Value	RMSE	R ²
0	Gradient Descent	0.0001	0.001	139.9	0.39
1	Gradient Descent	0.001	0.01	139.8	0.3
2	Gradient Descent	0.01	0.1	97.86	1.06
3	Stochastic Gradient Descent	0.0001	0.001	135.6	0.44
4	Stochastic Gradient Descent	0.001	0.01	111.2	0.80
5	Stochastic Gradient Descent	0.01	0.1	94.3	1.13

In [85]: *#Ridge Regression*

```
@dataclass
class RidgeRegressionPCA:
    """
    epsilon: Tolerance
    maxIteration: Maximum number of Iteration before the loop breaks.
    gd: Picks the option for gradient descent
        1: Gradient Descent
        2: Stochastic Descent
    lambda_value:
        0: Means no Regularization
        >0: Means Regularization

    """
    X : np.array
    y : np.array
    learningRate : float
    epsilon : float
    errors : list = field(default_factory=list)
    maxIteration : int = 50000
```

```

gd : int = 0
lambda_value : int = 0
sample_size : int = 100
s : int = 0

def splitData(self):
    X_train, X_test, y_train, y_test = train_test_split(self.X,
                                                         self.y,
                                                         test_size=0.3,
                                                         shuffle=True)

    return X_train, X_test, y_train, y_test

def addX0(self, X):

    return np.column_stack([np.ones(X.shape[0]), X])

def normalizeTrain(self, X):
    """Z score normalization on the training data set
    Args:
        X: Dataset
    Returns:
        X_norm: Normalized z score Training dataset
        mean: Mean of Training data
        std: Standard deviation of Training data
    """
    mean = np.mean(X, axis=0)
    std = np.std(X, axis=0)
    X_norm = (X - mean)/std
    X_norm = self.addX0(X_norm)
    return X_norm, mean, std

def predict(self, X):
    return X.dot(self.w)

def sse(self, X, y):
    y_hat = self.predict(X)

    return ((y_hat - y) ** 2).sum() # Norm 2

def costFunction(self, X, y):
    sse = self.sse(X, y)
    return sse/2 # Division by 2 to remove the derived power of 2

```

```

def rmse(self, X, y):
    return np.sqrt((self.sse(X, y))/ X.shape[0])

def r2_squared(self, X, y):
    y_hat = self.predict(X)
    return (np.exp(r2_score(y,y_hat)))

def error_report(self, X, y):
    print()
    print("SSE: {}".format(self.costFunction(X,y)))
    print("RMSE: {}".format(self.rmse(X, y)))
    print("R^2: {}".format(self.r2_squared(X, y)))

def costDerivative(self, X, y):
    y_hat = self.predict(X)
    m = y.shape[0]
    return 1/m * X.T.dot(y_hat - y)

def gradientDescent(self, X, y):
    self.errors = []
    prev_error = float(np.inf)
    for i in tqdm(range(self.maxIteration)):
        grad = self.costDerivative(X, y)
        # adding regularization
        linear_reg_term = self.lambda_value * self.w
        self.w = self.w - self.learningRate * (grad + linear_reg_term)
        current_error = self.rmse(X, y)
        if math.isnan(current_error):
            print("Out of Domain: Values Tending to Infinite")
            break
        self.errors.append(current_error)
        if abs(current_error - prev_error) < self.epsilon:
            print("The model stopped learning")
            self.s = i+1
            break
        prev_error = current_error
    # Showing error metrics
    self.error_report(X,y)

# Creating function for stochastic gradient
def stochasticgradientDescent(self, X, y, sample_size):

```

```
self.errors = []
prev_error = float(np.inf)
for i in tqdm(range(self.maxIteration)):

    index = np.random.choice(np.arange(len(X)), self.sample_size, replace = False)
    X_sample = X[index]
    y_sample = y[index]
    grad_sample = self.costDerivative(X_sample, y_sample)
    # adding regularization term
    linear_reg_term = self.lambda_value * self.w
    self.w = self.w - self.learningRate * (grad_sample + linear_reg_term)
    current_error = self.rmse(X, y)
    self.errors.append(current_error)
    if abs(current_error - prev_error) < self.epsilon:
        print("The model stopped learning")
        self.s = i+1
        break
    prev_error = current_error

# Showing error metrics
self.error_report(X,y)

def mse_var_bias(self, X, y):
    mse = self.costFunction(X, y) / len(y)
    y_hat = self.predict(X)
    mse_bias = np.mean((y - np.mean(y_hat)) ** 2)
    mse_var = np.var(y_hat)
    print("MSE: {:.2f}".format(mse))
    print("Bias: {:.2f}".format(mse_bias))
    print("Variance: {:.2f}".format(mse_var))

def fit(self):
    X_train_pca = self.addX0(self.X)
    y_train_pca = self.y

    if self.gd == 1:
        print("Solving using Gradient Descent")
        self.w = np.zeros(X_train_pca.shape[1])
        self.gradientDescent(X_train_pca, y_train_pca)

    print("For Test Data")
```


SSE: 134299061.6546501
RMSE: 139.44140856178768
R²: 0.40201829203907075
For Test Data

SSE: 31966517.490765467
RMSE: 146.96597669042745
R²: 0.43078472190006867
For Validation Data

SSE: 30127322.070236653
RMSE: 142.6514069006589
R²: 0.4176048208864371

Bias Variance Tradeoff

MSE: 10799.50

Bias: 21640.84

Variance: 0.34

```
Weights: [ 0.58066989  0.11012514 -0.0106553   0.21739268  0.1885022  -0.05554723
           0.08414391  0.03537448  0.09196659  0.00643864]
```

```
In [87]: ridge_pca = RidgeRegressionPCA(X = X_train_pca, # All columns except Appliances
                                         y = np.array(y_train_pca), # Appliances column
                                         learningRate=0.001,
                                         maxIteration = 60,
                                         epsilon=0.001,
                                         gd=1,
                                         lambda_value=0.01
                                         ).fit()
```

Solving using Gradient Descent

[illegible]

SSE: 125689508.65923412
RMSE: 134.8977749257706
R²: 0.4544205993627491
For Test Data

SSE: 30025040.792579234
RMSE: 142.43310614009548
R²: 0.4817815061874062
For Validation Data

SSE: 28260550.923242114
RMSE: 138.16120300711873
R²: 0.4690012288767707

Bias Variance Tradeoff

MSE: 10143.59

Bias: 20651.99

Variance: 26.29

```
Weights: [ 5.65430354  0.84085441 -0.08429968  2.04890732  1.77699125 -0.52845362
  0.80871702  0.3430303   0.88901573  0.06195987]
```

```
In [88]: ridge_pca = RidgeRegressionPCA(X = X_train_pca, # All columns except Appliances
                                         y = np.array(y_train_pca), # Appliances column
                                         learningRate=0.01,
                                         maxIteration = 60,
                                         epsilon=0.001,
                                         gd=1,
                                         lambda_value=0.1
                                         ).fit()
```

Solving using Gradient Descent

[illegible]

SSE: 81460529.35477279
 RMSE: 108.59976512097434
 R²: 0.8527458264551524
 For Test Data

SSE: 19947185.810308743
 RMSE: 116.09404915932862
 R²: 0.8611348651346823
 For Validation Data

SSE: 18685502.67233283
 RMSE: 112.343561304242
 R²: 0.8506076705752997

Bias Variance Tradeoff

MSE: 6738.91

Bias: 14996.89

Variance: 686.11

Weights: [42.83477568 1.74942251 -0.11504379 11.81453797 10.28372052 -3.23087414
 5.47771985 2.48839112 6.34347647 0.42965781]

```
In [89]: dff = [
    {"Approach": "Gradient Descent", "Learning Rate": "0.0001", "Lambda Value": "0.001", "RMSE": "139.4", "R^2": "0.40"},
    {"Approach": "Gradient Descent", "Learning Rate": "0.001", "Lambda Value": "0.01", "RMSE": "134.89", "R^2": "0.45"},
    {"Approach": "Gradient Descent", "Learning Rate": "0.01", "Lambda Value": "0.1", "RMSE": "108.52", "R^2": "0.85"},
    {"Approach": "Stochastic Gradient Descent", "Learning Rate": "0.0001", "Lambda Value": "0.001", "RMSE": "139.46", "R^2": "0.40"},
    {"Approach": "Stochastic Gradient Descent", "Learning Rate": "0.001", "Lambda Value": "0.01", "RMSE": "135.05", "R^2": "0.45"},
    {"Approach": "Stochastic Gradient Descent", "Learning Rate": "0.01", "Lambda Value": "0.1", "RMSE": "109.27", "R^2": "0.85"}
]

# Set the float format to scientific notation
pd.options.display.float_format = '{:.4e}'.format

# Create a pandas DataFrame with the data
results_table = pd.DataFrame(dff)

# Center-align the values and the column names in the DataFrame
styled_table = results_table.style.set_properties(**{'text-align': 'center'}).set_table_styles([
    {'selector': 'th',
     'props': [('text-align', 'center')]}
])
```



```
# Display the styled DataFrame
styled_table
```

Out[89]:

	Approach	Learning Rate	Lambda Value	RMSE	R^2
0	Gradient Descent	0.0001	0.001	139.4	0.40
1	Gradient Descent	0.001	0.01	134.89	0.45
2	Gradient Descent	0.01	0.1	108.52	0.85
3	Stochastic Gradient Descent	0.0001	0.001	139.46	0.40
4	Stochastic Gradient Descent	0.001	0.01	135.05	0.45
5	Stochastic Gradient Descent	0.01	0.1	109.27	0.84

In [90]:

```
@dataclass
class LassoRegressionPCA:
    """
    epsilon: Tolerance
    maxIteration: Maximum number of Iteration before the loop breaks.
    gd: Picks the option for gradient descent
        1: Gradient Descent
        2: Stochastic Descent
    alpha:
        0: Means no Regularization
        >0: Means Regularization

    """
    X : np.array
    y : np.array
    learningRate : float
    epsilon : float
    errors : list = field(default_factory=list)
    maxIteration : int = 50000
    gd : int = 0
    lambda_value : int = 0
    alpha : int = 0
    sample_size : int = 100
    s : int = 0

    def splitData(self):
        X_train, X_test, y_train, y_test = train_test_split(self.X,
                                                             self.y,
```

```

        test_size=0.3,
        shuffle=True)

    return X_train, X_test, y_train, y_test

def addX0(self, X):

    return np.column_stack([np.ones(X.shape[0]), X])

def normalizeTrain(self, X):
    """Z score normalization on the training data set
    Args:
        X: Dataset
    Returns:
        X_norm: Normalized z score Training dataset
        mean: Mean of Training data
        std: Standard deviation of Training data
    """
    mean = np.mean(X, axis=0)
    std = np.std(X, axis=0)
    X_norm = (X - mean)/std
    X_norm = self.addX0(X_norm)
    return X_norm, mean, std

def predict(self, X):
    return X.dot(self.w)

def sse(self, X, y):
    y_hat = self.predict(X)

    return ((y_hat - y) ** 2).sum() # Norm 2

def costFunction(self, X, y):
    sse = self.sse(X, y)
    return sse/2 # Division by 2 to remove the derived power of 2

def rmse(self, X, y):
    return np.sqrt((self.sse(X, y))/ X.shape[0])

def r2_squared(self, X, y):
    y_hat = self.predict(X)
    return (np.exp(r2_score(y,y_hat)))

def error_report(self, X, y):

```

```
print()
print("SSE: {}".format(self.costFunction(X,y)))
print("RMSE: {}".format(self.rmse(X, y)))
print("R^2: {}".format(self.r2_squared(X, y)))

def costDerivative(self, X, y):
    y_hat = self.predict(X)
    m = y.shape[0]
    return 1/m * X.T.dot(y_hat - y)

def gradientDescent(self, X, y):
    self.errors = []
    prev_error = float(np.inf)
    for i in tqdm(range(self.maxIteration)):
        grad = self.costDerivative(X, y)
        # adding regularization
        lasso_reg_term = self.lambda_value * np.sign(self.w)
        self.w = self.w - self.learningRate * (grad + lasso_reg_term)
        current_error = self.rmse(X, y)
        if math.isnan(current_error):
            print("Out of Domain: Values Tending to Infinite")
            break
        self.errors.append(current_error)
        if abs(current_error - prev_error) < self.epsilon:
            print("The model stopped learning")
            self.s = i+1
            break
        prev_error = current_error
    # Showing error metrics
    self.error_report(X,y)

# Creating function for stochastic gradient
def stochasticgradientDescent(self, X, y, sample_size):

    self.errors = []
    prev_error = float(np.inf)
    for i in tqdm(range(self.maxIteration)):

        index = np.random.choice(np.arange(len(X)), self.sample_size, replace=False)
        X_sample = X[index]
        y_sample = y[index]
        grad_sample = self.costDerivative(X_sample, y_sample)
```

```
# adding regularization term
lasso_reg_term = self.lambda_value * np.sign(self.w)
self.w = self.w - self.learningRate * (grad_sample + lasso_reg_term)
current_error = self.rmse(X, y)
self.errors.append(current_error)
if abs(current_error - prev_error) < self.epsilon:
    print("The model stopped learning")
    self.s = i+1
    break
prev_error = current_error

# Showing error metrics
self.error_report(X,y)

def mse_var_bias(self, X, y):
    mse = self.costFunction(X, y) / len(y)
    y_hat = self.predict(X)
    mse_bias = np.mean((y - np.mean(y_hat)) ** 2)
    mse_var = np.var(y_hat)
    print("MSE: {:.2f}".format(mse))
    print("Bias: {:.2f}".format(mse_bias))
    print("Variance: {:.2f}".format(mse_var))

def fit(self):
    X_train_pca = self.addX0(self.X)
    y_train_pca = self.y

    if self.gd == 1:
        print("Solving using Gradient Descent")
        self.w = np.zeros(X_train_pca.shape[1])
        self.gradientDescent(X_train_pca, y_train_pca)

        print("For Test Data")
        X_testdata_pca = self.addX0(X_test_pca)
        self.error_report(X_testdata_pca, y_test_pca)

        print("For Validation Data")
        X_valid_pca = self.addX0(X_val_pca)
        self.error_report(X_valid_pca, y_val_pca)

        print("\nBias Variance Tradeoff")
        self.mse_var_bias(X_testdata_pca, y_test_pca)

    else:
```



```
learningRate=0.001,  
maxIteration = 100,  
epsilon=0.00001,  
gd=1,  
lambda_value=0.01  
) .fit()
```

Solving using Gradient Descent

[illegible]

SSE: 120063442.02050084
RMSE: 131.84408994691768
R^2: 0.4923007222461251
For Test Data

SSE: 28753633.936766464
RMSE: 139.3848307326006
R²: 0.5184059157437678
For Validation Data

SSE: 27041218.208871197
RMSE: 135.14778134451416
R²: 0.505940747874053

Bias Variance Tradeoff

MSE: 9714.07

Bias: 19984.40

Variance: 62.26

```
Weights: [ 9.24206366  1.16418667 -0.11777836  3.26917127  2.83584396 -0.84790387
  1.30846842  0.5579987   1.44466736  0.09952408]
```

```
In [92]: lasso_pca = LassoRegressionPCA(X = X_train_pca, # All columns except Appliances
                                         y = np.array(y_train_pca), # Appliances column
                                         learningRate=0.01,
                                         maxIteration = 100,
                                         epsilon=0.00001,
                                         gd=1,
                                         lambda_value=0.1
                                         ).fit()
```

Solving using Gradient Descent


```
] )
```

```
# Display the styled DataFrame
styled_table
```

Out[93]:

	Approach	Learning Rate	Lambda Value	RMSE	R ²
0	Gradient Descent	0.0001	0.001	139.08	0.40
1	Gradient Descent	0.001	0.01	131.83	0.49
2	Gradient Descent	0.01	0.1	99.84	1.02
3	Stochastic Gradient Descent	0.0001	0.001	139.08	0.40
4	Stochastic Gradient Descent	0.001	0.01	131.89	0.49
5	Stochastic Gradient Descent	0.01	0.1	99.93	1.01

In [95]:

```
dff = [
    {"Approach": "Linear Regression (Gradient Descent)", "Learning Rate": "0.0001", "Lambda Value": "0.001", "RMSE": "139.08", "R^2": "0.40"},
    {"Approach": "Linear Regression (Stochastic Gradient Descent)", "Learning Rate": "0.001", "Lambda Value": "0.01", "RMSE": "131.83", "R^2": "0.49"},
    {"Approach": "Ridge Regression (Gradient Descent)", "Learning Rate": "0.01", "Lambda Value": "0.1", "RMSE": "99.84", "R^2": "1.02"},
    {"Approach": "Ridge Regression (Stochastic Gradient Descent)", "Learning Rate": "0.01", "Lambda Value": "0.1", "RMSE": "99.93", "R^2": "1.01"},
    {"Approach": "Lasso Regression (Gradient Descent)", "Learning Rate": "0.001", "Lambda Value": "0.01", "RMSE": "131.89", "R^2": "0.49"},
    {"Approach": "Lasso Regression (Stochastic Gradient Descent)", "Learning Rate": "0.001", "Lambda Value": "0.01", "RMSE": "131.83", "R^2": "0.49"},
    {"Approach": "Neural Nets", "Learning Rate": "-", "Lambda Value": "-", "RMSE": "79.73", "R^2": "0.40"},

    {"Approach": "K-Nearest Neighbors (KNN)", "Learning Rate": "0.001", "Lambda Value": "-", "RMSE": "71.69", "R^2": "0.40"},
    {"Approach": "K-Nearest Neighbors (KNN) - Validation", "Learning Rate": "0.001", "Lambda Value": "-", "RMSE": "71.69", "R^2": "0.40"}
]

# Set the float format to scientific notation
pd.options.display.float_format = '{:.4e}'.format

# Create a pandas DataFrame with the data
results_table = pd.DataFrame(dff)

# Center-align the values and the column names in the DataFrame
styled_table = results_table.style.set_properties(**{'text-align': 'left'}).set_table_styles([
    {'selector': 'th',
     'props': [('text-align', 'center')]}
])
```



```
# Display the styled DataFrame
styled_table
```

Out [95]:

	Approach	Learning Rate	Lambda Value	RMSE	R^2
0	Linear Regression (Gradient Descent)	0.0001	0.001	139.9	0.39
1	Linear Regression (Stochastic Gradient Descent)	0.001	0.01	111.2	0.80
2	Ridge Regression (Gradient Descent)	0.01	0.1	108.52	0.85
3	Ridge Regression (Stochastic Gradient Descent)	0.01	0.1	109.27	0.84
4	Lasso Regression (Gradient Descent)	0.001	0.01	131.83	0.49
5	Lasso Regression (Stochastic Gradient Descent)	0.001	0.01	131.89	0.49
6	Neural Nets	-	-	79.73	0.40
7	K-Nearest Neighbors (KNN)	0.001	-	71.69	0.53
8	K-Nearest Neighbors (KNN) - Validation	0.001	-	71.78	0.54

In [96]:

```
# Preprocess data
X_neural_pca = X_transformed # features
y_neural_pca = y_for_pca.values.reshape(-1, 1) # target

# Convert string data to float
X_neural_pca = X_neural_pca.astype(np.float32)

# Define the model
model_pca = Sequential()
model_pca.add(Dense(64, input_shape=(9,), activation='relu'))
model_pca.add(Dropout(0.2))
model_pca.add(Dense(32, activation='relu'))
model_pca.add(Dropout(0.2))
model_pca.add(Dense(1, activation='linear'))

# Compile the model
model_pca.compile(loss='mean_squared_error', optimizer='adam')

# Train the model
model_pca.fit(X_neural_pca, y_neural_pca, epochs=100, batch_size=32, validation_split=0.4)

# Make predictions
y_pred_neural_pca = model_pca.predict(X_neural_pca)
```

Epoch 1/100
371/371  **1s** 1ms/step - loss: 16488.8047 - val_loss: 10094.3730

Epoch 2/100
371/371  **0s** 933us/step - loss: 9722.3760 - val_loss: 9246.6318

Epoch 3/100
371/371  **0s** 905us/step - loss: 9821.7734 - val_loss: 9015.8262

Epoch 4/100
371/371  **0s** 909us/step - loss: 9425.7783 - val_loss: 8833.2627

Epoch 5/100
371/371  **0s** 910us/step - loss: 9392.8408 - val_loss: 8735.9170

Epoch 6/100
371/371  **0s** 860us/step - loss: 9195.9238 - val_loss: 8694.2139

Epoch 7/100
371/371  **0s** 908us/step - loss: 8945.3369 - val_loss: 8710.5967

Epoch 8/100
371/371  **0s** 905us/step - loss: 9206.2979 - val_loss: 8633.0469

Epoch 9/100
371/371  **0s** 882us/step - loss: 8839.6953 - val_loss: 8676.7510

Epoch 10/100
371/371  **0s** 895us/step - loss: 8632.3535 - val_loss: 8621.0391

Epoch 11/100
371/371  **0s** 901us/step - loss: 9537.3125 - val_loss: 8671.8291

Epoch 12/100
371/371  **0s** 904us/step - loss: 8817.8818 - val_loss: 8694.3389

Epoch 13/100
371/371  **0s** 920us/step - loss: 8926.7754 - val_loss: 8546.9600

Epoch 14/100
371/371  **0s** 896us/step - loss: 9319.8721 - val_loss: 8526.3086

Epoch 15/100
371/371  **0s** 887us/step - loss: 8799.7734 - val_loss: 8476.5273

Epoch 16/100
371/371  **0s** 907us/step - loss: 9092.3613 - val_loss: 8470.3672

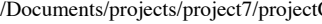
Epoch 17/100
371/371  **0s** 876us/step - loss: 9199.6436 - val_loss: 8486.5889

Epoch 18/100
371/371  **0s** 908us/step - loss: 8812.8760 - val_loss: 8484.0146

Epoch 19/100
371/371  **0s** 838us/step - loss: 8981.2852 - val_loss: 8485.6104

Epoch 20/100
371/371  **0s** 859us/step - loss: 8298.5449 - val_loss: 8449.4775

Epoch 21/100
371/371  **0s** 848us/step - loss: 8976.5264 - val_loss: 8577.9092

Epoch 22/100
371/371  **0s** 893us/step - loss: 8378.8818 - val_loss: 8612.5215

Epoch 23/100
371/371  **0s** 926us/step - loss: 8803.9736 - val_loss: 8667.3984

Epoch 24/100
371/371  **0s** 890us/step - loss: 8492.6963 - val_loss: 8425.7490

Epoch 25/100
371/371  **0s** 912us/step - loss: 8167.0674 - val_loss: 8506.2998

Epoch 26/100
371/371  **0s** 905us/step - loss: 8530.3438 - val_loss: 8596.0127

Epoch 27/100
371/371  **0s** 897us/step - loss: 8352.9199 - val_loss: 8820.1035

Epoch 28/100
371/371  **0s** 917us/step - loss: 8594.8184 - val_loss: 8641.3545

Epoch 29/100
371/371  **0s** 911us/step - loss: 8651.8467 - val_loss: 8672.3662

Epoch 30/100
371/371  **0s** 881us/step - loss: 8153.0376 - val_loss: 8963.2021

Epoch 31/100
371/371  **0s** 864us/step - loss: 8100.5537 - val_loss: 8825.6846

Epoch 32/100
371/371  **0s** 878us/step - loss: 7960.7656 - val_loss: 8934.6396

Epoch 33/100
371/371  **0s** 885us/step - loss: 8062.2109 - val_loss: 8974.7266

Epoch 34/100
371/371  **0s** 912us/step - loss: 7794.6108 - val_loss: 9090.0146

Epoch 35/100
371/371  **0s** 907us/step - loss: 8202.9385 - val_loss: 8933.5254

Epoch 36/100
371/371  **0s** 885us/step - loss: 8085.0020 - val_loss: 8975.0127

Epoch 37/100
371/371  **0s** 938us/step - loss: 8113.9409 - val_loss: 9173.8506

Epoch 38/100
371/371  **0s** 927us/step - loss: 7970.6626 - val_loss: 9324.9688

Epoch 39/100
371/371  **0s** 932us/step - loss: 7896.9419 - val_loss: 9247.1562

Epoch 40/100
371/371  **0s** 914us/step - loss: 7906.7485 - val_loss: 9153.9238

Epoch 41/100
371/371  **0s** 904us/step - loss: 7526.3242 - val_loss: 9146.7197

Epoch 42/100
371/371  **0s** 916us/step - loss: 7716.0195 - val_loss: 8781.9229

Epoch 43/100
371/371  **0s** 952us/step - loss: 7745.8369 - val_loss: 8915.6543

Epoch 44/100
371/371  **0s** 935us/step - loss: 7759.4497 - val_loss: 8793.5771

Epoch 45/100
371/371  **0s** 878us/step - loss: 7918.4453 - val_loss: 9151.5498

Epoch 46/100
371/371  **0s** 928us/step - loss: 7363.9854 - val_loss: 8953.6846

Epoch 47/100
371/371  **0s** 902us/step - loss: 7445.7183 - val_loss: 9105.4688

Epoch 48/100
371/371  **0s** 910us/step - loss: 7574.4756 - val_loss: 9052.8467

Epoch 49/100
371/371  **0s** 923us/step - loss: 7228.5356 - val_loss: 9063.3633

Epoch 50/100
371/371  **0s** 906us/step - loss: 7654.4287 - val_loss: 9020.2705

Epoch 51/100
371/371  **0s** 921us/step - loss: 7104.0508 - val_loss: 9126.4102

Epoch 52/100
371/371  **0s** 975us/step - loss: 7560.1616 - val_loss: 9192.3340

Epoch 53/100
371/371  **0s** 958us/step - loss: 7546.8188 - val_loss: 8821.6152

Epoch 54/100
371/371  **0s** 918us/step - loss: 7188.0454 - val_loss: 9317.5654

Epoch 55/100
371/371  **0s** 894us/step - loss: 7578.2437 - val_loss: 9376.7188

Epoch 56/100
371/371  **0s** 900us/step - loss: 7381.9565 - val_loss: 9106.9541

Epoch 57/100
371/371  **0s** 908us/step - loss: 7742.4688 - val_loss: 9181.8789

Epoch 58/100
371/371  **0s** 914us/step - loss: 7508.8545 - val_loss: 8980.0342

Epoch 59/100
371/371  **0s** 916us/step - loss: 7297.4453 - val_loss: 9132.6426

Epoch 60/100
371/371  **0s** 937us/step - loss: 7339.9438 - val_loss: 9330.1572

Epoch 61/100
371/371  **0s** 916us/step - loss: 6766.4819 - val_loss: 10027.6689

Epoch 62/100
371/371  **0s** 927us/step - loss: 7057.0298 - val_loss: 9836.4121

Epoch 63/100
371/371  **0s** 939us/step - loss: 7319.0903 - val_loss: 9803.2979

Epoch 64/100
371/371  **0s** 947us/step - loss: 7414.0269 - val_loss: 10393.0127

Epoch 65/100
371/371  **0s** 899us/step - loss: 7428.7554 - val_loss: 10807.0273

Epoch 66/100
371/371  **0s** 870us/step - loss: 6916.4697 - val_loss: 10629.8604

Epoch 67/100
371/371  **0s** 901us/step - loss: 6898.1963 - val_loss: 10759.1797

Epoch 68/100
371/371  **0s** 873us/step - loss: 7118.7437 - val_loss: 10431.4746

Epoch 69/100
371/371  **0s** 894us/step - loss: 7357.7593 - val_loss: 11076.5791

Epoch 70/100
371/371  **0s** 926us/step - loss: 7036.3379 - val_loss: 11810.2822

Epoch 71/100
371/371  **0s** 861us/step - loss: 7107.0615 - val_loss: 11527.6650

Epoch 72/100
371/371  **0s** 884us/step - loss: 6455.3721 - val_loss: 11349.0781

Epoch 73/100
371/371  **0s** 919us/step - loss: 6919.4751 - val_loss: 11954.1094

Epoch 74/100
371/371  **0s** 903us/step - loss: 7835.7827 - val_loss: 11895.3096

Epoch 75/100
371/371  **0s** 886us/step - loss: 7172.7295 - val_loss: 12172.5859

Epoch 76/100
371/371  **0s** 941us/step - loss: 7057.4980 - val_loss: 13158.7070

Epoch 77/100
371/371  **0s** 896us/step - loss: 6928.1235 - val_loss: 12867.4014

Epoch 78/100
371/371  **0s** 931us/step - loss: 6925.8350 - val_loss: 12779.2031

Epoch 79/100
371/371  **0s** 904us/step - loss: 7320.4912 - val_loss: 13261.6562

Epoch 80/100
371/371  **0s** 900us/step - loss: 6766.2056 - val_loss: 13349.4219

Epoch 81/100
371/371  **0s** 932us/step - loss: 7094.5332 - val_loss: 13987.5283

Epoch 82/100
371/371  **0s** 906us/step - loss: 7080.6294 - val_loss: 13533.0967

Epoch 83/100
371/371  **0s** 936us/step - loss: 6922.5620 - val_loss: 13077.8555

Epoch 84/100
371/371  **0s** 924us/step - loss: 6982.2573 - val_loss: 14028.3447

Epoch 85/100
371/371  **0s** 903us/step - loss: 6949.4697 - val_loss: 13553.0625

Epoch 86/100
371/371  **0s** 949us/step - loss: 6995.5986 - val_loss: 14197.5117

Epoch 87/100
371/371  **0s** 914us/step - loss: 6633.1699 - val_loss: 14736.7510

Epoch 88/100
371/371  **0s** 922us/step - loss: 7047.2114 - val_loss: 15215.5186

```

Epoch 89/100
371/371 ————— 0s 907us/step - loss: 7111.4146 - val_loss: 14461.4883
Epoch 90/100
371/371 ————— 0s 924us/step - loss: 7223.3589 - val_loss: 15333.4053
Epoch 91/100
371/371 ————— 0s 885us/step - loss: 7053.3560 - val_loss: 14741.2188
Epoch 92/100
371/371 ————— 0s 939us/step - loss: 7006.1064 - val_loss: 15272.9365
Epoch 93/100
371/371 ————— 0s 930us/step - loss: 6862.2095 - val_loss: 15946.4238
Epoch 94/100
371/371 ————— 0s 884us/step - loss: 6635.8052 - val_loss: 15170.4951
Epoch 95/100
371/371 ————— 0s 882us/step - loss: 6499.3569 - val_loss: 14847.5107
Epoch 96/100
371/371 ————— 0s 929us/step - loss: 7414.5596 - val_loss: 15862.6270
Epoch 97/100
371/371 ————— 0s 892us/step - loss: 6722.3013 - val_loss: 15201.5469
Epoch 98/100
371/371 ————— 0s 898us/step - loss: 7169.1279 - val_loss: 15164.4277
Epoch 99/100
371/371 ————— 0s 901us/step - loss: 6366.7412 - val_loss: 16721.1660
Epoch 100/100
371/371 ————— 0s 887us/step - loss: 6967.5400 - val_loss: 16291.7412
617/617 ————— 0s 489us/step

```

```

In [97]: # Calculate R-square, RMSE, SSE
r2_pca = r2_score(y_neural_pca, y_pred_neural_pca)
rmse_pca = mean_squared_error(y_neural_pca, y_pred_neural_pca, squared=False)
sse_pca = mean_squared_error(y_neural_pca, y_pred_neural_pca, squared=True) * len(y_neural_pca)
print('R2: {:.2f}'.format(r2_pca))
print("RMSE: {:.2f}".format(rmse_pca))
print("SSE: {:.2f}".format(sse_pca))

R2: 0.03
RMSE: 101.17
SSE: 201977124.20

```

```

In [99]: import matplotlib.pyplot as plt

# Example data from the table you provided
learning_rates = [0.0001, 0.001, 0.01] # The different learning rates
lambda_values = [0.001, 0.01, 0.1]      # The corresponding lambda values

```

```
# RMSE and R^2 values for Gradient Descent
gd_rmse = [139.6, 133.04, 94.06]
gd_r2 = [0.40, 0.47, 1.13]

# RMSE and R^2 values for Stochastic Gradient Descent
sgd_rmse = [139.65, 133.04, 94.06]
sgd_r2 = [0.40, 0.47, 1.13]

# Plotting RMSE values
plt.plot(learning_rates, gd_rmse, color='g', label='Gradient Descent RMSE', marker='o')
plt.plot(learning_rates, sgd_rmse, color='b', label='SGD RMSE', marker='o')

# Set the x-axis to a logarithmic scale
plt.xscale('log')

# Add labels and legend
plt.xlabel('Learning Rate (log scale)')
plt.ylabel('RMSE')
plt.legend(loc='best')

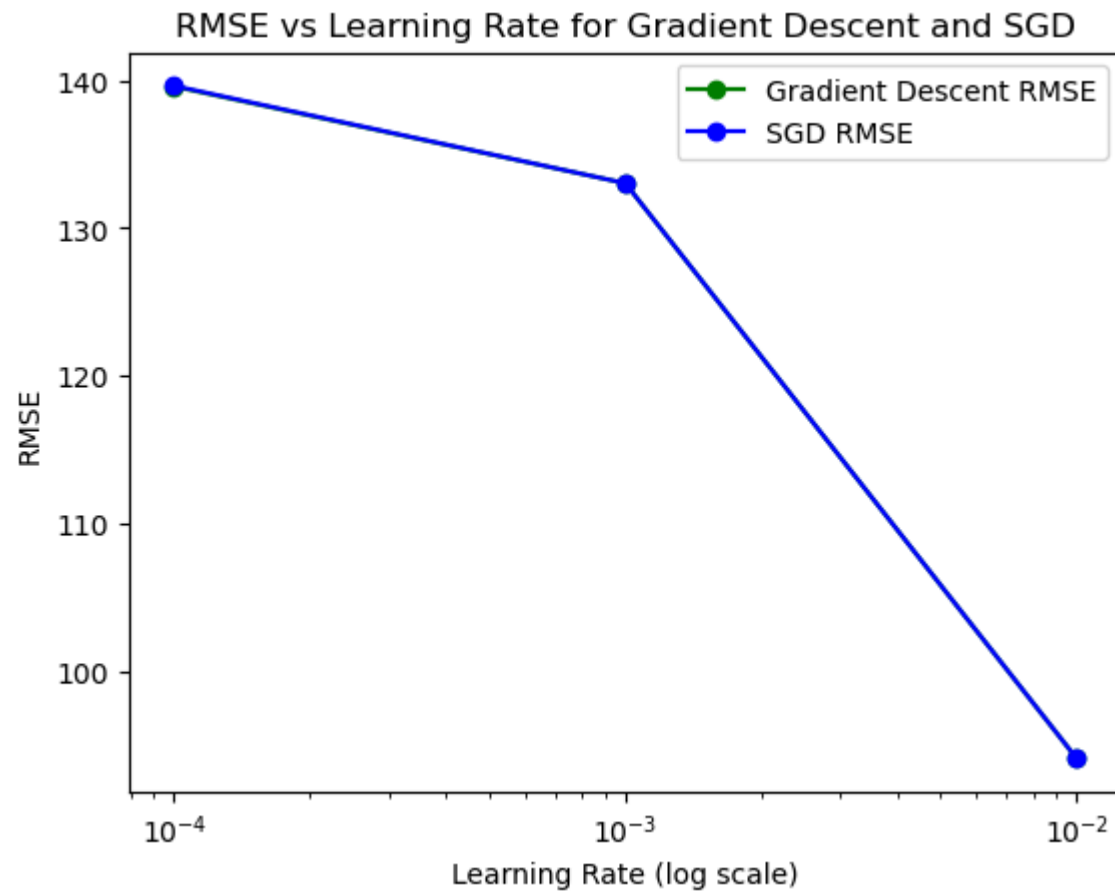
# Display the RMSE plot
plt.title('RMSE vs Learning Rate for Gradient Descent and SGD')
plt.show()

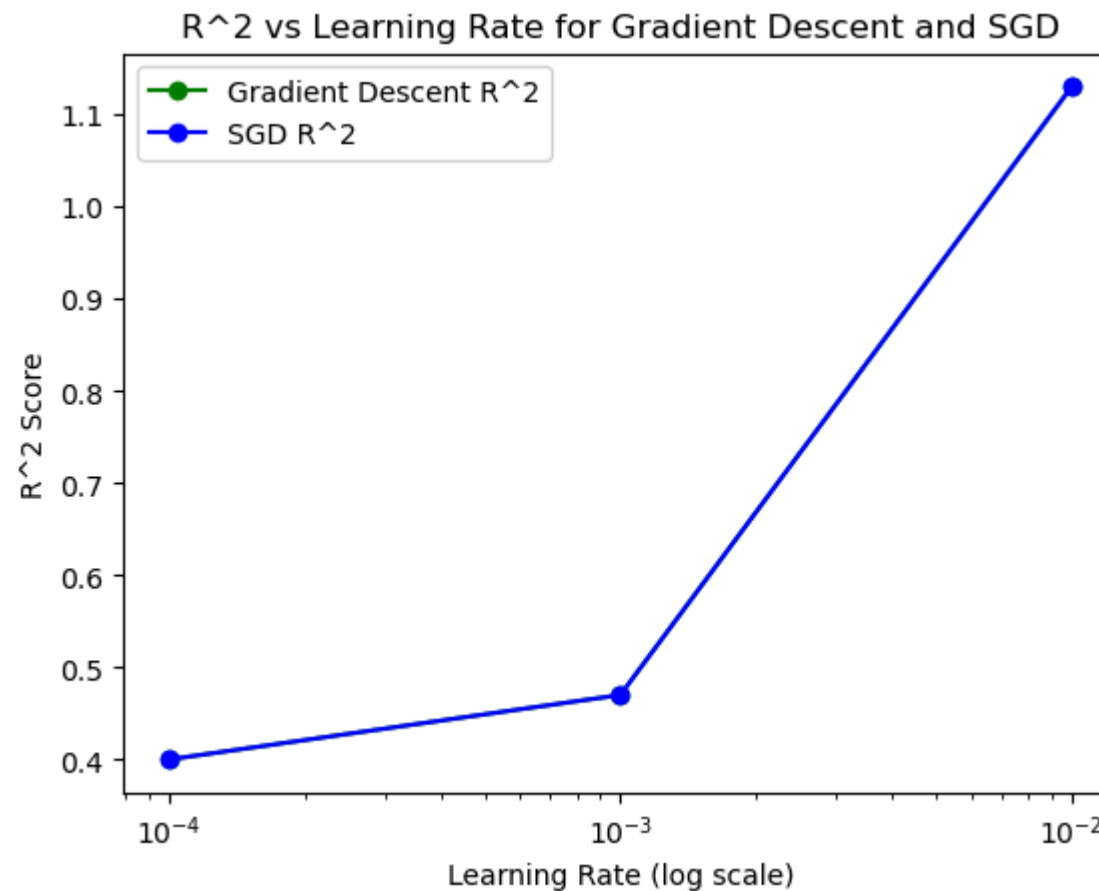
# Plotting R^2 values
plt.plot(learning_rates, gd_r2, color='g', label='Gradient Descent R^2', marker='o')
plt.plot(learning_rates, sgd_r2, color='b', label='SGD R^2', marker='o')

# Set the x-axis to a logarithmic scale
plt.xscale('log')

# Add labels and legend
plt.xlabel('Learning Rate (log scale)')
plt.ylabel('R^2 Score')
plt.legend(loc='best')

# Display the R^2 plot
plt.title('R^2 vs Learning Rate for Gradient Descent and SGD')
plt.show()
```





```
In [100... import matplotlib.pyplot as plt

# Updated data from the new table
learning_rates = [0.0001, 0.001, 0.01] # The different learning rates
lambda_values = [0.001, 0.01, 0.1]      # The corresponding lambda values

# RMSE and R^2 values for Gradient Descent
gd_rmse = [139.9, 139.7, 69.01]
gd_r2 = [0.39, 0.38, 1.70]

# RMSE and R^2 values for Stochastic Gradient Descent
sgd_rmse = [134.3, 99.01, 72.63]
sgd_r2 = [0.46, 1.03, 1.61]
```

```
# Plotting RMSE values
plt.plot(learning_rates, gd_rmse, color='g', label='Gradient Descent RMSE', marker='o')
plt.plot(learning_rates, sgd_rmse, color='b', label='SGD RMSE', marker='o')

# Set the x-axis to a logarithmic scale
plt.xscale('log')

# Add labels and legend
plt.xlabel('Learning Rate (log scale)')
plt.ylabel('RMSE')
plt.legend(loc='best')

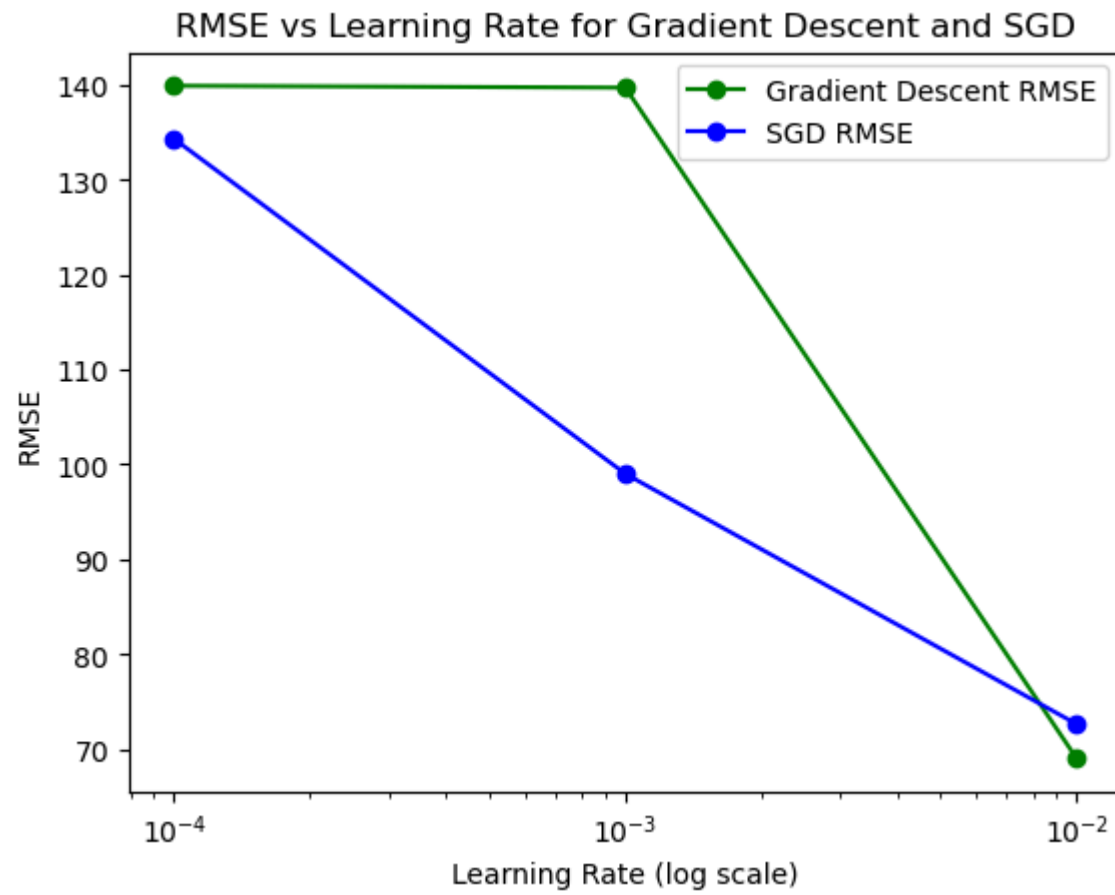
# Display the RMSE plot
plt.title('RMSE vs Learning Rate for Gradient Descent and SGD')
plt.show()

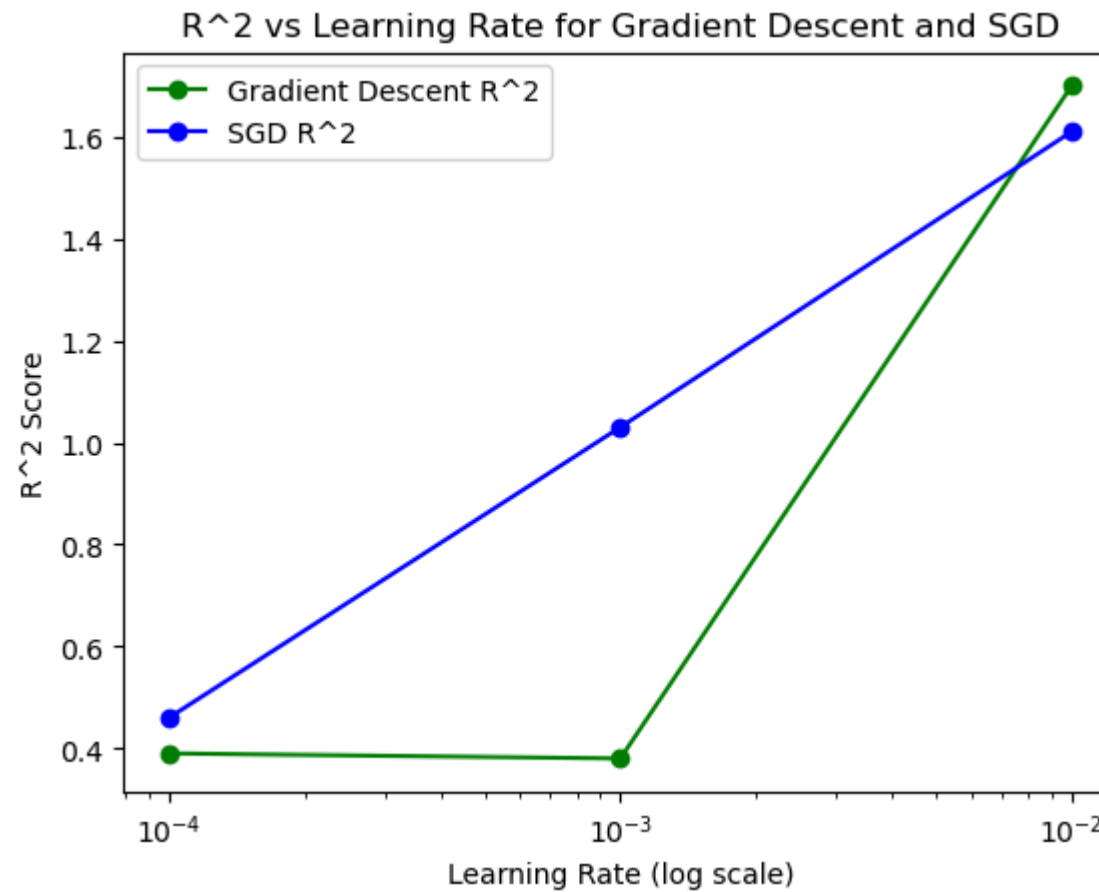
# Plotting R^2 values
plt.plot(learning_rates, gd_r2, color='g', label='Gradient Descent R^2', marker='o')
plt.plot(learning_rates, sgd_r2, color='b', label='SGD R^2', marker='o')

# Set the x-axis to a logarithmic scale
plt.xscale('log')

# Add labels and legend
plt.xlabel('Learning Rate (log scale)')
plt.ylabel('R^2 Score')
plt.legend(loc='best')

# Display the R^2 plot
plt.title('R^2 vs Learning Rate for Gradient Descent and SGD')
plt.show()
```





```
In [101... import matplotlib.pyplot as plt

# Updated data from the new table
learning_rates = [0.0001, 0.001, 0.01] # The different learning rates
lambda_values = [0.001, 0.01, 0.1]      # The corresponding lambda values

# RMSE and R^2 values for Gradient Descent
gd_rmse = [138.7, 129.02, 79.6]
gd_r2 = [0.40, 0.52, 1.45]

# RMSE and R^2 values for Stochastic Gradient Descent
sgd_rmse = [138.7, 129.4, 79.82]
sgd_r2 = [0.41, 0.52, 1.45]
```

```
# Plotting RMSE values
plt.plot(learning_rates, gd_rmse, color='g', label='Gradient Descent RMSE', marker='o')
plt.plot(learning_rates, sgd_rmse, color='b', label='SGD RMSE', marker='o')

# Set the x-axis to a logarithmic scale
plt.xscale('log')

# Add labels and legend
plt.xlabel('Learning Rate (log scale)')
plt.ylabel('RMSE')
plt.legend(loc='best')

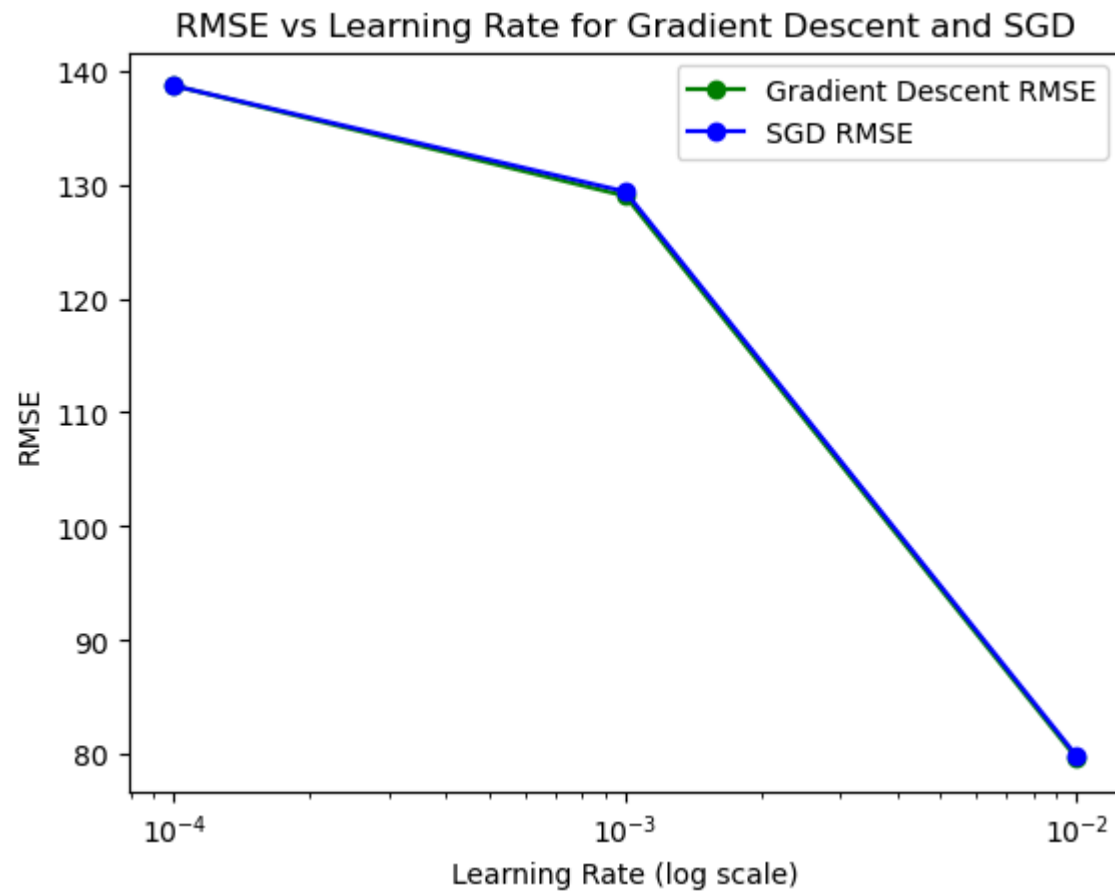
# Display the RMSE plot
plt.title('RMSE vs Learning Rate for Gradient Descent and SGD')
plt.show()

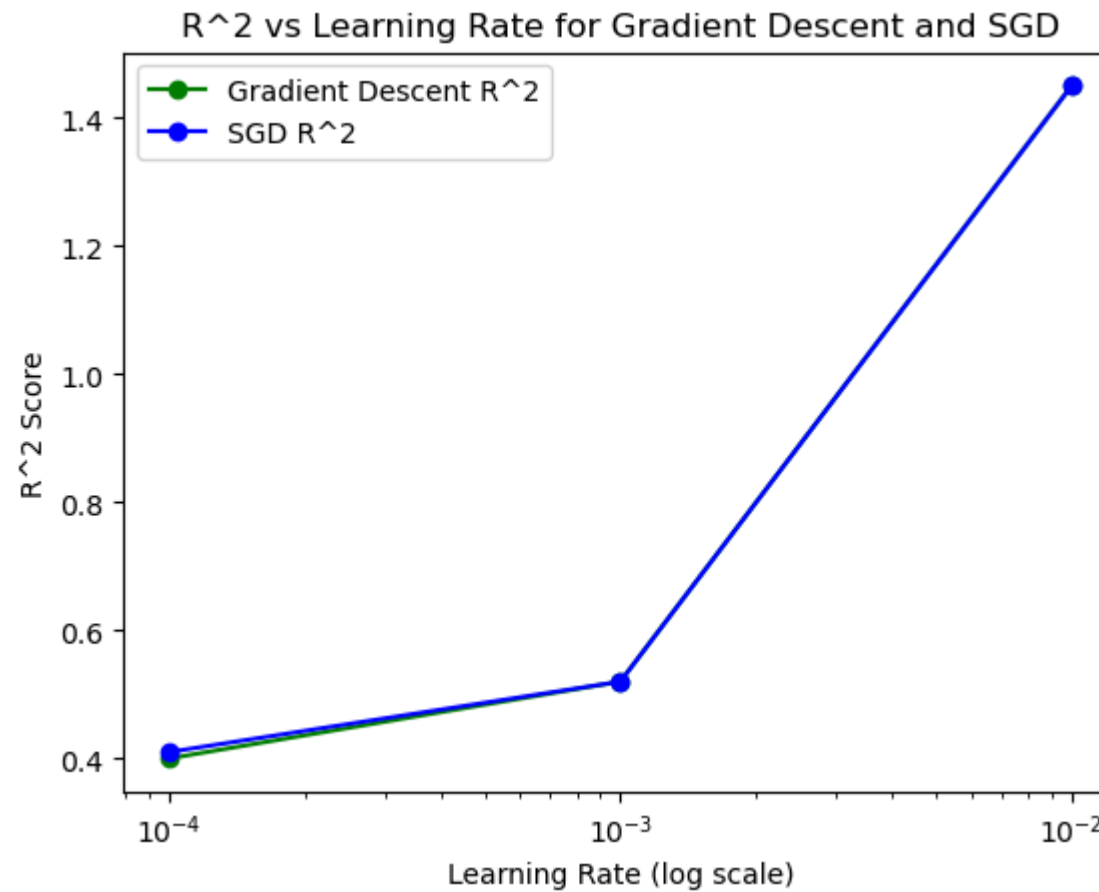
# Plotting R^2 values
plt.plot(learning_rates, gd_r2, color='g', label='Gradient Descent R^2', marker='o')
plt.plot(learning_rates, sgd_r2, color='b', label='SGD R^2', marker='o')

# Set the x-axis to a logarithmic scale
plt.xscale('log')

# Add labels and legend
plt.xlabel('Learning Rate (log scale)')
plt.ylabel('R^2 Score')
plt.legend(loc='best')

# Display the R^2 plot
plt.title('R^2 vs Learning Rate for Gradient Descent and SGD')
plt.show()
```





```
In [102... import matplotlib.pyplot as plt
import numpy as np

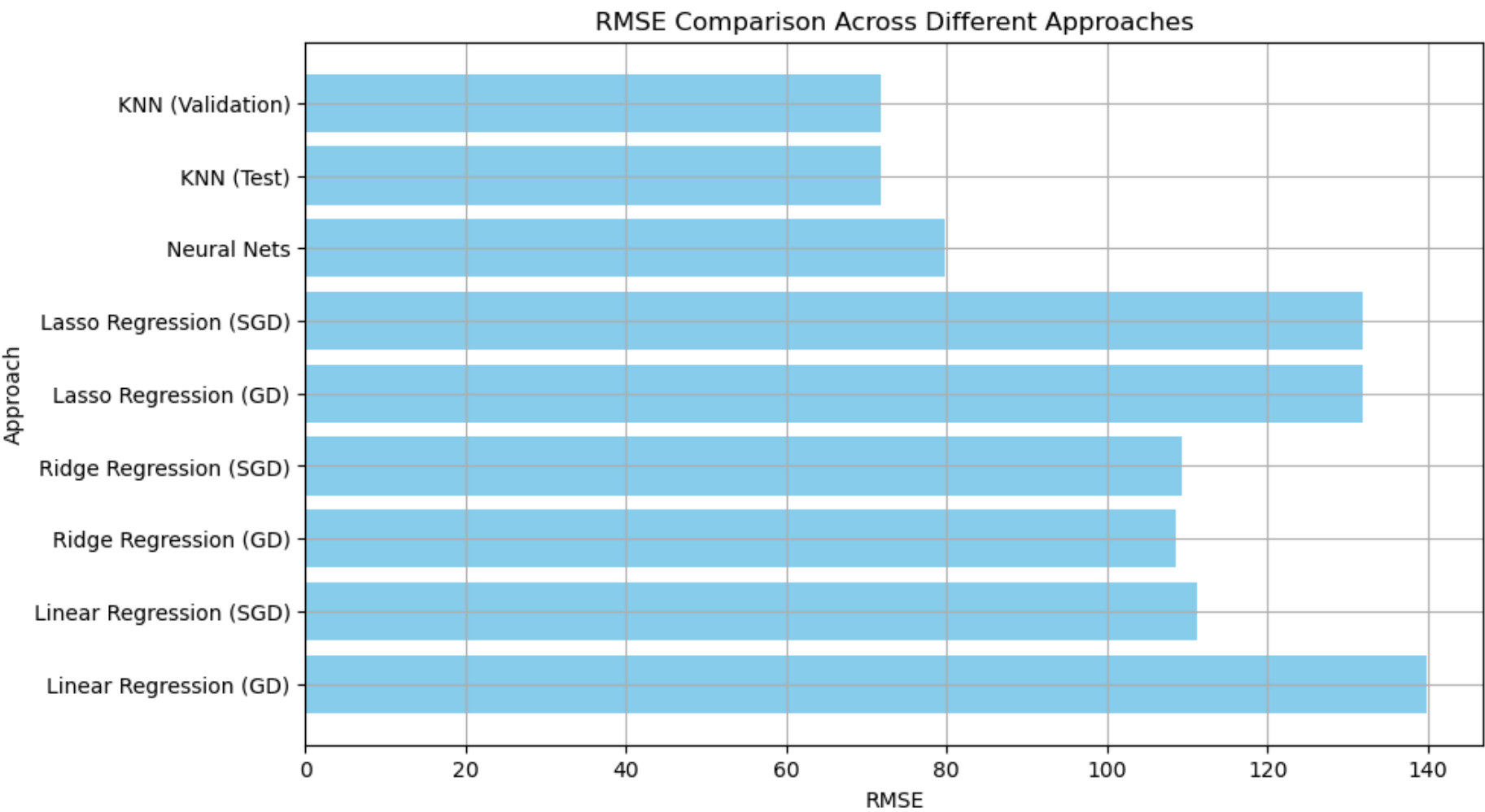
# Approaches and corresponding RMSE and R2 values
approaches = [
    'Linear Regression (GD)',
    'Linear Regression (SGD)',
    'Ridge Regression (GD)',
    'Ridge Regression (SGD)',
    'Lasso Regression (GD)',
    'Lasso Regression (SGD)',
    'Neural Nets',
    'KNN (Test)',
    'KNN (Validation)']
```

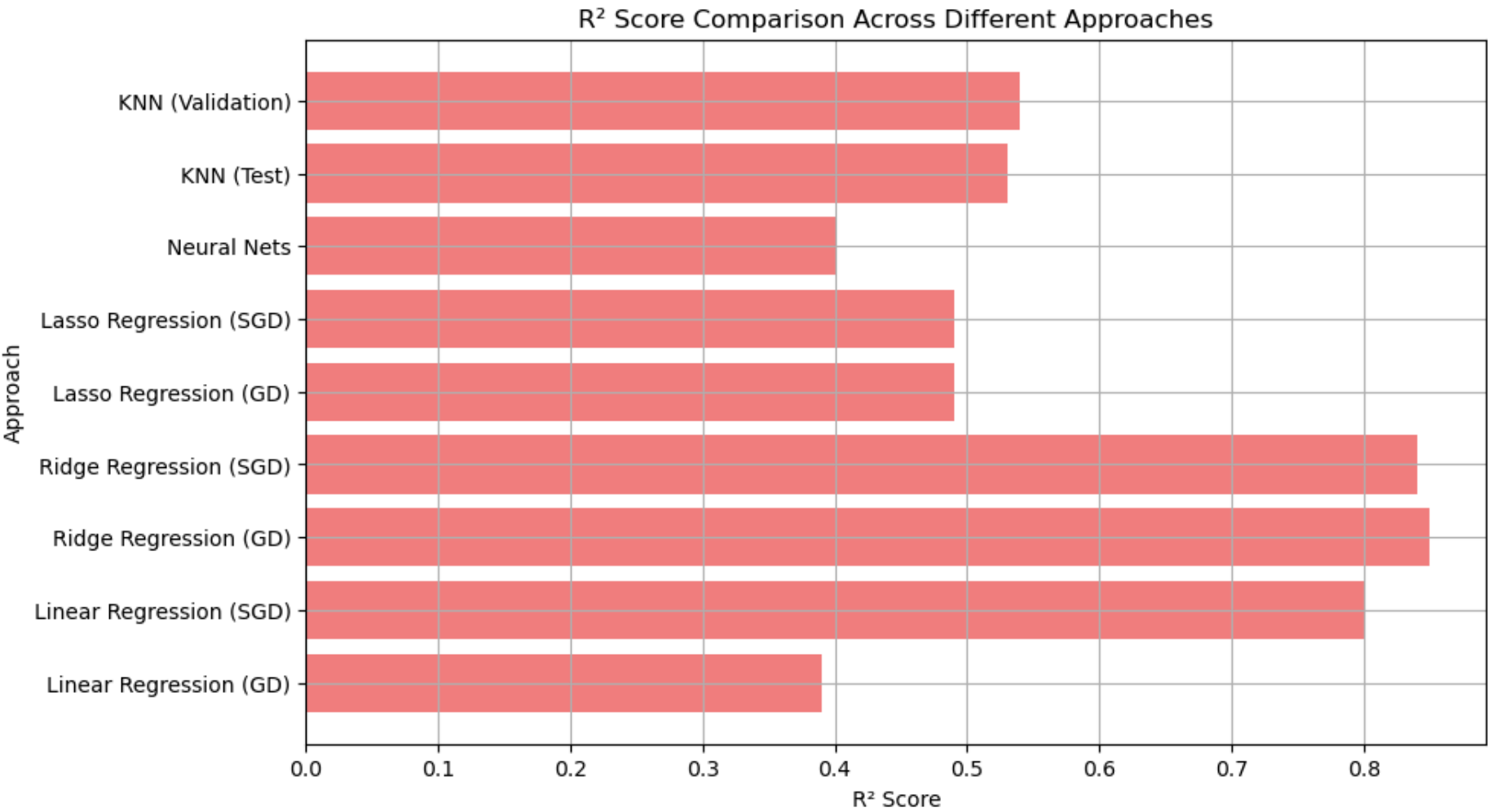
```
]

rmse_values = [139.9, 111.2, 108.52, 109.27, 131.83, 131.89, 79.73, 71.69, 71.78]
r2_values = [0.39, 0.80, 0.85, 0.84, 0.49, 0.49, 0.40, 0.53, 0.54]

# Plot for RMSE
plt.figure(figsize=(10, 6))
plt.barh(approaches, rmse_values, color='skyblue')
plt.xlabel('RMSE')
plt.ylabel('Approach')
plt.title('RMSE Comparison Across Different Approaches')
plt.grid(True)
plt.show()

# Plot for R2 Score
plt.figure(figsize=(10, 6))
plt.barh(approaches, r2_values, color='lightcoral')
plt.xlabel('R2 Score')
plt.ylabel('Approach')
plt.title('R2 Score Comparison Across Different Approaches')
plt.grid(True)
plt.show()
```



```
In [ ]:
```